

OpenDDA Reference Manual

DD27-200

OpenDDA Reference Manual

**DD27-200
Release 200
12/95**

Copyright, Notices, and Trademarks

Printed in U.S.A. – © Copyright 1995 by Honeywell Inc.

Revision 01 – October 15, 1996

While this information is presented in good faith and believed to be accurate, Honeywell disclaims the implied warranties of merchantability and fitness for a particular purpose and makes no express warranties except as may be stated in its written agreement with and for its customer.

In no event is Honeywell liable to anyone for any indirect, special or consequential damages. The information and specifications in this document ARE subject to change without notice.

TotalPlant and TDC 3000 are registered trademarks of Honeywell Inc.

Honeywell
Industrial Automation and Control
Automation College
2820 West Kelton Lane
Phoenix, AZ 85023

(602) 313-5669

Table of Contents

SECTION 1—USING THIS MANUAL	1
1.1 Overview	1
Introduction	1
What's in this section	1
1.2 About This Document	2
Purpose	2
Organization of this manual	2
Information covered in this manual	2
References for OpenDDA application types	2
Troubleshooting OpenDDA	2
1.3 Conventions Used in This Manual	3
Typed/Displayed text.....	3
Upper/lowercase letters	3
Optional parameters indicators	3
Applications.....	3
1.4 Overview of the Sections in This Manual	4
Document sections and descriptions	4
SECTION 2—INPUT/OUTPUT	5
2.1 Overview	5
Introduction	5
2.2 Terminology	5
Depopulate HLL	5
HLL.....	5
Input/Output	5
Populate HLL	5
Read Data.....	5
Write Data	5
2.3 OpenDDA Input/Output	6
Input/Output	6
Input/Output processing	6
Data Access Sets.....	6
Dynamic Selection Sets	7
The ALL Data Sets.....	7
Reference.....	7
SECTION 3—EXTERNAL DATA	9
3.1 Overview	9
Defining external data	9
What is an external data block?	9
Components of an External Data Block (EDB)	9
EDB location	9
3.2 External Data Block Format	10
Format of the EDB	10
Format of the External Data Statement.....	10
EDB comments and comment characters.....	11

Table of Contents

3.3	EDB Rules	12
	Basic EDB rules	12
	EDB Keywords	13
3.4	HLL-Specific Syntax	14
	Introduction	14
	FORTRAN comment characters	14
	C comment characters	14
	C++ comment characters	14
	Fortran array identifiers	14
	C/C++ array identifiers	14
	Fortran array range value delimiters	14
	C/C++ array range value delimiters	14
		14
3.5	EDB Data Types	15
	Using EDB data types	15
	EDB data types	16
	Using OpenDDA types	17
	Default EDB data types	17
	Rules	17
	POINT_ID and EXTERNAL_ID value types	18
	DDA_POINT_ID_TYPE	18
	Writing External IDs in C	18
	String truncation	18
3.6	EXTDEF Directive	19
	Definition	19
	Format	19
	Changing the default access name	19
	Default value	20
	Rules	20
3.7	Inconsistency Error	21
	What is an inconsistency error	21
	What happens when an inconsistency error occurs	21
	User action for RETURN	21
	Example	22
	Inconsistency error dump information	23
	Example inconsistency error dump	24
3.8	OpenDDA Sets	25
	Two types of sets	25
	Relationship of sets	25
	EDB Example	26
	Table of set assignments	26
	The 'ALL' selection set	26
	What is a Data Access Set	27
	What is a Dynamic Selection Set	27
	Maximum sets allowed	27

Table of Contents

3.9	Run Time Selection of External Data	28
	Introduction	28
	Remapping CIO concept.....	28
	The remapping tools	28
	Dynamic Selection Sets concept.....	28
	Dynamic Selection Sets tools.....	29
	References.....	30
SECTION 4—EDB DATA ACCESS VARIABLES		31
4.1	Overview	31
	Introduction	31
	What is included	31
4.2	Value Variable	32
	Definition	32
	Rules	32
	Determine Value Variable data type	32
	Variables and data type declarations	32
	When Value Variables are strings	33
	How to use Value Variable arrays	33
	Subrange specification	33
	Array subrange statement format.....	33
	Array index difference between FORTRAN and C.....	33
	When Value Variables are arrays	34
	Array examples	35
	Using multi-dimensional Value Variable arrays.....	36
	How to read array values	37
4.3	Test Values	38
	Definition	38
	Rules	38
	Test Value data types	38
	Accessing CDS parameters of type logical	39
	Test data	39
	Using arrays of Test Values	39
	If not enough Test Values to fill an array.....	39
4.4	Value Variable Options	40
	Options.....	40
	/data_type	40
	/TEST_VALUE and /NOTEST_VALUE qualifier	40
	Default behavior	40
	Overriding the EDB	41
4.5	Input/Output Qualifiers	42
	Definition	42
	INPUT keyword	42
	OUTPUT keyword.....	42
	If a variable is both input and output	42
	If neither INPUT nor OUTPUT is specified.....	43
	If INPUT and OUTPUT are specified	43
	If only one Location Specification is specified.....	43

Table of Contents

4.6	Location Specification	44
	Definition	44
	Format.....	44
4.7	CURRENT Location Specification	45
	Format.....	45
	access_name	45
	Default access_name.....	45
	ng\	45
	point.parameter	45
	index.....	45
4.8	NOX Location Specification	46
	Definition	46
	Function	46
	Format.....	46
	Syntax	46
	Initial value	46
	Uses	47
	Dynamic selection caution	47
	Discussion of NOX demo program.....	48
	NOX demo program	49
4.9	LCN Data	53
	TDC 3000 ^x database.....	53
4.10	Ancillary Variable Summary	54
	Using ancillary EDB variables	54
	Ancillary variable syntax.....	54
	Ancillary EDB variables.....	55
4.11	Value Status Variable	56
	Accessing Value Status Variables	56
	Value Status values	56
	Declaring Value Status Variables.....	56
	Using Status Values.....	56
4.12	Store Code Variable	57
	Store Code Variables	57
	Store code constants	57
	Declaring Store Code Variables.....	57
	Platform restrictions	58
	Store Code Defaults.....	58
4.13	Store Status Variable	59
	Accessing Store Status Variables	59
	Store Status values	59
	Declaring store status variables	59
	Using Status Values.....	59
4.14	Data Access Time Variable	60
	Accessing Data Access Time Variables.....	60
	Declaring Data Access Time Variables.....	61
	DDA_TIME_TYPE.....	61

Table of Contents

4.15	Test Value Variable	62
	Accessing Test Value Variables.....	62
	Declaring Test Value Variables.....	62
4.16	SET_LIST Directive	63
	Definition	63
	SET_LIST syntax	63
	SET_LIST rules.....	63
	Dynamic Selection Set assignment.....	63
		63
SECTION 5—EDB DYNAMIC SELECTION VARIABLES		65
5.1	Overview	65
	Introduction	65
	What is included.....	65
	Alias Variables and Dynamic Selection.....	65
5.2	Alias Variable Statement	66
	Overview	66
	Alias Variable statement syntax	66
	alias_variable description.....	66
	Declaring Alias Variables	66
	external_ref_	
	keyword definition	67
	Dynamic Selection Sets	67
	Maximum number of sets.....	67
	Rules.....	67
	Examples	68
5.3	Declaring High-Level Language Alias Variables	69
	Alias Variable type and size.....	69
	OpenDDA types supplied.....	69
	C support provided.....	69
	C example	69
	FORTRAN support provided	70
	FORTRAN example	70
5.4	Using Dynamic Selection Indicators	71
	Overview	71
	Example	71
	Rules.....	71
	Combinations of indicators.....	72
	NULL External Reference Components.....	72
	NULL vs. DDA_NULL.....	73
	Initialization of dynamic components	73
	Additional examples	74

Table of Contents

5.5 Alias Variable Arrays	75
Overview	75
Rules, properties, and relationships.....	75
Using an Alias Variable array.....	76
Using an Alias Variable that is not an array	77
Declaring full name arrays	77
Examples	78
Examples continued.....	79
5.6 Loading Alias Variables	80
Overview	80
External Reference components are strings.....	80
C examples	80
Loading Alias Variables that include a Network Gateway (NG) name in C.....	80
FORTRAN examples	81
Loading DDA_NULL.....	81
Saving External References.....	81
5.7 Indexes and Dynamic Selection	82
Overview	82
System behavior factors.....	82
Table of combinations.....	83
Example 1	84
Example 2	84
Example 3	84
Example 4	84
Important consideration.....	85
Example 5	85
5.8 External Reference Status Variables	86
Overview	86
Syntax	86
Rules.....	86
Declaring External Reference Status Variables	86
External Reference Status values.....	86
Examples	87
Examples continued.....	88
5.9 Modify Code Variables	89
Overview	89
EDB Syntax.....	89
Rules.....	89
Validation of Modify Codes	89
Default.....	89
Declaring Modify Code Variables.....	90
Examples	90
.....	90

Table of Contents

SECTION 6—OPENDDA EXECUTION STATEMENTS	91
6.1 Overview	91
Introduction	91
OpenDDA precompiler	91
Advantages of OpenDDA Execution Statements.....	91
Print options	91
6.2 OpenDDA Execution Statement Syntax	92
Elements of an EXEC DDA statement	92
Required elements	92
Order	93
Example	93
Character strings.....	93
Brackets used in examples	93
6.3 Include File	94
Description	94
Referencing the Include file.....	94
FORTRAN include file example	95
6.4 EXEC Statement Summary	96
Commands.....	96
Qualifiers	96
Keywords	96
Summary of statements	96
CLEAR CIO.....	96
HIBERNATE EVENT.....	97
INITIALIZE	97
LOAD ALIAS	97
LOAD MODIFY_CODE	97
LOAD STORE_CODE.....	97
MODIFY CIO	97
READ	97
READ CIO.....	97
SELECT CIO	97
TERMINATE	97
WRITE.....	97
WRITE CIO	97
6.5 Summary of EXEC DDA Variables	98
Purpose of the variables	98
Summary of Execution Statement variables	98
6.6 EXEC DDA Statement Notation for this Section	99
EXEC definition notation	99
DDA usage.....	99
Type	99
DDA data type.....	99
Access.....	99
Purpose.....	99
Using the Status Mnemonic	99

Table of Contents

6.7	CLEAR CIO Statement	100
	Description	100
	Format.....	100
	[ID=cio_id]	100
	STATUS= exec_status.....	101
	Declaring exec_status variables	101
	Example	101
6.8	HIBERNATE EVENT Statement	102
	Description	102
	Format.....	102
	EVENT_DATA=	
	evt_data	102
	OpenDDA types supplied.....	103
	STATUS= exec_status.....	104
	Declaring exec_status variables	104
	Event Status values	105
	Example	105
6.9	INITIALIZE Statement	107
	Description	107
	What INITIALIZE does	107
	Location.....	107
	Format.....	107
	Development Directory.....	107
	STATUS= exec_status.....	108
	Declaring exec_status variables	109
	Example	110
6.10	LOAD ALIAS Statement	111
	Description	111
	Format.....	111
	SET=	111
	If selection_set_	
	name is undefined.....	111
	STATUS= exec_status.....	112
	Declaring exec_status variables	112
	Example	113
		113
6.11	LOAD MODIFY_CODE Statement	114
	Description	114
	Format.....	114
	VALUE=	
	modify_code.....	114
	Function	114
	STATUS= exec_status.....	115
	Declaring exec_status variables	115
	Comparison to other LOAD qualifiers	115
	Examples	116

Table of Contents

6.12	LOAD STORE_CODE Statement	117
	Description	117
	Format.....	117
	SET=set_name	117
	If set_name is undefined	117
	STATUS= exec_status.....	118
	Declaring exec_status variables	118
	What the LOAD STORE_CODE statement does.....	118
	Memory resident CIO file store codes.....	119
	When to use the LOAD STORE_CODE statement.....	119
	Example	120
6.13	MODIFY CIO Statement	121
	Description	121
	Data access priority.....	121
	Format.....	121
	SET=	121
	If selection_set_name is undefined.....	121
	STATUS= exec_status.....	122
	Declaring exec_status variables	123
	MODIFY CIO operation.....	123
	If the new External References produce errors.....	124
	Effect of MODIFY CIO errors at data access time	124
	Effect of NULL External References at data access time	124
	Incremental changes.....	125
	External Reference Status values returned	126
	Examples	128
6.14	READ Statement	129
	Description	129
	Data access priority.....	129
	Format.....	129
	SET=set_name	129
	If set_name is undefined	129
	STATUS= exec_status.....	130
	Declaring exec_status variables	131
	Value Status values returned	132
	Example	133
6.15	READ CIO Statement	134
	Description	134
	CIO Signature validation	134
	Format.....	134
	[FILE=file_name].....	134
	[ID=cio_id]	134
	STATUS= exec_status.....	135
	Declaring exec_status variables	136
	READ CIO usage illustrations	137
	Example	137

Table of Contents

6.16	SELECT CIO Statement	138
	Description	138
	Format.....	138
	[ID=cio_id]	138
	STATUS= exec_status.....	139
	Declaring exec_status variables	139
	Impact on HLL variables	139
	Example	140
6.17	TERMINATE Statement	141
	Description	141
	What TERMINATE does	141
	Location.....	141
	Format.....	141
	STATUS= exec_status.....	141
	Declaring exec_status variables	142
	Example	142
6.18	WRITE Statement	143
	Description	143
	Data access priority.....	143
	Controlling writing of elements of a set	143
	Format.....	143
	SET=set_name	143
	If set_name is undefined	144
	STATUS= exec_status.....	144
	Declaring exec_status variables	145
	Store Status values returned.....	146
	Example	148
	Writing External IDs in C	149
6.19	WRITE CIO Statement	150
	Description	150
	CIO Signature unchanged.....	150
	Format.....	150
	[FILE=file_name]	150
	[ID=cio_id]	150
	[OVERWRITE=	
	overwrite_option].....	151
	STATUS= exec_status.....	151
	Declaring exec_status variables	152
	Example	152
	WRITE CIO usage illustrations	153
	OVERWRITE option usage illustrations.....	154
6.20	OpenDDA exec_status	155
	exec_status.....	155
	User-specified status variable	155
	Declaring exec_status variables	155
	Attributes	155
	Good Status	155
	Determining status	156
	exec_status generation	157

Table of Contents

SECTION 7—DEFINITION (DEF) FILE	159
7.1 Overview	159
Definition (DEF) file description.....	159
What the DEF file does	159
Directives are optional.....	159
DEF file is not required.....	160
7.2 DEF File	161
Creating a DEF file	161
Syntax statements.....	161
Rules	161
Duplicate statements.....	161
Checking the DEF file for errors.....	161
Errors and warnings in DEF files.....	162
7.3 DEF File Format	163
Format of DEF file	163
Statements no longer supported	163
7.4 System Default DEF File	164
Customizing your system default DEF file	164
7.5 DEF File Example	165
DEF file example.....	165
7.6 DATA_PRIORITY Directive	166
Definition	166
Syntax	166
Keywords	166
No LOW DA Servers Configured	166
Default.....	166
Example:	166
7.7 INITIAL_VALUES Directive	167
Definition	167
Syntax	167
Qualifiers	167
Runtime behavior	167
STORE_CODE= [STORE NOSTORE]	167
7.8 CIO_SIGNATURE Directive	168
Definition	168
Syntax	168
Keywords	168
Default.....	168
Example	168
7.9 INCONSISTENCY Directive	169
Definition	169
Syntax	169
Keywords	169
Default.....	169
Example	169

Table of Contents

7.10	PRINT Directive	170
	Definition	170
	Syntax	170
	Keywords	170
	Default.....	170
	Example	170
	MESSAGES=[ON OFF YES NO TRUE FALSE].....	171
	STATS=[ON OFF YES NO TRUE FALSE]	172
	VALUES=[ON OFF YES NO TRUE FALSE].....	173
	PRINT output example.....	177
SECTION 8—FORTRAN APPLICATIONS		179
8.1	Overview	179
	Introduction	179
	In this section	179
	Using FORTRAN programs	179
	FORTRAN Include file.....	179
	OpenDDA types supplied.....	179
	Sample FORTRAN applications.....	179
8.2	OpenDDA FORTRAN Template	180
	Sample FORTRAN template.....	180
	FORTRAN EDB syntax	181
	Reference.....	181
8.3	DEF File Template	182
	DEF template file.....	182
	Same template for FORTRAN and C.....	182
	Changing the DEF template file	182
8.4	FORTRAN Application Example	183
	Application example	183
8.5	FORTRAN External Data Block Example	186
	EDB example	186
8.6	Examples of Assigning Values to FORTRAN Variables	188
	Examples	188
SECTION 9—C APPLICATIONS		189
9.1	Overview	189
	Introduction	189
	In this section	189
	Using C programs	189
	Using a C++ compiler.....	189
	C Include file	189
	OpenDDA types supplied.....	189
	Sample C applications	189
9.2	OpenDDA C Template	190
	Sample C template.....	190
	C EDB syntax.....	191
	Reference.....	191

Table of Contents

9.3	DEF File Template	192
	DEF template file.....	192
	Same template for FORTRAN and C.....	192
	Changing the DEF template file	192
9.4	C Application Example	193
	Application example	193
9.5	C External Data Block Example	196
	EDB example	196
9.6	Examples of Assigning Values to C Variables	198
	Examples	198
SECTION 10—TROUBLESHOOTING		199
10.1	Overview	199
	Introduction	199
	The General Output file.....	199
	Access the current OpenDDA version	199
	When reporting a problem.....	199
	Application execution produces a core dump	199
10.2	Known and/or Suspected Problems	200
	Array anomaly	200
	Application names conflict with UNIX file names	200
10.3	Problem Symptoms and Possible Actions	201
	PRINT VALUES shows status %HGSC-E-NULL_VALUE	201
	Consistency error	201
	Privilege problem installing or executing.....	201
	Errors returned by CL initiation	202
	No output file and bad status in CL block.....	202
	Application won't write values	203
	Can't access DDA executable.....	203
	EDB parsing error	203
	Erroneous data.....	203
	Source file not found	204
10.4	Error Message Troubleshooting	205
	"No .cio file"	205
	HGSC_E_ BADFLDCMB	205
GLOSSARY		207

Table of Contents

Section 1—Using This Manual

1.1 Overview

Introduction

This manual covers the definition and syntax of the various parts of an Open DDA application and how to

- define external data
- create OpenDDA execution statements
- create a Definition (DEF) file, use FORTRAN or C/C++ with your OpenDDA applications,
- incorporate either of the two OpenDDA techniques to select External Data References at run time:
 - remap CIO files, and
 - use Dynamic Selection Sets.

What's in this section

This section contains

- a review of this manual's organization,
- the conventions used, and
- a description of the sections and appendices of this manual.

1.2 About This Document

Purpose	The purpose of this manual is to define the OpenDDA-specific elements of your applications. The definitions and syntax for each part of the External Data Block (EDB), OpenDDA Execution Statements, and the Definition (DEF) file are explained.
Organization of this manual	The sections and topics of this manual are arranged to lead you through the various parts of an OpenDDA application and how to properly construct and apply them. For your convenience, a glossary and an index are located at the end of the manual.
Information covered in this manual	This manual reviews the application-specific information required to support an OpenDDA application. For the basics of OpenDDA, refer to the <i>OpenDDA User's Guide</i> .
References for OpenDDA application types	This manual reviews the application-specific conventions required by OpenDDA to support FORTRAN and C/C++ applications, but does not provide reference material about these programming languages. You may need one or more of the following Hewlett-Packard manuals, available on CD-ROM, depending on the functions to be included in your applications: • <i>HP FORTRAN/9000 Programmer's Guide</i> This manual is a general user's guide for a FORTRAN user. • <i>HP FORTRAN/9000 Programmer's Reference</i> This manual is a detailed reference for a FORTRAN user. • <i>HP C Programmer's Guide</i> • <i>HP C/HP-UX Reference Manual</i> • <i>HP C++ Programmer's Guide</i> • <i>C++ Developer User's Guide</i>
Troubleshooting OpenDDA	For assistance in troubleshooting OpenDDA, refer to Section 10—"Troubleshooting" starting on page 199.

1.3 Conventions Used in This Manual

Typed/Displayed text Information messages displayed by OpenDDA, or information entered by a user, is indicated by this typeface:

```
EXEC DDA INITIALIZE STATUS = exec_status;  
(example of user-entered application information)  
  
Begin processing EDB...  
DDA-S, EDB processing completed successfully.  
(example of OpenDDA information message)
```

Upper/lowercase letters In examples for your OpenDDA applications, uppercase text is used to indicate information items as they must be entered (usually keywords):

```
C  EXTERNAL DATA BEGIN
```

Lowercase text is used to indicate information items whose names you can choose:

```
application_name
```

The upper/lowercase format does NOT apply to command line instructions. Enter command line instructions case EXACTLY as they appear.

ATTENTION

OpenDDA constructs are not case-sensitive. Either upper or lowercase may be used.

OpenDDA command line entries **are case-sensitive**, refer to 5.2 and 5.3 of the *OpenDDA User's Guide*.

Optional parameters indicators Bracket symbols [] are used in some examples to indicate optional parameters.

Example:

```
value_variable = test_value [/data_type] [/[NO]TEST_VALUE]
```

Applications Throughout this manual, the term “application” refers to applications created using OpenDDA, unless otherwise stated.

1.4 Overview of the Sections in This Manual

Document sections and descriptions

The following table contains a brief description of the sections in this manual.

Table 1-1 OpenDDA Reference Manual Sections

Section Name	Description
Using This Manual	The section you are reading now. It explains the conventions and organization of the OpenDDA Reference Manual, this document.
Input and Output	Describes the behavior of Input/Output to/from a DCS in OpenDDA applications.
External Data	Explains how you define the external data for your OpenDDA applications in an OpenDDA application External Data Block (EDB).
EDB Data Access Variables	In this section, you learn about the EDB Value Variable, the native language variable that OpenDDA will load your input data into and/or where OpenDDA will retrieve values to output.
Dynamic Selection Variables	In this section, you learn about the EDB Alias Variable, the native language variable that your application loads with the name of an External Reference component. The Alias Variable is part of the Dynamic Selection Sets functionality that allows you to select External References at run time.
OpenDDA Execution Statements	OpenDDA Execution Statements allow you to specify actions within your OpenDDA applications. This section covers the syntax and provides examples of how to embed Execution Statements in your OpenDDA applications.
Definition (DEF) File	Describes the Definition (DEF) file. The DEF file allows you to customize your OpenDDA applications by defining characteristics associated with the application when it executes. Reviews the use and function of the various DEF file directives and qualifiers.
FORTTRAN Applications	OpenDDA uses FORTRAN applications to simplify access to DCS data. This section discusses OpenDDA FORTRAN syntax and contains FORTRAN examples of a DEF file, an EDB and a sample application.
C/C++ Applications	OpenDDA uses C/C++ applications to simplify access to DCS data. This section discusses OpenDDA C/C++ syntax and contains examples of a DEF file, EDB, and a sample application.
Troubleshooting	Lists any OpenDDA known and/or suspected problems, and troubleshooting solutions.

Section 2—Input/Output

2.1 Overview

Introduction This section describes the behavior of Input/Output data to and from a distributed control system (DCS) in OpenDDA applications.

2.2 Terminology

Depopulate HLL Depopulate HLL—High-Level Language variables associated with the statement will be moved to OpenDDA in preparation for writing to an external location.

HLL HLL—High-Level Language (FORTRAN or C/C++) used to code the application.

Input/Output Input/Output—You can specify/control when I/O occurs to the data platform. HLL variables are populated/depopulated by OpenDDA-generated code within the body of the application.

Populate HLL Populate HLL—High-Level Language variables associated with the statement will be populated by OpenDDA with values from the external locations.

Read Data Read Data—All external data associated with the statement will be read into OpenDDA in preparation for populating the HLL variables.

Write Data Write Data—Values associated with the statement that were depopulated into OpenDDA will be written to the external locations.

2.3 OpenDDA Input/Output

Input/Output

The OpenDDA Input/Output functions read and write, directly accessing the external data. There is no intermediate database that holds the values.

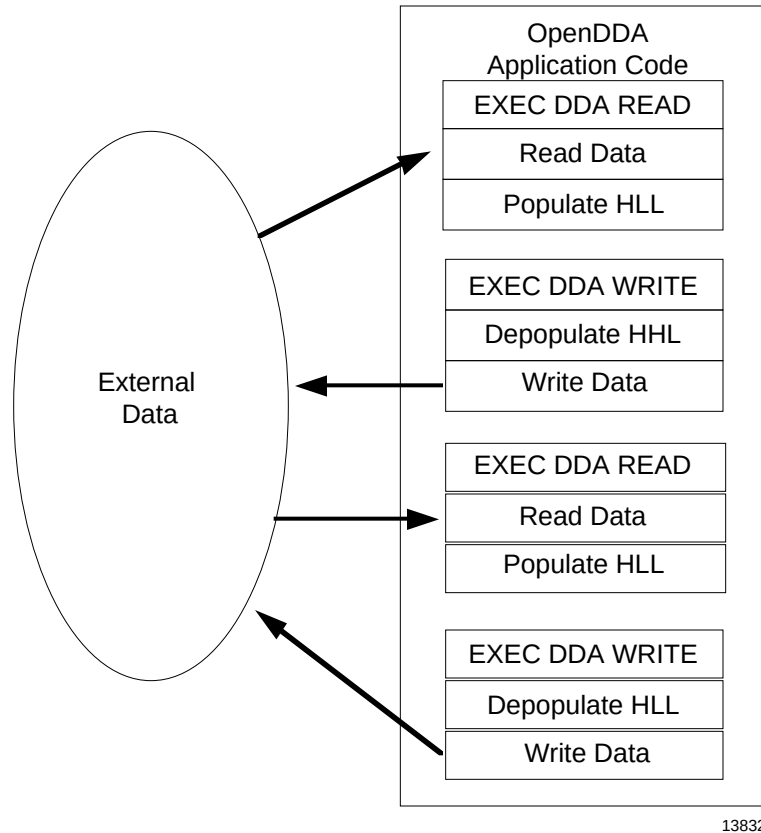


Figure 2-1 Input/Output Actions

Input/Output processing

Execution of the OpenDDA read function results in data fetched from the external locations and the moving of that data into the HLL (High-Level Language) variables.

Execution of the OpenDDA write function results in moving data from the HLL variables and storing that data to the external locations.

Data Access Sets

OpenDDA reads and writes data that is defined in an individual Data Access Set on request of the user.

Data Access Sets are defined by specifying SET_LIST in the External Data Block (EDB).

Refer to 4.16 on page 63 for a description of the SET_LIST directive.

Example:

```
EXEC DDA READ SET='test_data'
```

Data Access Sets provide the flexibility to read and write subsets of all the data, and the ability to reread and rewrite.

Dynamic Selection Sets

OpenDDA provides Dynamic Selection Sets to determine:

- which External Reference components will be modified by the EXEC DDA MODIFY CIO statement, and
- the Alias Variables that will specify the names of the External Reference components that will be dynamically selected.

Dynamic Selection Sets are defined by specifying SET_LIST in the External Data Block (EDB). All Alias Variables defined in an External Data Statement will be included in a Dynamic Selection Set or Sets named in the SET_LIST directive.

Refer to 4.16 on page 63 for a description of the SET_LIST directive.

The ALL Data Sets

OpenDDA provides the ALL Data Access Set for Input/Output and the ALL Dynamic Selection Set for Dynamic Selection. The ALL Data Access Set consists of all of the EDB data references for the application. The ALL Dynamic Selection Set consists of all EDB Alias Variables for the application.

Example:

```
EXEC DDA READ SET='ALL'
```

Reference

Refer to subsection 3.8 for a general discussion of OpenDDA sets.

Input/Output

2.3 OpenDDA Input/Output

Section 3—External Data

3.1 Overview

Defining external data	You define the external data for your OpenDDA application in your application's External Data Block (EDB).
What is an external data block?	An external data block (EDB) is a special section of your application source code that is required for each OpenDDA application. The EDB is processed by OpenDDA and used to establish a relationship between your native language application variables and the external data values.
Components of an External Data Block (EDB)	The External Data Block (EDB) describes the association among application variables, data points, data status variables, and other ancillary information. The components of an EDB are • HLL (High-Level Language) variable names for data values, statuses, alias components, and ancillary information • External location specifications • Test values • EDB keywords EDB keywords are reserved and may be used only as keywords. Refer to Table 3-1 "EDB Keywords" on page 13.
EDB location	The EDB can be included in your application source file or it can be in a separate file. The EDB has required beginning and ending statements. It must be a commented section of your application source code, using the syntax for comments that is appropriate for your source language (FORTRAN or C). It must be in the form of comments whether it is included in the source file or is in a separate file.

3.2 External Data Block Format

Format of the EDB The EDB consists of one or more External Data Statements bounded by required beginning and ending statements and with optional EXTDEF directives. This format is listed below:

```
EXTERNAL DATA BEGIN
[EXTDEF ([access_name, symbolic_name])]
external_data_statement;
[external_data_statement; ...]
EXTERNAL DATA END
```

[] indicate optional elements

... indicates additional optional elements

Each External Data Statement *must be* terminated by a semicolon (;)

Format of the External Data Statement

The format of the External Data Statement is as follows:

```
value_variable = test_value [/data_type] [/[NO]TEST_VALUE]
[value_status_variable = INPUT [location_spec]]
[store_code_variable = OUTPUT [location_spec]]
[store_status_variable = STORE_STATUS]
[data_access_time_variable = TIMESTAMP]
[test_variable = TEST]
[alias_variable = external_ref_keyword]
[external_ref_status_variable = EXT_REF_STATUS]
[modify_code_variable = MODIFY_CODE]
[SET_LIST = set_name_1[, set_name_2, ...]];
```

[] indicate optional elements

... indicates additional optional elements

EDB comments and comment characters

The EDB must be enclosed within comment characters that are appropriate for the application HLL (high-level language—FORTRAN, C, or C++) so that the HLL compiler will not attempt to compile the EDB block.

In addition, EDB comments (comments that will not be processed by the OpenDDA EDB parser) may be embedded anywhere within the EDB by bounding the comment text with the EDB comment character—the exclamation point (!). In FORTRAN, if the comment is not bounded with exclamations, the comment is terminated at the end of the EDB line. In C/C++, the EDB comment must be bounded with exclamations.

Example (FORTRAN source code):

```
C  EXTERNAL DATA BEGIN
C      !This procedure evaluates the flow ratio.
C  EXTDEF (AXM, UNIT1)
C  !value variable! FLOW = 12.3
C      FLOW_STATUS = INPUT (UNIT1, FC101.PV)
C      FLOW_STORE_CODE = OUTPUT
C      FLOW_STORE_STATUS = STORE_STATUS
C      FLOW_TIME = TIMESTAMP;
C  EXTERNAL DATA END
```

C/C++, the EDB comment must be bounded with exclamations.

Example (C source code):

```
/* EXTERNAL DATA BEGIN
    !This procedure evaluates
    the flow ratio.!
EXTDEF (AXM, UNIT1)
!value variable! FLOW = 12.3
    FLOW_STATUS = INPUT (UNIT1, FC101.PV)
    FLOW_STORE_CODE = OUTPUT
    FLOW_STORE_STATUS = STORE_STATUS
    FLOW_TIME = TIMESTAMP;
EXTERNAL DATA END    */
```

3.3 EDB Rules

Basic EDB rules

Some basic rules governing the EDB are listed below:

- An EDB is required for an OpenDDA application. **It must have at least one value**, even if it is a dummy value in test mode.
- An EDB is bounded by the statements
 - `EXTERNAL DATA BEGIN`
(identifies the initialization or beginning of the EDB)
 - `EXTERNAL DATA END`
(identifies the termination or end of the EDB)
- The EDB must be enclosed within comment characters that are appropriate for the specific high-level language (FORTRAN, C, or C++).
- EDB comments can be embedded anywhere within the EDB by bounding the comment text with exclamations (!).
- An EDB can be placed anywhere in an application.
- Only one EDB is permitted per file. Additional EDBs are ignored.
- Quoted strings within the EDB are identified by single quotes ('...')
- Quoted strings cannot continue to another line.
- Each keyword can be used only once per External Data Statement.

EDB Keywords

There are several reserved keywords used in OpenDDA application EDBs. *Do not* use these words to define your variables.

Table 3-1 EDB Keywords

Reserved Word	Note
AUTO_VAR	Reserved for future use
EXTDEF	An optional directive that allows you to define symbolic names for platform access names
SET_LIST	Used to assign a Value Variable to one or more Data Access Sets, and to one or more Dynamic Selection Sets if any components of the Value Variable are set up for Dynamic Data Selection

3.4 HLL-Specific Syntax

Introduction	This subsection contains EDB syntax that is dependent on the high-level language that you are using (FORTRAN, C, or C++).
FORTRAN comment characters	The acceptable EDB syntax for FORTRAN comment characters are any one of the following: • an uppercase or lowercase C, or • an asterisk (*), or • exclamation point (!) in the first column of each line in the EDB. The comment is always terminated at the end of the line.
C comment characters	The EDB syntax for C comment characters is • /* at the beginning of the EDB • */ at the end of the EDB
C++ comment characters	The EDB syntax for C++ comment characters is: • // in the first column of each line of the EDB, or • /* at the beginning and */ at the end of the EDB (same as for C)
Fortran array identifiers	The EDB syntax for FORTRAN array identifiers • use parentheses to enclose the array range Example: <code>array_variable(n:m)</code>
C/C++ array identifiers	The EDB syntax for C/C++ array identifiers • use square brackets to enclose the array range Example: <code>array_variable[n:m]</code>
Fortran array range value delimiters	The EDB syntax for FORTRAN array range value delimiters • use a colon to delimit the range values Example: <code>array_variable(n:m)</code>
C/C++ array range value delimiters	The EDB syntax for C/C++ array range value delimiters • use a colon to delimit the range values Example: <code>array_variable[n:m]</code>

3.5 EDB Data Types

Using EDB data types

OpenDDA allows access to several different types of external point data using EDB data types. They allow you to specify the type of data you expect to read/write to the location specified in your defined location specification.

ATTENTION

The following table shows the data types as they exist in the current implementation. However, for portability and migration considerations, Honeywell recommends that you declare your high-level language variables using constructs provided by OpenDDA in the include files `dda_c.h` and `dda_fortran.f`. Examples of the use of these constructs are provided in the topic "Using OpenDDA types" following the table.

EDB data types This table describes each of the available EDB data types.

Table 3-2 EDB Data Types

Data Type...	Allows access to a...	FORTRAN	C/C++
/Real	32-bit floating point value	Real*4	float
/Integer	4-byte integer value	Integer*4	long int
/Ordinal	4-byte integer which is an ordinal representation of an enumeration value	Integer*4	long int
/Enumerated	fixed-length ASCII string of 8 characters	Character*8	char[9]
/String	fixed-length ASCII string up to 40 characters	Character*40	char[41]
/Absolute	structure representing OpenDDA absolute time (Refer to Table 4-2 on page 38.)	Structure /DDA_TIME_TYPE/ (Refer to 4.14)	DDA_TIME_TYPE (Refer to 4.14)
/Delta	structure representing OpenDDA delta time (Refer to Table 4-2 on page 38)	Structure /DDA_TIME_TYPE/ (Refer to 4.14)	DDA_TIME_TYPE (Refer to 4.14)
/Point_Id	structure representing the LCN internal ID of a point (Refer to Table 4-2 on page 38)	Structure /DDA_POINT_ID_TYPE/	DDA_POINT_ID_TYPE
/External_Id	fixed-length ASCII string of 19 characters representing an LCN point name and LCN name	Character*19	char[20]

Using OpenDDA types

The following examples illustrate the use of the constructs provided by OpenDDA in the include files `dda_c.h` and `dda_fortran.f`. Note: the FORTRAN include file does not include size definitions for REAL or INTEGER types.

C Examples:

```
DDA_REAL_TYPE real_var;
DDA_INTEGER_TYPE integer_var;
DDA_ORDINAL_TYPE ordinal_var;
DDA_ENUMERATED_TYPE enumeration_var;
DDA_STRING_TYPE string_var;
DDA_TIME_TYPE time_var;
DDA_POINT_ID_TYPE point_id_var;
DDA_EXTERNAL_ID_TYPE external_id_var;
```

FORTRAN Examples:

```
CHARACTER*(DDA_ENUMERATED_SIZE) enumeration_var
CHARACTER*(DDA_STRING_SIZE) string_var
RECORD /DDA_TIME_TYPE/ time_var
RECORD /DDA_POINT_ID_TYPE/ point_id_var
CHARACTER*(DDA_EXTERNAL_ID_SIZE) external_id_var
```

Default EDB data types

If a `value_variable` does not have its data type specified, OpenDDA determines the data type from the `test_value` specified.

Table 3-3 Default EDB Data Types

Data Type	test_value
Real	contains a period '.' or exponent 'Ennn'
String	is enclosed in single quotes (' ')
Ordinal	anything else

Rules

Some EDB data type rules

- Each `value_variable` declared in an EDB should be followed by a data type declaration:
 - Example: `TEST_VALUE = 35/INTEGER`
- Each data type reference must be preceded by a "/" slash character.

POINT_ID and EXTERNAL_ID value types

When accessing POINT_ID and EXTERNAL_ID value types on your LCN with a defined Network Gateway, the NG name is attached to the value.

Therefore, if you read an EXTERNAL_ID you will see the name in the following format:

ng\point

DDA_POINT_ID _TYPE

DDA_POINT_ID_TYPE is a record structure that holds the internal identification of a point on the LCN.

The structure contains four 2-byte integers.

WARNING

The information in variables of this type are internal structures.

The individual fields should never need to be changed.

Writing External IDs in C

When using a C program to write out an entity name to variable of type Entity_Id, the value to be written is usually copied into the Value Variable using the strcpy function. If the string contains a backslash ('\') character, strcpy will treat it as an escape character. If the entity name has an NG name preceding the point name, then the NG name and point name must be separated by a double backslash ('\\'). This will then be treated as a single backslash internally.

Example:

To copy the entity name "SS\A100" into a Value Variable:

```
strcpy(External_Id_Variable, "SS\\A100");
```

NOTE: This is only true for C programs. FORTRAN programs use a single backslash.

String truncation

If the `value_variable` declared in your application's EDB for a STRING data type is less than 40 for FORTRAN or 41 for C, the value read from the LCN may be truncated to the size of the `value_variable`.

3.6 EXTDEF Directive

Definition EXTDEF is an optional directive that allows you to define symbolic names for platform access names.

ATTENTION

In Release 200 of OpenDDA, the only access_name supported is "AXM."
Differing symbolic_names may be used to position the application to take advantage of future capability.

Format

The format for the EXTDEF directive is

```
EXTDEF (access_name, symbolic_name [, access_name,
symbolic_name]...)
```

where

access_name = is the name assigned by the OpenDDA system installer to allow users to reference a data platform with a specific set of parameters.
Maximum of 16 characters

symbolic_name = is a name that the OpenDDA user can assign (using the EXTDEF directive) to associate an access name.
Maximum of 32 characters

Example:

```
EXTDEF (AXM, TOWER1)
EXTDEF (AXM, TOWER2)
.
.
flow rate = INPUT CURRENT(TOWER1, FC100.PV);
level = INPUT CURRENT(TOWER2, LC250.PV);
```

Changing the default access name

If the keyword DEFAULT is used for the symbolic_name, the EXTDEF directive defines the default access_name.

The new default access_name is used when defining the location_spec of an INPUT or OUTPUT variable if the access name is omitted.

Example:

```
EXTDEF (AXM, DEFAULT)
...
var1_sts = INPUT CURRENT(, FC101.PV);
```

This example defines the default access_name as AXM, and uses the default access_name in the INPUT location_spec.

Default value If the DEFAULT is not specified, "axm" will be used when the `access_name` is omitted.

Rules Use the following rules when specifying the EXTDEF directive.

- The EXTDEF directive can appear anywhere in the EDB as long as you define your `symbolic_name` before you use it in the `location_spec`.
- Multiple EXTDEF directives are allowed in an EDB.
- Maximum number of unique EXTDEF directives in an application is 16.
- Must precede at least one `value_variable`.
- Must be part of an External Data Statement.

3.7 Inconsistency Error

What is an inconsistency error

An inconsistency error occurs when the definition of an external variable in the .CIO file* does not match the HLL value_variable definition in data size and/or array size.

Example:

EDB contains: `value = 1.0/REAL`

FORTTRAN HLL definition is: `CHARACTER*8 VALUE`

*The .CIO file is generated from the EDB when you initiate the `-pe` or `-cb` function of the OpenDDA Command Line Interface (refer to 5.9 in the *OpenDDA Users Guide*).

What happens when an inconsistency error occurs

The behavior when an inconsistency error occurs depends on the INCONSISTENCY directive in the .DEF file. This directive is:

```
{ INCONSISTENCY = [ABORT|RETURN] }
```

The directive is optional. If it is not present, the default is ABORT.

If ABORT is specified (explicitly or by default), the application will abort upon finding an inconsistency error.

If RETURN is specified, the application will not be aborted and a status is returned to the user (DDA_INCONSISTERR).

In either case, information is printed out to the .dda_out file (regardless of the print switches in the .DEF file) to aid the user in identifying what caused the error.

User action for RETURN

The user should not attempt to recover from an inconsistency error. The recommended action is to orderly shut down the application and correct the problem.

Example The .DEF file contains (in part):

```
INCONSISTENCY = RETURN;
```

EDB contains (in part):

```
val1 = 1.0 /REAL  
val1_vs = INPUT CURRENT (,ststgdx.strval[1])
```

HLL declarations:

```
char val1[41];  
long int val1_vs;
```

Code:

```
EXEC DDA READ SET = 'ALL', STATUS = exec_status;
```

Results:

The .dda_out file will indicate an inconsistency error with variable val1.
The exec_status will contain the error status DDA_INCONSISTERR.

Inconsistency error dump information

The inconsistency error dump information contains the following:

The value variable name indicates the name of the value variable within the EDB that is associated with the error.

The data item indicates which data item within the EDB statement caused the inconsistency error. It can be:

VALUE	the value variable
VALUSTAT	the return status variable
STORCODE	the store code variable
STORSTAT	the store status variable
TIMESTAMP	the TIMESTAMP ancillary variable
TEST	the TEST ancillary variable
FULL_NAME	the FULL_NAME external reference variable
POINT_NAME	the POINT_NAME external reference variable
PARAM_NAME	the PARAM_NAME external reference variable
INDEX_NAME	the INDEX_NAME external reference variable
EXT_REF_STATUS	the external reference status variable
MODIFY_CODE	the modify code variable

The data index is the value used to index into the variable, if it was an array. If the variable is not an array, this value will be 1.

The supplied data buffer size is the size (in bytes) of the HLL variable. The expected is the size of the data buffer that OpenDDA was expecting.

The data item's type is the data type specified in the EDB for this variable. This can be:

	OpenDDA Type
REAL	/real
INTEGER	/integer
ENUMERATED	/enumerated
ORDINAL	/ordinal
STRING	/string
TIME VALUE	/absolute or /delta
POINT ID	/point_id
EXTERNAL	/external_id

**Example
inconsistency
error dump**

Example dump (assuming INCONSISTENCY directive in .DEF file is set to ABORT):

ERROR Inconsistency error encountered. Application Terminating.

<<< Inconsistency Error Dump Information >>>

Value-Variable Name was	: val1
Data Item was	: VALUE
Data Index (if array)	: 1
Supplied Data Buffer Size was	: 41 (Expected: 4)
Data Item's Type was	: REAL

3.8 OpenDDA Sets

Two types of sets OpenDDA uses sets to define which External Data Statements are acted upon by certain EXEC DDA statements. There are two types of sets.

Table 3-4 Sets Used in OpenDDA

Type of Set	Purpose	OpenDDA Usage
Data Access Sets	Determine which External References will be read or written.	EXEC DDA READ EXEC DDA WRITE EXEC DDA LOAD STORE_CODE
Dynamic Selection Sets	Defines • which External Data Statement's External References will be modified, and • which Alias Variables will specify the new External References.	EXEC DDA MODIFY CIO EXEC DDA LOAD ALIAS

Relationship of sets

In this release, Dynamic Selection Sets are related to, and have the same names as, corresponding Data Access Sets.

In an External Data Statement, a Value Variable can be assigned to one or more Data Access Sets by the SET_LIST directive. If the External Data Statement specifies that one or more components of the External Reference are to be dynamically selected and therefore have Alias Variables defined for those components, then the Alias Variables are assigned to one or more Dynamic Selection Set(s) by the SET_LIST directive. Therefore, for a single EDB statement, items that have dynamic components will be included in both Data Access Sets and Dynamic Selection Sets of the same name, as defined in the SET_LIST directive.

The following example illustrates the assignment of variables to Data Access Sets and Dynamic Selection Sets.

EDB Example

The following EDB statements illustrate the assignment of Value Variables and Alias Variables to sets.

```
Var1 = 1.0 /REAL
Var1Status = INPUT CURRENT (AXM, *.PV)
Var1Point_Name = POINT_NAME
SET_LIST = Set_1
```

In the above partial External Data Statement, Value Variable var1 is assigned to the Data Access Set Set_1, and because the Point Name component of the External Reference is designated for Dynamic Selection, the Alias Variable Var1Point_Name is also assigned to the Dynamic Selection Set Set_1.

```
Var2 = 1.0 /REAL
Var2Status = INPUT CURRENT (AXM, A100.PV)
SET_LIST = Set_2
```

In the above case, Var2 is assigned to the Data Access Set Set_2. Since no Dynamic Selection is specified, no variables are assigned to Dynamic Selection Sets.

```
Var3 = 1.0 /REAL
Var3Status = INPUT CURRENT (AXM, B100.*)
Var3Param_Name = PARAM_NAME
SET_LIST = Set_1, Set_2
```

In the above partial External Data Statement, Value Variable var3 is assigned to the Data Access Sets Set_1 and Set_2, and because the Parameter Name component of the External Reference is designated for Dynamic Selection, the Alias Variable Var3Param_Name is assigned to Dynamic Selection Sets Set_1 and Set_2.

Table of set assignments

The following table summarizes how Value Variables and Alias Variables are assigned to sets for the above example.

Table 3-5 Table of Set Assignments for the Example

Data Access Sets		Dynamic Selection Sets	
Set Name	Value Variable	Set Name	Alias Variable
Set_1	Var1 Var3	Set_1	Var1Point_Name Var3Param_Name
Set_2	Var2 Var3	Set_2	Var3Param_Name

The 'ALL' selection set

Both Data Access Sets and Dynamic Selection Sets have an implicit 'ALL' set. The set name 'ALL' must not be used to name any user-defined sets.

What is a Data Access Set

A Data Access Set is set of Value Variables that can be read to or written from as a group by an EXEC DDA READ or WRITE, or can be loaded with a Store Code as a group by an EXEC DDA LOAD STORE_CODE.

All Value Variables defined within the EDB are members of the 'ALL' Data Access Set.

A Data Access Set contains the group of Value Variables assigned to it.

A Value Variable may be assigned to and belong to multiple Data Access Sets.

What is a Dynamic Selection Set

A Dynamic Selection Set is a set of Alias Variables that are dynamically selected (changed) as a group during application execution. External References are selected by the EXEC DDA MODIFY CIO statement on a Dynamic Selection Set level. Members of a Dynamic Selection Set can also have their Alias Variables loaded as a group during application execution by the EXEC DDA LOAD ALIAS statement.

All Alias Variables defined within the EDB are members of the 'ALL' Dynamic Selection Set.

A Dynamic Selection Set contains the group of Alias Variables assigned to it.

An Alias Variable may belong to multiple Dynamic Selection Sets.

Maximum sets allowed

The maximum number of Sets allowed per External Data Statement is 100.

3.9 Run Time Selection of External Data

Introduction	OpenDDA provides two techniques for selection of External Data References at run time. They are: • Remapping CIO, and • Dynamic Selection Sets.
Remapping CIO concept	OpenDDA provides the user with the tools to dynamically remap the application variables to a different set or sets of external data at run time by allowing an application to read multiple CIO files into memory and to select which one will be active at any given time. OpenDDA EXEC DDA functions allow the application to read, write, and clear CIO files to/from memory, and select which one is to be active.
The remapping tools	Four OpenDDA EXEC DDA statements are provided to support remapping. • EXEC DDA READ CIO—reads a specified CIO file from disk into memory • EXEC DDA WRITE CIO—writes a specified CIO file from memory to disk • EXEC DDA SELECT CIO—Selects a specified memory-resident CIO file to be the active one • EXEC DDA CLEAR CIO—Clears a specified memory-resident CIO file from memory
Dynamic Selection Sets concept	OpenDDA contains functionality that allows the user to select External Reference components at run time (Dynamic Selection). The External Reference components that can be dynamically selected are: • Point name • Parameter name • Index name • Full name, with or without index, as follows: – Point name and parameter name – Point name, parameter name, and index name

**Dynamic
Selection Sets
tools**

OpenDDA provides the following EDB support for Dynamic Selection Sets:

- **Dynamic Selection Indicators**—allows the user to indicate which component or components will be dynamically selected by an EXEC DDA MODIFY CIO statement. This is accomplished by using the DDA_NULL value in an External Reference specification. The DDA_NULL is defined as the asterisk character. For example, an External Reference specified as A100.* indicates that the parameter component will be dynamically selected.
- **Alias Variables**—native high-level language variables that the application loads with desired External Reference component names prior to executing an EXEC DDA MODIFY CIO statement. An Alias Variable must be associated with each Dynamic Selection Indicator.
- **External Reference Status Variable**—optional variable that can be included in External Data Statements in the EDB. You can use it to receive a status that indicates whether or not the External Reference was successfully modified by a EXEC DDA MODIFY CIO operation.
- **Modify Code Variable**—optional variable that can be included in External Data Statements in the EDB. You can use it to determine whether or not External Reference names in the memory resident copy of the CIO file will be updated with names in Alias Variables by an EXEC DDA MODIFY CIO.

OpenDDA provides the following EXEC DDA statements to support Dynamic Selection of data:

- **EXEC DDA MODIFY CIO**—selects new External References for all Value Variables that have associated Alias Variables that are members of the specified Dynamic Selection Set or Sets. These Alias Variables must be loaded with the desired External Reference component names prior to invocation of this statement.
- **EXEC DDA LOAD ALIAS**—loads Alias Variables for a specified Dynamic Selection Set or Sets with the corresponding External Reference component values that are stored in the OpenDDA control structures in the memory resident version of the active CIO
- **EXEC DDA LOAD MODIFY_CODE**—automatically loads all defined Modify Code variables with a user-specified Modify Code

References

For more information on these topics, see to the following references:

Table 3-6 Table of References

Topic	Reference
Dynamic Selection Indicators	Refer to “Using Dynamic Selection Indicators” on page 71.
Alias Variables	Refer to “Alias Variable Statement” on page 66.
External Reference Status	Refer to “External Reference Status Variables” on page 86.
Modify Code Variable	Refer to “Modify Code Variables” on page 89.
EXEC DDA MODIFY CIO	Refer to “MODIFY CIO Statement” on page 121.
EXEC DDA LOAD ALIAS	Refer to “LOAD ALIAS Statement” on page 111.
EXEC DDA LOAD MODIFY_CODE	Refer to “LOAD MODIFY_CODE Statement” on page 114.

Section 4—EDB Data Access Variables

4.1 Overview

Introduction	Data Access functionality allows an application to associate External Data References with HLL variables. This section describes the Data Access Variables associated with Data Access.
What is included	This section covers data access functionality, including the following topics: • Value Variable • Test Values • Input/Output Qualifiers • Location Specification • CURRENT Location Specification • NOX Location Specification • LCN Data • Value Status Variable • Store Code Variable • Store Status Variable • Data Access Time Variable • Test Value Variable • Ancillary Variable Summary • SET_LIST Directive

4.2 Value Variable

Definition The Value Variable is an HLL (High-Level Language) variable that OpenDDA stores input data values into and gets output data values from.

Rules Use the following rules to specify Value Variables.

- there is no limit to the number of Value Variables in an EDB
- the Value Variable may be a field in a record
- the Value Variable may be an element of an array
- the Value Variable may be a range within an array, indicated by a subrange specification
- the Value Variable name is limited to 80 characters
- the data platform data type must match the Value Variable data type

Determine Value Variable data type The data type of the Value Variable can be explicitly specified, or the test value can determine the data type. Refer to “Default EDB data types” in EDB Data Types External Data on page 17.

Variables and data type declarations The array information and the implied data types given within an EDB are known only to OpenDDA. You must declare your HLL variables and their data types within your FORTRAN or C code.

Example: (FORTRAN source code)

Assume the following EDB is used:

```
C  EXTERNAL DATA BEGIN
C    level = 50 /INTEGER
C    level_status = INPUT CURRENT (AXM,L100.VALUE);
C    area1_motora_state = 'off' /STRING
C    area1_out = OUTPUT CURRENT (AXM,PUMP101.OP);
C  EXTERNAL DATA END
```

The following declarations or their equivalent are required in the application:

```
INTEGER*4    level
INTEGER*4    level_status
CHARACTER*(DDA_STRING_SIZE) area1_motora_state
INTEGER*4    area1_out
```

When Value Variables are strings

If a Value Variable declared in an EDB is a string type, the input and output is adjusted to fit the external data location. The following table lists the actions taken when the string sizes are not the same.

Table 4-1 String Lengths

Data Direction	Value Variable Shorter		Value Variable Longer	
	FORTRAN	C	FORTRAN	C
Input (Read)	The input string is truncated to fit the Value Variable	The input string is truncated and null-terminated to fit the Value Variable	The input string is appended with spaces to fill the Value Variable	The input string is null-terminated at the end of the data returned
Output (Write)	The output string is appended with spaces to fill the external data location		The output string is truncated to fit the external data location	

How to use Value Variable arrays

Single and multi-dimensional arrays can be used to input or output External Data.

Subrange specification

A subrange specification is a shorthand notation provided by OpenDDA that allows you to easily associate a Value Variable with a range of array data values.

- Only **one** subrange specification is allowed per Value Variable.

Array subrange statement format

The general format of the two array statement types that can be used in an EDB to provide array access are:

For FORTRAN:

```
C value_variable(n:m) = test_value
```

```
C value_variable(n:m) = test_valuen, ... , test_valuem
```

For C:

```
/*
```

```
    value_variable[n:m] = test_value
```

```
    value_variable[n:m] = test_valuen, ... , test_valuem
```

```
*/
```

where

- n and m are decimal integer values with a range of 0-9999
- n can be any value within the range of the array
- m > n.

Array index difference between FORTRAN and C

There is a potentially confusing array index difference between FORTRAN and C. In FORTRAN the first element of an array is (1), whereas in C the first element is [0].

When Value Variables are arrays

If a Value Variable declared in an EDB is an array, the following ancillary variables (see 4.10 on page 54) must also be arrays of the same size:

- Store Status
- Value Status
- Store Code
- Test
- Timestamp

The indexes need not match.

Array examples***Example (FORTRAN source code):***

```

C   EXTERNAL DATA BEGIN
C   gendata_flows(1:4) = 10.0, 11.0, 12.0, 13.0 /REAL
C   genstatus(1:4) = INPUT CURRENT (AXM,HC301.FLOWVAL(1));
C   report_lines(5:14) = 'Some text ' /STRING
C   report_out(1:10) = OUTPUT CURRENT(AXM,FC101.STR(1));
C   EXTERNAL DATA END
...
      REAL          gendata_flows(4)
      INTEGER*4     genstatus(4)
      CHARACTER*(DDA_STRING_SIZE) report_lines(14)
      INTEGER*4     report_out(10)

```

Example (C source code):

```

/*
  external data begin

    gendata_flow[0:3] = 10.0, 11.0, 12.0, 13.0 /REAL
    genstatus[0:3] = INPUT CURRENT (AXM,hc301.flowval(1));
    report_lines[4:13] = 'Some text ' /STRING
    report_out[0:9] = OUTPUT CURRENT(AXM,fc101.str(1));

  external data end
*/
...

DDA_REAL_TYPE      gendata_flow[4];
DDA_INTEGER_TYPE   genstatus[4];

DDA_STRING_TYPE    report_lines[14];
DDA_INTEGER_TYPE   report_out[10];

```

In the previous examples:

gendata_flows is an array of 4 elements with the Test Value (see 4.3 on page 38) of those elements being 10.0, 11.0, 12.0, 13.0, respectively.

genstatus is an array of 4 elements that will receive the value status for the FLOWVAL(1), FLOWVAL(2), FLOWVAL(3), FLOWVAL(4) associated with the point HC301.

report_lines is an array of 14 elements (with the last 10 elements used) with all test values of those elements being "Some text ".

report_out is an array of 10 elements that will specify the store codes for controlling the writing to the STR array parameters associated with the point FC101.

Using multi-dimensional Value Variable arrays

OpenDDA only supports reading and writing of one-dimensional arrays as shown in the previous examples; however, this does not limit the use of multi-dimensional arrays as potential HLL variables used in EDB definitions. As long as there is not more than one array range specified in the HLL variable declaration in the EDB, the use of HLL variables which are multi-dimensional arrays is allowed. The following example illustrates this concept:

Example (C source code):

```

/*
 *  EXTERNAL DATA BEGIN
 *
 *    gendata_ flows[1][0:3] = 1.0 /REAL
 *    genstatus[0:3] = INPUT CURRENT (AXM, HC301.FLOWVAL(1))
 *
 *    flowrate[0][2][1] = 1.0 /REAL
 *    flowrate_status = INPUT CURRENT (AXM, HC302.FLOWRT(2))
 *
 *  EXTERNAL DATA END
 */
. . .
DDA_REAL_TYPE      gendata_flows[2][4];
DDA_INTEGER_TYPE    genstatus[4];
DDA_REAL_TYPE       flowrate[2][3][3];
DDA_INTEGER_TYPE     flowrate_status;

```

In this example, `gendata_flows` is a two-dimensional array which will have the first four elements of the array element `gendata_flows[1]` (which is itself an array) populated with `HC301.FLOWVAL(1)`, `HC301.FLOWVAL(2)`, `HC301.FLOWVAL(3)`, and `HC301.FLOWVAL(4)`, respectively. The status variable must have space for four statuses, hence its declaration as a four element array.

`flowrate` is a three-dimensional array variable which is being populated with one value, `HC302.FLOWRT(2)`. This value will be assigned to the second element of the array referenced by the third element of the array referenced by `flowrate[0]`. `flowrate_status` will contain the corresponding status of the value read.

How to read array values

When you want to read an element of an LCN array, use the starting number of the array in the point's index.

Examples (FORTRAN source code):

```
v1_vlu_st(1:4)=INPUT CURRENT(axm,A100.STRING(1));  
v2_vlu_st(5:10)=INPUT CURRENT(axm,F120.STRING(5));
```

Examples (C source code):

```
v1_vlu_st[1:4]=INPUT CURRENT(axm,a100.string(1));  
v2_vlu_st[5:10]=INPUT CURRENT(axm,f120.string(5));
```

However, the variable would need to be declared as:

```
DDA_INTEGER_TYPE    v1_vlu_st[5];  
DDA_INTEGER_TYPE    v2_vlu_st[11];
```

because C arrays are 0-based. In this example, `v1_vlu_st[0]` is not used. Similarly, `v2_vlu_st[0]` through `v2_vlu_st[4]` is not used.

4.3 Test Values

Definition The Test Value is the value that OpenDDA will input to the Value Variable when the application is executing in test mode.

Rules The following rules apply to Test Values:

- The Test Value can be a maximum of 40 characters.
- The maximum number of test values allowed for each Value Variable is 100.

Test Value data types The Test Value also identifies the data type for the Value Variable unless the data type is explicitly stated. The following table lists the various data types and their TDC 3000^x equivalents. Refer to “Using EDB data types” in EDB Data Types External Data on page 15.

Table 4-2 Test Value Data Types

EDB Data Type Declaration	TDC 3000 ^x Data Type	Example
If the test value has either a period “.” or exponent (Enn)...		
/REAL†	REAL	-3.5, 6E12
If the test value is enclosed in single quotes*		
/STRING†	ASCII	'Flow ratio complete'
/ENUMERATED	ENUMERATION	'ON','OFF'
/ABSOLUTE‡ (Refer to 4.14)	TIME	'12-OCT-1994 12:03:15.23' Format: DD-MMM-YYYY HH:MM:SS.CC
/DELTA‡ (Refer to 4.14)	TIME	'28 08:32:52.16' Format: DDDD HH:MM:SS.CC
/POINT_ID‡ (can be entered in integer or hex notation) (Refer to 4.9)	ENTITY ID	'5672 3492 5734 6836' '%x00F6 %x005D %x0400 %x000C' Format: 0000 0000 0000 0000 (Point IDs are read from left to right.)
/EXTERNAL_ID	ENTITY ID	'FI205'
If the test value has none of these: “.”, (Enn) or single quotes		
/INTEGER	INTEGER	5000, 3, 86
/ORDINAL†	ENUMERATION	0, 1, 2
†Default data type *Note: Use ONLY single quotes ‡Even though the test value is specified as a string, the data is manipulated as a structure. (Refer to Table 3-2 on page 16.)		

Accessing CDS parameters of type logical

Do not confuse FORTRAN logical variables with LCN Custom Data Segment (CDS) parameters of type logical. CDS logical parameters are accessed using an EDB data type of either /ENUMERATED or /ORDINAL. The values returned are shown below.

Table 4-3 Returned Values when Accessing CDS Logical Parameters

CDS Value	EDB Data Type: /ENUMERATED	EDB Data Type: /ORDINAL
ON	'ON'	1
OFF	'OFF'	0

Test data

The Test Values in EDB statements form the initial set of test data.

You determine whether your application uses:

- a static set of test data, or
- live data, or
- a combination of the two

when it executes.

Using arrays of Test Values**Example:**

```
Level_Est(1:4) = 15.2, 15.8, 16.3, 16.7
```

In this statement, the values 15.2, 15.8, 16.3, and 16.7 are assigned to Level_Est(1), Level_Est(2), Level_Est(3), and Level_Est(4), respectively.

Test values will be assigned sequentially to the elements of the array Level_Est(1:4).

If not enough Test Values to fill an array

If there are not enough Test Values to fill an array, the **last** Test Value in the list will be assigned to the remaining elements of the Value Variable array.

Example:

```
C  EXTERNAL DATA BEGIN
C  ...
C    Flow_calc(1:4) = 2.468/REAL
C  ...
C    recipe_inflow(1:6) = 5.0, 6.5, 9.5, 12.0 /REAL
C  ...
C  EXTERNAL DATA END
```

In this example

2.468 is assigned to all elements of the array Flow_calc(1:4).

Recipe_inflow is an array of 6 elements with the initial values of those elements being 5.0, 6.5, 9.5, 12.0, 12.0, and 12.0. There are only four Test Values, so the last one is assigned to the rest of the array elements.

4.4 Value Variable Options

Options Value Variables have the following valid options or switches:

- /data_type
- /TEST_VALUE, /NOTEST_VALUE

/data_type /data_type identifies the data type of the value_variable. Refer to Table 3-2 on page 16 for a description of the available EDB data types.

/TEST_VALUE and /NOTEST_VALUE qualifier The /TEST_VALUE and the /NOTEST_VALUE qualifiers indicate whether the value supplied to the application is the user-supplied test value or the value contained in the external location.

This table shows the effect of the /TEST_VALUE and the /NOTEST_VALUE qualifiers on data transfer.

Table 4-4 /TEST_VALUE and /NOTEST_VALUE Qualifiers

Data Transfer Direction	/TEST_VALUE	/NOTEST_VALUE
Input data	Test value is input	Value from external data source is input
Output data	No data is output to the external data location	Data is output to the external data location.

Default behavior If you do not specify a qualifier, the default is /NOTEST_VALUE .

Overriding the EDB

The Test Value qualifiers (/TEST_VALUE and /NOTEST_VALUE) can be overridden using the OpenDDA Command Line Interface (CLI). The CLI allows you to specify the variable's value source as EDB, TEST, or EXTERNAL. (Refer to 5.3 in the *OpenDDA User Guide*.)

Example:

```
C value1=1.0/TEST_VALUE
C   value1_status=INPUT CURRENT(AXM,point.pv);
C value2=2.0/NOTEST_VALUE
C   value2_status=OUTPUT CURRENT(AXM,point.pv);
C value3=5.2
C   value3_status=INPUT CURRENT(AXM,point.pv)
C   value3_store_code=OUTPUT;
```

This table shows the value each of the previous Value Variables will have depending on the value source selected.

Table 4-5 Results from the Previous Example

Variable	Command Line Option		
	(none)	(-T)	(-X)
value1	1.0	1.0	point.pv
value2	point.pv	2.0	point.pv
value3	point.pv	5.2	point.pv

4.5 Input/Output Qualifiers

Definition The INPUT and OUTPUT keyword assign the direction of data transfer for the value_variable.

INPUT keyword The INPUT keyword defines the Value Variable as receiving its input from an External Data source.

Example:

```
C  EXTERNAL DATA BEGIN
C    temp_data_in = 200.0/REAL
C    temp_data_in_status = INPUT CURRENT(AXM,temp_data.sp);
C  EXTERNAL DATA END
```

In this example, the temp_data_in variable will receive the value in the external data variable, temp_data.sp. The temp_data_in_status variable will receive the status value associated with temp_data.sp.

OUTPUT keyword The OUTPUT keyword defines the value of the Value Variable as the output to an external data item.

Example:

```
C  EXTERNAL DATA BEGIN
C    flow_out = 500.0 /REAL
C    flow_out_store_code = OUTPUT CURRENT(AXM,flow_out.sp)
C    flow_out_store_status = STORE_STATUS;
C  EXTERNAL DATA END
```

In this example, the flow_out_store_code variable will contain the store code, which determines whether to store the flow_out value to the external data source variable, flow_out.sp. Also, the flow_out_store_status variable will contain the status of the EXEC DDA WRITE operation for the flow_out Value Variable.

If a variable is both input and output A variable that both inputs and outputs data (presumably because the value is modified by the application) is defined by specifying both the INPUT and OUTPUT qualifiers.

Example:

```
C  EXTERNAL DATA BEGIN
C    special_data = 200.0 /REAL
C    special_data_status = INPUT CURRENT(AXM,special.data)
C    special_data_store_code = OUTPUT;
C  EXTERNAL DATA END
```

ATTENTION

OUTPUT does not require a location_spec if INPUT has specified it.

**If neither INPUT
nor OUTPUT is
specified**

If neither INPUT or OUTPUT is specified, the Value Variable is simply assigned the value of the Test Value and becomes a NOX variable. Refer to “NOX Location Specification” on page 46.

Example:

```
C EXTERNAL DATA BEGIN
C   special_data = 200 /REAL;
C EXTERNAL DATA END
```

**If INPUT and
OUTPUT are
specified**

If both an INPUT and OUTPUT qualifier are specified and they conflict, the INPUT definition will be used.

Example:

```
C EXTERNAL DATA BEGIN
C   FC101_flow(1:6) = 5.0, 6.5, 9.5, 12.0 /REAL
C   flow_status(3:8)= INPUT CURRENT(AXM,FC101.FLOW(1))
C   flow_store_code(3:8) = OUTPUT CURRENT(AXM,FC103.FLOW(1));
C EXTERNAL DATA END
```

Both input and output will reference FC101. The output reference FC103 will be ignored.

**If only one
Location
Specification is
specified**

If only one Location Specification is specified, it will be used for both input and output. (Refer to 4.6 on page 44.)

Example:

```
C EXTERNAL DATA BEGIN
C   FC101_flow(1:6) = 5.0, 6.5, 9.5, 12.0 /REAL
C   flow_status(3:8) = INPUT CURRENT(AXM,FC101.FLOW(1))
C   flow_store_code(3:8) = OUTPUT;
C EXTERNAL DATA END
```

Both input and output will reference FC101.

4.6 Location Specification

Definition The Location Specification identifies the External Reference. The Location Specification must immediately follow the qualifier for which it applies (INPUT or OUTPUT).

Format There are two values for the Location Specification :

- CURRENT—for data access (LCN and other DCS platforms)
- NOX—for no exchange of data with an external source or destination

4.7 CURRENT Location Specification

Format CURRENT([access_name], [ng\]point.parameter [(index)])

access_name The access_name field can be either

- “AXM”, or
- a symbolic name as previously defined using the EXTDEF directive.

Default access_name If no access_name is specified, the value defaults to “AXM”.

CAUTION

Do not use the following reserved ACCESS NAMES:

DDA	FILE	NOX	TEST
HOST	FI	NX	TS

ng The ng\ field contains an up to two character alphanumeric field followed by a backslash. Refer to subsection 2.2 of the *Network Gateway Implementation and Operation* manual.

point.parameter The point.parameter field is a DCS point name and parameter name.

index When the point.parameter is an array, the following conditions apply:

- If no subrange is used, the index indicates the array element to be accessed.
- If a subrange has been specified, the index indicates the starting array element to be accessed.

4.8 NOX Location Specification

Definition	The "NOX" Location Specification type defines a variable that is not associated with any external entity. "NOX" stands for "NO eXchange of data with an external source or destination."
Function	NOX variables can be written to or read from the memory resident CIO with EXEC DDA WRITE and EXEC DDA READ commands, but no external data is transferred.
Format	NOX
Syntax	There are three basic methods of specifying NOX. Method 1 Example: <pre>val1 = 1.0 /REAL val1_status = INPUT NOX;</pre> Method 2 Example: <pre>val1 = 1.0 /REAL val1_status = INPUT;</pre> Method 3 Example: <pre>val1 = 1.0 /REAL;</pre> Method 1 is the full specification, and Method 2 is a shorthand version of Method 1 in which NOX is implied because no External Reference is specified with the INPUT keyword. Method 1 and Method 2 can be used as placeholders for future "live" references. Method 3 is an abbreviated version that does not include a status variable definition. Method 3 cannot be changed later to an actual External Reference without causing a CIO Signature error. It is, however, an effective way to pass a constant to the application through the EDB. The constant can be changed without having to recompile and relink the application.
Initial value	If a NOX variable is read before it is written, the test value will be read.

Uses

Some potential uses for NOX are:

- NOX definitions can be used as place holders for future use. One can later efficiently convert NOX variables to "live" data references with little rework and no application rebuilding other than modifying the EDB and parsing to create a new CIO.
- Data such as text, limits, or configuration can be read by an application program from an EDB that is separate from the source code (see 4.4 on page 33 of the *OpenDDA User's Guide*). In this manner, an application can be "customized" without recompilation, simply by changing data in the separate EDB (and parsing it). To extend the concept further, a single application program can be applied to multiple unit instances by using a separate EDB for each instance, with each EDB containing the instance-specific information in NOX variables.
- NOX variables can be written by one subroutine or function, and read by another subroutine or function, as a means of passing parameters. This would be an alternative to other methods such as:
 - Arguments included in the call
 - Global variables
 - Passing pointers to arrays or structures in C

Dynamic selection caution

The CURRENT Location Specification must be used with External References that are to be dynamically selected. (Refer to "MODIFY CIO Statement" on page 121.) Using NOX will result in an error.

Discussion of NOX demo program

Since NOX stands for "NO eXchange," the EDB Value Variables that are defined as NOX are not associated with an actual external data platform; therefore, upon an EXEC DDA READ or EXEC DDA WRITE there is no exchange of data with any external data platform. The value contained in the NOX Value Variable is placed in the memory resident CIO upon an EXEC DDA WRITE and returned by an EXEC DDA READ (the first EXEC DDA READ of a NOX results in the test value being returned— see Use 1 discussion below).

The example program illustrates two uses of NOX Value Variables:

- Use 1—NOX is used to create place-holders for three more temperature inputs, and
- Use 2—NOX is used to define application constants pertaining to the number of current 'live' temperature inputs and an application description string.

Use 1 is an excellent method for "building in" scalability and flexible expansion of external data references. By using NOX definitions as place-holders, you can efficiently add 'live' data references with little rework and no application rebuilding other than modification of the EDB and parsing to create a new CIO.

Use 2, like Use 1, is another way to insulate applications from having to be recompiled and relinked by utilizing the parsing of the EDB to create a new CIO with the incorporated changes rather than editing application source. The executable can be rerun with the changed constant being used.

NOX demo program

The following is a listing of the NOX demo program that is distributed with the other OpenDDA examples in the /opt/DDA/examples directory.

```
/*
 * NOX_DEMO.C
 *
 *
 *          COPYRIGHT © 1995
 *      H O N E Y W E L L   I N C O R P O R A T E D
 *      INDUSTRIAL AUTOMATION AND CONTROLS DIVISION
 *
 *          ** ALL RIGHTS RESERVED **
 *
 * This software is a copyrighted work and/or information protected
 * as a trade secret. Legal rights of Honeywell Inc. in this
 * software is distinct from ownership of any medium in which the
 * software is embodied. Copyright or trade secret notices included
 * must be reproduced in any copies authorized by Honeywell Inc.
 *
 * The information in this software is subject to change without
 * notice and should not be considered as a commitment by Honeywell
 * Industrial Automation and Controls Division.
 *
 ****
 *
 * This program demonstrates the utilization of OpenDDA EDB NOX value
 * variables. The program has been simplified to better show the NOX
 * interaction.
 *
 * Two uses of NOX value variables are demonstrated in the program:
 *
 *      1) As place-holders for the definition of future 'real',
 *         platform-associated value variables.
 *
 *      2) As a mechanism for defining application constants which allow
 *         for flexible change without recompile or relink of the application.
 *
 * The application is coded such that by simply editing the EDB and
 * modifying the NOX entries, additional temperature inputs will be used
 * in the application's processing. Coinciding with this would be the
 * change of the NumOfTempInputs to match the new 'live' number of inputs.
 * All this is accomplished with no modification to the executable -
 * just parse the EDB and rerun!
 */
```

EDB Data Access Variables

4.8 NOX Location Specification

```
/*
    External Data Begin

    ! Temperature Inputs - two live with three NOX place-holders !
    InTemp1 = 0.0 /REAL
    InTemp1_Status = INPUT CURRENT (AXM, fic101.temp);

    InTemp2 = 0.0 /REAL
    InTemp2_Status = INPUT CURRENT (AXM, fic102.temp);

    InTemp3 = 0.0 /REAL
    InTemp3_Status = INPUT NOX;

    InTemp4 = 0.0 /REAL
    InTemp4_Status = INPUT NOX;

    InTemp5 = 0.0 /REAL
    InTemp5_Status = INPUT NOX;

    ! NOX constants !
    NumOfTempInputs = 2;
    AppName = 'TempControl for Tank #1';

    External Data End

*/

/* Standard C header files */
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

/* OpenDDA header file */
#include "dda_c.h"

int
main( void )
{

    /* Temperature Inputs */
    DDA_REAL_TYPE      InTemp1, InTemp2, InTemp3, InTemp4, InTemp5;

    /* # of 'live' temperature inputs */
    DDA_INTEGER_TYPE   NumOfTempInputs;

    /* Description of Application */
    DDA_STRING_TYPE     AppName;
```

```

/* Temperature Input Statuses */
DDA_STATUS_TYPE    InTemp1_Status, InTemp2_Status;
DDA_STATUS_TYPE    InTemp3_Status, InTemp4_Status, InTemp5_Status;

/* EXEC DDA Overall Status */
DDA_STATUS_TYPE    ExecStatus;

/* Variables used to compute Temperature Average */
float  TotalTemp = 0;
float  TempAverage;

long i;

/* ... Begin (NOX_DEMO) ..... */

/* Initialize OpenDDA environment */
EXEC DDA INITIALIZE STATUS = ExecStatus;
if (!(ExecStatus & 1)) {
    fprintf(stderr, "Initialization failed with status %d\n", ExecStatus);
    exit(1);
}

EXEC DDA READ SET= 'ALL', STATUS = ExecStatus;
if (ExecStatus != HGSC_VALID) {
    fprintf(stderr, "READ failed with status %d\n", ExecStatus);
    exit(1);
}

/* Successful read; Now compute Average Temperature */

for (i = 1; i <= NumOfTempInputs; i++) {
    switch (i) {
        case 1: TotalTemp += InTemp1; break;
        case 2: TotalTemp += InTemp2; break;
        case 3: TotalTemp += InTemp3; break;
        case 4: TotalTemp += InTemp4; break;
        case 5: TotalTemp += InTemp5; break;
    }
}

TempAverage = TotalTemp / NumOfTempInputs;

printf("Temperature Control Application ID: %s\n", AppName);
printf("# of 'active' Inputs: %d - Average Temperature: %f\n",
    NumOfTempInputs, TempAverage);

```

EDB Data Access Variables

4.8 NOX Location Specification

```
EXEC DDA TERMINATE STATUS = ExecStatus;
if (!(ExecStatus & 1)) {
    fprintf(stderr, "Termination failed with status %d\n", ExecStatus);
    exit(1);
}
}
```

4.9 LCN Data

TDC 3000^x database

The TDC 3000^x database consists of *data points* that are records of related data items called *parameters*. TDC parameters consist of an optional ng\ followed by the data point name and parameter name separated by "." and an optional index enclosed in parentheses:

ng\point.parameter[(index)]

Example:

```
C  EXTERNAL DATA BEGIN
C    EXTDEF (AXM, DEFAULT)
C    tc201_pv = 250.0 /REAL
C    tc201_pv_status = INPUT CURRENT (AXM, tc201.pv);
C    tc201_sp = 300.0 /REAL
C    tc201_sp_store_code = OUTPUT CURRENT (AXM, tc201.sp);
C    feed_tmp_pvhitp = 455.0 /REAL
C    status3 = INPUT CURRENT (AXM, feed_tmp.pvhitp);
C  EXTERNAL DATA END
```

In this example:

At application execution time, the variable tc201_pv will receive its input from tc201.pv and tc201_pv_status will contain a value indicating the Value Status associated with the read of tc201_pv.

The variable tc201_sp will output its value to tc201.sp only if the output Store Code Variable, tc201_sp_store_code, was set to DDASC_STORE prior to the store. Refer to 6.18.

The variable feed_tmp_pvhitp will receive its input from feed_tmp.pvhitp. status3 will contain a value indicating the Value Status associated with feed_tmp_pvhitp.

4.10 Ancillary Variable Summary

Using ancillary EDB variables

Using ancillary EDB variables, you can obtain “ancillary” information about a particular external DCS data point. The information is placed in your HLL application variables at runtime and can be manipulated.

ATTENTION

These variables are not written back to the external data location, they are *read-only*.

Ancillary variable syntax

The EDB syntax to obtain these ancillary values is listed below. The ancillary variables appear in **bold** text.

```
value_variable = test_value [,test_valuen] /data_type  
value_status_variable = INPUT CURRENT(location_spec)  
store_code_variable = OUTPUT CURRENT(location_spec)  
store_status_variable = STORE_STATUS  
data_access_time_variable = TIMESTAMP  
test_value = TEST;
```

Ancillary EDB variables

This table describes each of the available ancillary EDB variables.

Table 4-6 Ancillary EDB Variables

Variable	Description	HLL Variable Declaration
STORE_STATUS	The status associated with the output of the data value to its external location (refer to 4.13)	In C: DDA_STATUS_TYPE store_status_var; In FORTRAN: INTEGER*4 store_status_var
TIMESTAMP	The timestamp of the access to the data value (refer to 4.14)	In C: DDA_TIME_TYPE time_var; In FORTRAN: RECORD /DDA_TIME_TYPE/ time_var
TEST	The test value as defined in the EDB (refer to 4.3 and 4.15)	C Examples: DDA_REAL_TYPE real_var; DDA_INTEGER_TYPE integer_var; DDA_STRING_TYPE string_var; FORTRAN Examples: REAL*4 real_var INTEGER*4 integer_var CHARACTER*(DDA_STRING_SIZE) string_var
VALUE_STATUS	The status associated with the input of the data value from its external location (refer to 4.11)	In C: DDA_STATUS_TYPE value_status_var; In FORTRAN: INTEGER*4 value_status_var
STORE_CODE	The code that determines whether or not to store the value of the Value Variable to the external location (refer to 4.12)	In C: DDA_CODE_TYPE store_code_var; In FORTRAN: INTEGER*4 store_code_var

4.11 Value Status Variable

Accessing Value Status Variables

You can access the Value Status Variable anywhere in your application after the value_variable has been read, to determine the status of the value that was input to your Value Variable.

Example:

In this FORTRAN example, you can access the variable `genstatus(1)` to check the status of the input into variable `gendata_flows(1)`.

```
C  EXTERNAL DATA BEGIN
C    gendata_flows(1:4) = 10.0, 11.0, 12.0, 13.0 /REAL
C      genstatus(1:4) = INPUT CURRENT (AXM,hc301.RLARR(1));
C  EXTERNAL DATA END
C...
```

```
REAL*4      gendata_flows(4)
INTEGER*4    genstatus(4)
INTEGER*4    exec_status
.
.
EXEC DDA READ SET = 'ALL,' STATUS = exec_status;
.
IF (genstatus(1) .NE. HGSC_VALID) THEN
  PRINT *, 'Bad Status for gendata_flows :', genstatus(1)
```

Value Status values

Refer to Table 6-16 on page 132 for the Value Status values that can be returned.

Declaring Value Status Variables

OpenDDA provides a status type in the include file `dda_c.h` that can be used to declare Value Status Variables in C programs. Honeywell recommends using this type for portability and migration considerations. FORTRAN programs use `INTEGER*4` for Value Status Variables.

C Example:

```
DDA_STATUS_TYPE value_status_var;
```

FORTRAN Example:

```
INTEGER*4 value_status_var
```

Using Status Values

Refer to 7.2 of the *OpenDDA User's Guide* for information on interpreting OpenDDA Status Values.

4.12 Store Code Variable

Store Code Variables

Store Code Variables determine the action to be taken when outputting the value for a Value Variable.

The action choices are:

- store the value
- store a NaN (Not a Number)
- do not store any value

Example:

In this example, `flow_out_store_code` will contain the store code.

```
C  EXTERNAL DATA BEGIN
C    flow_out = 500.0 /REAL
C    flow_out_store_code = OUTPUT CURRENT(AXM, flow_out.sp);
C  EXTERNAL DATA END
```

Store code constants

OpenDDA provides three store code constants that should be used by your HLL source code. The following table lists the constants and what they indicate.

Table 4-7 Store Code Constants

Store Code Constant	Store Code indicates...
DDASC_STORE	store value
DDASC_STORE_NAN	store NaN (Not a Number)
DDASC_NO_STORE	do not store value

ATTENTION

Use the constants provided in the HLL (High-Level Language) include file:
`/opt/DDA/include/dda_fortran.f` (FORTRAN compiler)
`/opt/DDA/include/dda_c.h` (ANSI C or C++ compiler)

Declaring Store Code Variables

OpenDDA provides a status type in the include file `dda_c.h` that can be used to declare Store Code Variables in C programs. Honeywell recommends using this type for portability and migration considerations. FORTRAN programs use `INTEGER*4` for Store Code Variables.

C Example:

```
DDA_CODE_TYPE store_code_var;
```

FORTRAN Example:

```
INTEGER*4 store_code_var
```

**Platform
restrictions**

Some DCS platforms may restrict the use of DDASC_STORE and DDASC_STORE_NAN.

For example, on the LCN-based platform, DDASC_STORE_NAN supports only the REAL data type. Refer to your platform manual for specifics.

**Store Code
Defaults**

The Store Code variables are initialized to whatever value is specified in the .DEF file when you do an EXEC DDA LOAD STORE_CODE. If there isn't a value specified in the .DEF file for the Store Code, then the Store Code Variable will be initialized to NO_STORE. Refer to "DEF file is not required" in Overview Definition (DEF) File on page 160.

If the user does not call EXEC DDA LOAD prior to a Write, then the Store Code Variable will not be initialized. It is up to the user to initialize the Store Code Variable. If the user does not initialize the Store Code Variable, an error indicating an invalid Store Code (DDA_STORE_CODE_INVALID) will be returned.

4.13 Store Status Variable

Accessing Store Status Variables

The Store Status Variable can be referenced anytime after the Value Variable has been output and it will contain the status of the output to the external location.

Example:

```

C  EXTERNAL DATA BEGIN
C    gendata_flows = 10.0/REAL
C      genstore_code = OUTPUT CURRENT (AXM, genflow.sp)
C      genstore_status = STORE_STATUS;
C  EXTERNAL DATA END
...
REAL      gendata_flows
INTEGER*4  genstore_code
INTEGER*4  genstore_status
INTEGER*4  exec_status
...
EXEC DDA WRITE SET='ALL',
              STATUS=exec_status;
IF (IAND (exec_status,1) .NE. 1) THEN
  PRINT *, 'Bad Exec status for gendata_flows:'
  PRINT *, exec_status
ENDIF
IF (IAND (genstore_status,1) .NE. 1) THEN
  PRINT *, 'Bad Store status for gendata_flows:'
  PRINT *, genstore_status
ENDIF
...

```

Store Status values

Refer to Table 6-21 on page 146 for the Store Status values that can be returned.

Declaring store status variables

OpenDDA provides a status type in the include file `dda_c.h` that can be used to declare Store Status Variables in C programs. Honeywell recommends using this type for portability and migration considerations. FORTRAN programs use `INTEGER*4` for Store Status Variables.

C Example:

```
DDA_STATUS_TYPE store_status_var;
```

FORTRAN Example:

```
INTEGER*4 store_status_var
```

Using Status Values

Refer to 7.2 in the *OpenDDA User's Guide* for examples of using OpenDDA Status Values.

4.14 Data Access Time Variable

Accessing Data Access Time Variables

The Data Access Time Variable can be referenced anytime after the `value_variable` has been read and it will contain the timestamp of when the read of the value took place.

Example:

```
C  EXTERNAL DATA BEGIN
C    gendata_flows = 10.0/REAL
C      genvalue_status = INPUT CURRENT(AXM, genflow.pv)
C      data_access_time = TIMESTAMP;
C  EXTERNAL DATA END
...
REAL      gendata_flows
INTEGER*4  genvalue_status
RECORD /DDA_TIME_TYPE/ data_access_time
CHARACTER*22 time_string
...
C
C  Write the data_access_time variable to the string
C  variable: "time_string" in the form:
C  year/month/day hour:minute:second.hundredths
C  for subsequent printing
C
C      WRITE(time_string,100)
C      - data_access_time.year,
C      - data_access_time.month,
C      - data_access_time.day,
C      - data_access_time.hour,
C      - data_access_time.minute,
C      - data_access_time.second,
C      - data_access_time.sec_100th
100 FORMAT (I4,2('/',I2.2),X,2(I2.2,':'),I2.2,','I2.2)
PRINT *, 'Time of data acquisition for gendata_flows:'
PRINT *, time_string
```

Declaring Data Access Time Variables

The timestamps returned in the `TIMESTAMP` variable are formatted as OpenDDA time types.

`data_access_time` must be declared as `DDA_TIME_TYPE` in your source code. `DDA_TIME_TYPE` constructs are contained in the OpenDDA include files `dda_c.h` and `dda_fortran.f`.

C Example:

```
DDA_TIME_TYPE data_access_time;
```

FORTRAN Example:

```
RECORD /DDA_TIME_TYPE/ data_access_time
```

ATTENTION

When accessing LCN data, the value returned to `TIMESTAMP` is retrieved from the LCN in Release 200. In previous releases, it was obtained from the HP-UX clock.

DDA_TIME_TYPE

`DDA_TIME_TYPE` is a record structure that holds OpenDDA's representation of a time value.

The structure contains 9, 2-byte integers with the fields in the following order:

Field	Description
year	common era year
month	month of the year – January = 1
day	day of the month
hour	hours since midnight
minute	minutes after the hour
second	seconds after minute
sec_100th	(if available) the hundred of seconds after the second, else will be 0
day_of_week	day of the week – Sunday = 0 (The value is negative if this information is not known.)
day_light	daylight savings time flag If the flag is >0 = daylight savings is in effect =0 = daylight savings is not in effect, <0 = the information is not available

If the year and the month are both zero, OpenDDA interprets the time value to be a delta time; otherwise, it is interpreted as an absolute time.

4.15 Test Value Variable

Accessing Test Value Variables

The Test Value Variable can be referenced anytime after the value_variable has been read. It will contain the test value as defined in the External Data Block.

Example:

```
C    EXTERNAL DATA BEGIN
C    gendata_flows = 10.0 /REAL
C        genvalue_status = INPUT CURRENT(AXM, genflow.pv)
C        test_value = TEST;
C    EXTERNAL DATA END
...
      REAL*4          gendata_flow
      INTEGER*4 genvalue_status
      REAL*4          test_value
...
C    Write out test value
      PRINT*, 'Test value for gendata_flows is ', test_value
```

Declaring Test Value Variables

The values returned in the Test Value Variable are of the same type as the value_variable. Therefore, the Test Value Variable and the Value Variable must be declared as the same type.

Example:

If the Value Variable is gendata_flow of type /REAL, and if the Test Value Variable is test_value, both must be declared as reals.

C declaration:

```
DDA_REAL_TYPE  gendata_flow;
DDA_REAL_TYPE  test_value;
```

FORTTRAN declaration:

```
REAL*4          gendata_flow
REAL*4          test_value
```

4.16 SET_LIST Directive

Definition SET_LIST is an optional directive used to associate a value_variable with one or more Data Access Sets.

SET_LIST syntax The syntax for a SET_LIST EDB entry is:

```
value_variable=test /type
  value_status=INPUT CURRENT(location_specifier)
  store_code=OUTPUT CURRENT(location_specifier)
  store_status_variable=STORE_STATUS
  SET_LIST=set_name1[, set_name2, ...]
  .
  .
  .;
```

SET_LIST rules Follow these rules when specifying the SET_LIST directive.

- set_name is a literal character string that may be optionally bounded by single quotes (' ')
- set_name has a maximum length of 40 characters
- single quotes cannot be used as part of the set_name
- at least one set_name must be specified with the SET_LIST directive
Example: SET_LIST=TestSet
- set names are case-insensitive.
- the SET_LIST directive can specify a maximum of 100 set names
- an External Data Statement can contain only one SET_LIST directive

ATTENTION

Every value_variable within an EDB, by default, is defined as a member of the ALL Data Access Set.

Dynamic Selection Set assignment

If External Reference components are defined for dynamic selection at run time, the External Data Statement that defines the Value Variable will also define Alias Variables for those components that are to be dynamically selected. For these External Data Statements that include Alias Variables, the SET_LIST directive also assigns the Alias Variables to Dynamic Selection Set(s) that have the same name(s) as the Data Access Sets defined in the SET_LIST directive. (Refer to 3.8 on page 25.)

Section 5—EDB Dynamic Selection Variables

5.1 Overview

Introduction Dynamic Data Selection functionality allows an application to modify the External Data References within the OpenDDA memory resident CIO at run time. Any External Reference Component can be modified (point, parameter, or index), or combinations thereof. Also, External References can be grouped and modified in sets—Dynamic Selection Sets. This section describes the Dynamic Selection variables associated with Dynamic Data Selection.

What is included This section covers:

- Alias Variable Statement
- Declaring Alias Variables
- Using Dynamic Selection Indicators
- Alias Variable Arrays
- Loading Alias Variables
- Indexes and Dynamic Selection
- External Reference Status Variable
- Modify Code Variable

Alias Variables and Dynamic Selection Alias Variables are an integral part of Dynamic Data Selection functionality. The following table summarizes how you dynamically select the External Data Reference of a single Value Variable.

Table 5-1 Dynamically Selecting an External Data Reference

Step	When	Action
1	Build Time	Use an External Data Statement to: • Associate a native high-level language Alias Variable or Alias Variables with one or more of the various components of an External Data Reference – point – parameter – index • Associate the native high-level language Alias Variable(s) with a Selection Set.
2	Build Time	Do a normal application build
3	Run Time	Put the desired External Reference name(s) into your designated variable(s)
4	Run Time	Use an EXEC DDA MODIFY CIO statement, specifying the Dynamic Selection Set containing the desired External Data Reference
5	Run Time	The next EXEC DDA READ or EXEC DDA WRITE reads from or writes to the new External Reference

5.2 Alias Variable Statement

Overview	There are three types of variables that can be declared in External Data Statements to set up dynamic selection. They are: • Alias Variables (required for External References with components that can be dynamically modified) • External Reference Status Variables (optional) • Modify Code Variables (optional) This subsection will cover Alias Variables.
Alias Variable statement syntax	The syntax for the required Alias Variable statement is as follows: <pre>alias_variable = external_ref_keyword</pre> where <code>external_ref_keyword</code> is one of the following: • POINT_NAME • PARAM_NAME • INDEX_NAME • FULL_NAME
alias_variable description	<code>alias_variable</code> is an HLL variable name that is supplied by you. The Alias Variable name is limited to 80 characters.
Declaring Alias Variables	OpenDDA provides predefined constructs for declaring HLL Alias Variables. These constructs are contained in the include files <code>dda_c.h</code> and <code>dda_fortran.f</code>. We suggest that developers use the Honeywell-supplied definitions for future migration and portability (Refer to "OpenDDA types supplied" on page 69 for examples.)

**external_ref_
keyword definition**

external_ref_keyword is used to indicate:

- which component of the External Reference is to be dynamically changeable
- format of the reference

The following table defines the external_ref_keywords and shows their application to the LCN environment.

Table 5-2 Definition of external_ref_keywords

Keyword	Component	OpenDDA Format	TDC 3000 ^X
POINT_NAME	point	ASCII	Entity ID
PARAM_NAME	parameter	ASCII	Parameter
INDEX_NAME	index	ASCII	Integer Index (enumerated index is not supported by OpenDDA)
FULL_NAME	point.param(index) or point.param	ASCII	Variable

**Dynamic
Selection Sets**

An Alias Variable is a member of:

- each set listed with the SET_LIST keyword of the same External Data Statement, and
- the 'ALL' set.

**Maximum number
of sets**

The maximum number of Dynamic Selection Sets that can be associated with a SET_LIST directive is 100.

Rules

The following rules apply to External Data Statements that use Alias Variables:

- Multiple external_ref_keywords of the same type in one External Data Statement are not allowed
- The FULL_NAME external_ref_keyword is mutually exclusive with the other external_ref_keywords

See the examples that follow.

Examples

Following are examples of Alias Variable statements in an External Data Statement.

Valid example:

```
var1_point_name = POINT_NAME
```

```
var1_param_name = PARAM_NAME
```

Invalid Example:

```
alias_var1 = POINT_NAME
```

```
alias_var2 = PARAM_NAME
```

```
alias_var3 = POINT_NAME
```

The above example is invalid because POINT_NAME cannot be used twice in the same External Data Statement.

Invalid Example:

```
a1_var = FULL_NAME
```

```
a2_var = POINT_NAME
```

The above example is invalid. The POINT_NAME keyword cannot be used because when the FULL_NAME keyword is used in an External Data Statement, no other keywords can be used.

5.3 Declaring High-Level Language Alias Variables

Alias Variable type and size

Alias Variables are of type character. The following table gives the maximum lengths for Alias Variables.

Table 5-3 Alias Variable Type and Size

Alias Variable Type	Maximum Number of Characters
FULL_NAME	35
POINT_NAME	19
PARAM_NAME	8
INDEX_NAME	5

The length of Alias Variables is flexible. To conserve application memory, you can define variables that are shorter than the values given in the table. Honeywell recommends that developers use the constructs provided by OpenDDA for portability and migration.

OpenDDA types supplied

OpenDDA provides predefined constructs that can be used in OpenDDA applications when declaring high-level language Alias Variables. These constructs are contained in the high-level language include files `dda_c.h` and `dda_fortran.f`. These constructs provide the user with the following benefits:

- Ease of use—the user is not burdened with having to remember specifics when defining high-level language Alias Variables.
- Portability/migration—by coding with these definitions, users will only need to recompile code if the underlying representation of Alias Variables should change.

C support provided

The following abstract data types are provided in the C language support include file `dda_c.h`. They can be used by OpenDDA application programmers to declare C language Alias Variables.

```
DDA_FULL_NAME_TYPE
DDA_POINT_NAME_TYPE
DDA_PARAM_NAME_TYPE
DDA_INDEX_NAME_TYPE
```

C example

```
DDA_FULL_NAME_TYPE    full_name_var;
DDA_POINT_NAME_TYPE   point_name_var;
DDA_PARAM_NAME_TYPE   param_name_var;
DDA_INDEX_NAME_TYPE   index_name_var;
```

**FORTTRAN
support provided**

The following parameters are in the FORTRAN language support include file `dda_fortran.f`. They can be used by OpenDDA application programmers to declare FORTRAN language Alias Variables.

```
DDA_FULL_NAME_SIZE  
DDA_POINT_NAME_SIZE  
DDA_PARAM_NAME_SIZE  
DDA_INDEX_NAME_SIZE
```

**FORTTRAN
example**

```
CHARACTER*(DDA_FULL_NAME_SIZE)    full_name_var  
CHARACTER*(DDA_POINT_NAME_SIZE)    point_name_var  
CHARACTER*(DDA_PARAM_NAME_SIZE)    param_name_var  
CHARACTER*(DDA_INDEX_NAME_SIZE)    index_name_var
```

5.4 Using Dynamic Selection Indicators

Overview In order to designate that a component of an External Data Reference is to be dynamically selected, a Dynamic Selection Indicator is used when the External Data Reference is defined in the External Data Statement. The Dynamic Selection Indicator is the asterisk character ("*"). The Dynamic Selection Indicator acts like a marker or placeholder, indicating that an Alias Variable will be defined for that component of the External Data Reference.

Example The following is an example of the use of a Dynamic Selection Indicator and an Alias Variable in an External Data Statement:

```
Var1 = 1.0/REAL  
Var1Status = INPUT CURRENT (AXM, *.PV)  
Var1Point_Name = POINT_NAME  
SET_LIST = Boiler1;
```

In this example, the External Reference for the Value Variable `Var1` is defined as `*.PV`. The "*" indicates that the point component of `Var1` can be dynamically selected at run time. An Alias Variable must be defined into which your application will load the desired point name prior to dynamic selection. The Alias Variable `Var1Point_Name` is defined in the third line. The External Data Statement is a member of the Data Access Set "Boiler1" and the Dynamic Selection Set "Boiler1."

Rules The following rules apply to the use of Dynamic Selection Indicators:

- For each Dynamic Selection Indicator present in an External Data Statement, there must be a corresponding Alias Variable defined in the External Data Statement.
- For each Alias Variable defined in an External Data Statement, there must be a corresponding Dynamic Selection Indicator.
- External data cannot be read or written until it has been dynamically selected by:
 - Loading Alias Variables with valid external component names
 - Executing an EXEC DDA MODIFY CIO statement

Combinations of indicators

The following table shows the permissible combinations of Dynamic Selection Indicators:

Table 5-4 Permissible Combinations of Dynamic Selection Indicators

Permissible Combination	Comments
*.param	External Data Statement must have a POINT_NAME Alias Variable.
*.param(1)	External Data Statement must have a POINT_NAME Alias Variable.
.	External Data Statement must have both POINT_NAME and PARAM_NAME Alias Variables.
.(*)	The External Data Statement must have POINT_NAME, PARAM_NAME, and INDEX_NAME Alias Variables or a FULL_NAME Alias Variable can be used. If a FULL_NAME Alias Variable is used, none of the other external_reference_keywords can be used.
.param()	External Data Statement must have a POINT_NAME and INDEX_NAME Alias Variables.
point.param(*)	External Data Statement must have must have an INDEX_NAME Alias Variable
point.*	External Data Statement must have a PARAM_NAME Alias Variable.
point.*(11)	External Data Statement must have a PARAM_NAME Alias Variable.
point.*(*)	External Data Statement must have PARAM_NAME and INDEX_NAME Alias Variables.

NULL External Reference Components

An External Data Reference in a Memory Resident CIO file is considered NULL under these conditions:

- an External Reference Component of an External Data Statement is specified with a Dynamic Selection Indicator and an EXEC DDA MODIFY CIO Execution Statement has not yet successfully acted upon the External Reference.
- an EXEC DDA MODIFY CIO Execution Statement has modified the External Data Reference with an External Reference Variable containing DDA_NULL.

Any component of an External Data Reference which is NULL causes the entire External Reference to be unusable - no data access can be performed.

**NULL vs.
DDA_NULL**

The following table illustrates the distinction between NULL and DDA_NULL.

Table 5-5 Distinction Between NULL and DDA_NULL

NULL	The value of the External Data Reference contained in the memory resident CIO file is undefined.
DDA_NULL	A special indicator that -- when placed into an External Reference Variable -- instructs EXEC DDA MODIFY CIO to make the External Data Reference contained in the memory resident CIO file NULL. DDA_NULL is a constant provided in the HLL include files.

**Initialization of
dynamic
components**

When the EDB is parsed and the CIO file created, all External Reference components will be set to NULL. During execution of the EXEC DDA INITIALIZE statement, OpenDDA reads these NULL External Reference components into the memory resident CIO. Data access cannot be performed for these External References until they are modified with an EXEC DDA MODIFY CIO Execution Statement.

Additional examples

The following examples illustrate the use, and misuse, of Alias Variables and Dynamic Selection Indicators:

Invalid Example:

```
Var1 = 1.0 /REAL  
Var1Status = INPUT CURRENT (AXM, A100.PV)  
Var1Point_Name = POINT_NAME  
SET_LIST = Boiler1;
```

In the above example, the Alias Variable Var1Point_Name cannot be defined because the Dynamic Selection Indicator "*" is not present for the point name.

Valid Example:

```
Var1 = 1.0 /REAL  
Var1Status = INPUT CURRENT (AXM, *.* )  
Var1Point_Name = POINT_NAME  
Var1Param_Name = PARAM_NAME  
SET_LIST = Boiler1;
```

In the above example, the Alias Variables Var1Point_Name and Var1Param_Name can be loaded with the point and parameter names, respectively, for the External Reference. The external data cannot be accessed until the names are dynamically selected, yet the associated Value Status Variables will indicate attempted data access actions through incomplete references.

Invalid Example:

```
Var1 = 1.0 /REAL  
Var1Status = INPUT CURRENT (AXM, *.* ( ))  
Var1Full_Name = FULL_NAME  
Var1Point_Name = POINT_NAME  
SET_LIST = Boiler1;
```

This is not allowed. A FULL_NAME Alias Variable and POINT_NAME (or any other) Alias Variable cannot both be specified in the same External Data Statement.

5.5 Alias Variable Arrays

Overview	Value Variables can be arrays, as opposed to scalar or single-valued entities. When a Value Variable is an array, the corresponding Alias Variable can be, but is not required to be, an array. Dynamic Selection and accessing of array Value Variables is accomplished through the use of indexes, which is covered in the next subsection.
Rules, properties, and relationships	If an Alias Variable is an array, the following apply: • The corresponding Value Variable must be an array, and must be specified using a subrange specification • The Alias Variable must be specified using the <code>FULL_NAME</code> keyword • The Alias Variable must be specified using a subrange specification • The subrange specifications of the Alias Variable and the Value Variable must be the same size • The actual subrange specifications do not have to be identical—they can have different starting (and ending) elements • A one-to-one relationship exists between the elements of the Alias Variable array and the Value Variable array—the lowest element in the Alias Variable array corresponds to the lowest element of the Value Variable array, the successive elements in each array correspond, and so forth • The contents of each element of the Alias Variable array will define the external reference of the corresponding Value Variable array element after the Dynamic Selection process

Using an Alias Variable array

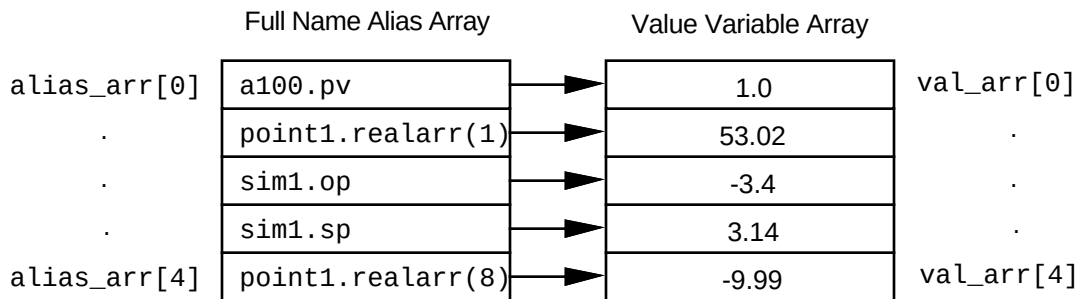
To use an Alias Variable array, the application program loads each element of the Alias Variable array with the desired External Reference Name, and performs an EXEC DDA MODIFY CIO operation.

Example:

Given the following 'C' External Data Statement:

```
...
val_arr[0:4] = 1.0 /REAL
val_arr_status[0:4] = INPUT CURRENT(*.*( *))
alias_arr[0:4] = FULL_NAME
alias_arr_sts[0:4] = EXT_REF_STATUS;
...
```

In the following figure, each of the five elements of the Alias Variable array contains the name of an External Reference. After Dynamic Selection through an EXEC DDA MODIFY CIO operation, an EXEC DDA READ operation will read the value of each of the five values of type REAL from the External References into the corresponding elements of the Value Variable array.



54016

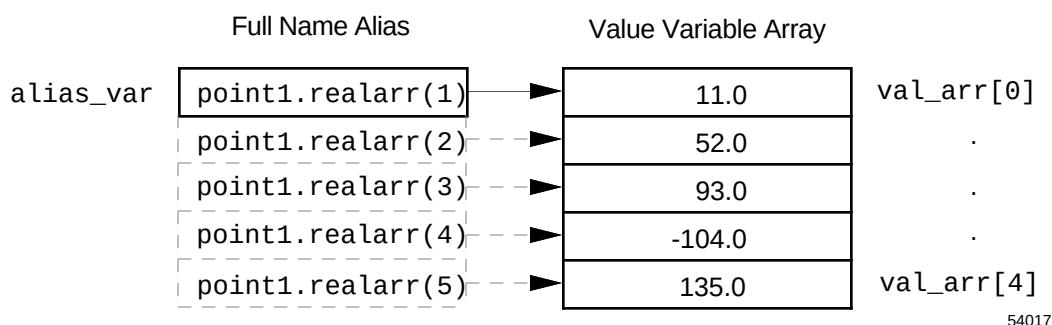
Using an Alias Variable that is not an array

Example:

Given the following 'C' External Data Statement:

```
...
val_arr[0:4] = 1.0 /REAL
val_arr_status[0:4] = INPUT CURRENT(*.*( *))
alias_arr = FULL_NAME
alias_arr_sts = EXT_REF_STATUS;
...
```

In the following figure, the FULL_NAME Alias Variable is not an array, and the Value Variable is an array. This configuration provides a subtle difference from the previous figure, in which both variables were arrays. This second configuration, which will be covered more fully in the next subsection dealing with indexes, illustrates "anchored" access of an external array. The external array will be sequentially accessed, beginning with the element indicated by the index value in the Alias Variable (in this case, 1), and the values accessed will be read into the Value Variable array, beginning with the first element, until the dimension of the Value Variable array is satisfied.



Declaring full name arrays

The following examples show how to declare Full Name Alias Arrays using the constructs provided in the include files dda_c.h and dda_fortran.f.

C Example:

```
DDA_FULL_NAME_TYPE full_name_var_arr[10];
```

FORTTRAN Example:

```
CHARACTER*(DDA_FULL_NAME_SIZE) full_name_arr(10)
```

Examples

The following examples illustrate the use, and misuse, of Alias Variable arrays.

Invalid Example:

```
Var1[0:9] = 1.0 /REAL
Var1Status[0:9] = INPUT CURRENT (AXM, *.PV)
Var1Point_Name[0:9] = POINT_NAME
SET_LIST = Boiler1;
```

The above example is invalid because POINT_NAME cannot have an Alias Variable array.

Invalid Example:

```
Var1[0:9] = 1.0 /REAL
Var1Status[0:9] = INPUT CURRENT (AXM, *.*(*))
Var1Full_Name[1:9] = FULL_NAME
SET_LIST = Boiler1;
```

The above example will generate a build time error since the subrange specification of Var1 and Var1Full_Name are not the same size.

Valid Example:

```
Var1[0:9] = 1.0 /REAL
Var1Status[0:9] = INPUT CURRENT (AXM, *.*(*))
Var1Full_Name[0:9] = FULL_NAME
SET_LIST = Boiler1;
```

In the above example, the Value Variable Var1 is an array of 10 elements. The full name External Reference for each of the 10 elements can be dynamically selected. The Alias Variable Var1Full_Name is an array of 10 elements. There is a one-to-one relationship between each element of Var1 and Var1Full_Name starting at element zero for each array. Var1 will be populated with 10 values, but they will come from as many as 10 external data locations.

Valid Example:

```
Var1[0:9] = 1.0 /REAL
Var1Status[0:9] = INPUT CURRENT (AXM, *.*(*))
Var1Full_Name[11:20] = FULL_NAME
SET_LIST = Boiler1;
```

The above example is almost identical to the previous example. The only difference is that the one-to-one relationship between each element of Var1 and Var1Full_Name is anchored at Var1[0] and Var1Full_Name[11]. An EXEC DDA READ will cause the External Reference specified in Var1Full_Name[11] to be accessed and the value will be stored in Var1[0]. Similarly, the External Reference specified in Var1Full_Name[12] will be accessed and the value will be stored in Var1[1], and so forth, until 10 values have been accessed.

Examples,
continued

Invalid Example:

```
Var1[0:9] = 1.0 /REAL  
Var1Status[0:9] = INPUT CURRENT (AXM,A100.* )  
Var1Param_Name[0:9] = PARAM_NAME  
SET_LIST = Boiler1;
```

The above example generates a build time error because an Alias Variable for a parameter can only be a scalar or a single element of an array.

Valid Example:

```
Var1[0:9] = 1.0 /REAL  
Var1Status[0:9] = INPUT CURRENT (AXM,A100.* )  
Var1Param_Name[0] = PARAM_NAME  
SET_LIST = Boiler1;
```

This example is similar to the previous example, except that the Alias Variable for the parameter is a single element of an array, which is permissible.

5.6 Loading Alias Variables

Overview

At any given point in time, the memory resident CIO contains the current External Reference Name for each Alias Variable. They can be changed any time by the OpenDDA application. This is how to change an External Reference:

- load a valid External Reference component into each of the Alias Variables, and
- execute an EXEC DDA MODIFY CIO Execution Statement.

This subsection describes how to load External Reference components into the Alias Variables.

External Reference components are strings

Point Name, Parameter Name, Index Name, and Full Name Alias Variables are ASCII strings. To load an External Reference component into the Alias Variable, simply use the constructs provided by the high-level language to place an ASCII string into the Alias Variable. The ASCII string that you place into the Alias Variable should be identical to the ASCII string that you would use in an External Data Statement to indicate an External Reference.

C examples

All Alias Variable strings are null-terminated. Null strings will be handled as invalid/incomplete references and are subject to the platform-specific resolution error handling.

```
strcpy(Point_Alias_Variable, "A100");
strcpy(Parameter_Alias_Variable, "COEFF");
strcpy(Index_Alias_Variable, "2");
strcpy(Full_Name_Alias_Variable, "A100.COEFF(2)");
```

Loading Alias Variables that include a Network Gateway (NG) name in C

When using a C program to load an Alias Variable of type FULL_NAME or POINT_NAME, the value to be loaded is copied into the Alias Variable using the strcpy function. If the string contains a backslash ('\') character, strcpy will treat it as an escape character. If a point component includes an NG name, it will include a backslash separating the NG name and the point name. Therefore, when using the strcpy function when the point name component has an NG name preceeding the point name, the NG name and point name must be separated by a double backslash ('\\'). This will then be treated as a single backslash internally.

Examples:

```
strcpy(full_name_alias_var, "SS\\A100.PV");
strcpy(point_name_alias_var, "SS\\B100");
```

NOTE: This is only true for C programs. FORTRAN programs use a single backslash.

**FORTTRAN
examples**

All Alias Variable strings should adhere to the normal FORTRAN rules for strings. Strings that are smaller in size than the declared string variable must be blank-filled.

```
Point_Alias_Variable = 'A100'  
Parameter_Alias_Variable = 'COEFF'  
Index_Alias_Variable = '2'  
Full_Name_Alias_Variable = 'A100.COEFF(2)'
```

**Loading
DDA_NULL**

If an Alias Variable is loaded with the predefined value DDA_NULL, an EXEC DDA MODIFY CIO does not treat this as an error condition. It will **not** force the overall status from the MODIFY to be HGSC_CHECK_STATUS. The External Reference Status Value will return DDA_EXTREF_INCOMPLETE. If a read or write is attempted, the Value Status or Store Status will return DDA_EXTREF_NULL. A NULL reference is treated as one that is not being used at this time. Therefore, the overall status of the call is not flagged with an error, but the individual status (external reference, value, store) variables return sufficient information for the user to detect NULL references at run time.

**Saving External
References**

Once External Reference components are loaded into the memory resident CIO, they can be saved to disk for future usage by using the EXEC DDA WRITE CIO function. (Refer to "WRITE CIO Statement" on page 150.) This procedure is the basis of the functionality known as "persistence" of selected External References.

5.7 Indexes and Dynamic Selection

Overview Indexes associated with External References can be dynamically selected at run time by using one of the following methods:

- FULL_NAME aliasing
- INDEX_NAME aliasing

System behavior factors When indexes are dynamically selected, the behavior of the system depends on several factors.

- Whether FULL_NAME or INDEX_NAME aliasing is used
- Whether the Alias Variable is an array
- Whether the Value Variable is an array
- Whether or not an index is used in a FULL_NAME Alias Variable

The various legal combinations of these factors, and the behavior associated with each, are summarized in the following table, with examples for each combination.

Table of combinations

The following table summarizes the legal combinations and associated system behavior. An example number is provided for each combination, referencing the relevant example.

Table 5-6 Summary of Index Usage

Method of Aliasing	Alias Variable is an Array?	Value Variable is an Array?	Properties/Behavior	Example Number
FULL_NAME	Yes	Yes	- The index is optional in each element of the Alias Variable array. - The Alias Variable array contains a list of individual External References. - You use the index when you want to access a single value from an External Reference that is an array—for example, <code>pt1.par1(4)</code>. - You don't use the index when you want to access a scalar External Reference—for example, <code>sim1.pv</code>.	1
	No	No	- The index is optional. - Provides single value access to either a scalar or a single element of an External Reference that is an array. Use of the index is the same as in the previous example, except that the Alias Variable contains a single External Reference name rather than a list.	2
	No	Yes	- The index must be included in the Alias Variable. - The index will anchor (specify the starting element of) access at the element specified (for example, <code>pt1.par1(1)</code>) and successive values will be accessed until the number of values accessed equals the size of the Value Variable array range.	3
INDEX_NAME	No	Yes	- The index must be specified and is used to access multiple values from an External Reference that is an array. - Access is anchored at the value specified in the index and successive values will be accessed until the number of values accessed equals the size of the Value Variable array range.	4
	No	No	The index must be specified and is used to access a single value from an External Reference that is an array.	5

Example 1

Valid Example:

```
Var1[0:9] = 1.0 /REAL  
Var1Status[0:9] = INPUT CURRENT (AXM, *.*(*) )  
Var1Full_Name[0:9] = FULL_NAME  
SET_LIST = Boiler1;
```

In this example, the Alias Variable array Var1Full_Name[0:9] is loaded with 10 independent External References. After an EXEC DDA MODIFY CIO operation, an EXEC DDA READ will access the 10 External Reference values and place them in the Value Variable array Var1[0:9]. The index is optional in the External Reference specification in each element of the Alias Variable array. If present, it is used to access a single element of an external array. If not present, a scalar is referenced.

Example 2

Valid Example:

```
Var1 = 1.0 /REAL  
Var1Status = INPUT CURRENT (AXM, *.*(*) )  
Var1Full_Name = FULL_NAME  
SET_LIST = Boiler1;
```

This is the single value access version of Example 1. In this case, neither the Value Variable nor the Alias Variable is an array. As in Example 1, the use of the index is optional. If present, it is used to access a single element of an external array. If not present, a scalar is referenced.

Example 3

Valid Example:

```
Var1[0:9] = 1.0 /REAL  
Var1Status[0:9] = INPUT CURRENT (AXM, *.*(*) )  
Var1Full_Name = FULL_NAME  
SET_LIST = Boiler1;
```

In this example, Var1Full_Name must contain an index because the Value Variable, Var1[0:9], is an array. An EXEC DDA READ will cause Var1[0:9] to be given 10 sequential values starting (anchored) at the location specified by the index in the Var1Full_Name Alias Variable.

Example 4

Valid Example:

```
Var1[0:9] = 1.0 /REAL  
Var1Status[0:9] = INPUT CURRENT (AXM, pt1.parm1(*) )  
Var1Index_Name = INDEX_NAME  
SET_LIST = Boiler1;
```

This example illustrates how to access an array of values from a dynamically specified External Reference. The index is used to determine the starting point (anchor) to READ from the pt1.parm1 array. The index is required.

Important consideration

Examples 3 and 4 illustrate the use of an anchor point when accessing an external array for reading/writing sequential values from/to a Value Variable that is also an array. The anchor serves to locate the starting element of the external array. There must be sufficient elements following the anchor point in the external array to match up with the subrange of the Value Variable, or the modify operation will fail for the associated Value Variable (even when some elements do match up.)

For example, assume we wish to read five values from an external array of 10 elements, `AMPOINT.CDSPAR(1..10)`, into a Value Variable array with five elements, `Var1[0:4]`. The index is used to specify the anchor point. Index values in the range of 1 to 6 are valid. For example, with an index value of 6, elements 6-10 of `AMPOINT.CDSPAR[1..10]` can be accessed. With an index of 7 or greater, however, the `AMPOINT` array subrange is exceeded, and a `MODIFY CIO` will fail, since the array is treated as a single reference.

Example 5

Valid Example:

```
Var1 = 1.0 /REAL
Var1Status = INPUT CURRENT (AXM, *.*(*) )
Var1Point_Name = POINT_NAME
Var1Param_Name = PARAM_NAME
Var1Index_Name = INDEX_NAME
SET_LIST = Boiler1;
```

In this example, a point name, parameter name, and index name must be modified (see `EXEC DDA MODIFY CIO`) before a valid `READ` is allowed. This type of EDB specification only allows for access of a single value contained in an external data location that is an array. The value specified in `Var1Index_Name` will determine which element of the array is accessed.

5.8 External Reference Status Variables

Overview An External Reference Status Variable is an HLL variable designated to receive a status code when dynamic selection is involved. It is used to indicate whether or not the EXEC DDA MODIFY CIO executed successfully for the External Data Statement to which it belongs.

ATTENTION

Usage of External Reference Status Variables is optional; however, Honeywell strongly recommends that all OpenDDA applications that use Dynamic Selection also make use of External Reference Status Variables, and check these variables after each EXEC DDA MODIFY CIO.

Syntax The External Reference Status Variable is defined using the External Reference Status keyword, EXT_REF_STATUS. Syntax of the External Reference Status Variable statement is:

```
ext_ref_status_variable = EXT_REF_STATUS
```

Rules The following rules apply when using External Reference Status Variables.

- When a FULL_NAME alias array is used, the External Reference Status Variable must also be an array of the same size.
- There can only be one External Reference Status Variable per External Data Statement.

Declaring External Reference Status Variables

The following examples show how to declare External Reference Status Variables. The C example uses the type provided in the include file dda_c.h. FORTRAN applications use INTEGER*4.

C Example:

```
DDA_STATUS_TYPE ext_ref_status_var;
```

FORTRAN Example:

```
INTEGER*4 ext_ref_status_var
```

External Reference Status values

Refer to “External Reference Status values returned” in MODIFY CIO Statement OpenDDA Execution Statements on page 126.

Examples

Valid Example:

```
Var1 = 1.0 /REAL
Var1Status = INPUT CURRENT (AXM, *.*(*) )
Var1Point_Name = POINT_NAME
Var1Param_Name = PARAM_NAME
Var1Index_Name = INDEX_NAME
Var1_Ext_Ref_Status = EXT_REF_STATUS
SET_LIST = Boiler1;
```

The above example is valid. The point name, parameter name, and index name variables combine to comprise a single External Reference. Var1_Ext_Ref_Status is the location that will receive status information resulting from an EXEC DDA MODIFY CIO.

Valid Example:

```
Var1[0:9] = 1.0 /REAL
Var1Status[0:9] = INPUT CURRENT (AXM, *.*(*) )
Var1Point_Name = POINT_NAME
Var1Param_Name = PARAM_NAME
Var1Index_Name = INDEX_NAME
Var1_Ext_Ref_Status = EXT_REF_STATUS
SET_LIST = Boiler1;
```

The above valid example is similar to the previous example, except that the Value Variable is an array. Although ten values will be read into Var1[0:9], the point, parameter, and index names still form a single External Reference, which acts to anchor access from an external array (see Example 4 in the subsection "Indexes and Dynamic Selection").

Valid Example:

```
Var1[0:9] = 1.0 /REAL
Var1Status[0:9] = INPUT CURRENT (AXM, *.*(*) )
Var1Full_Name[0:9] = FULL_NAME
Var1_Ext_Ref_Status[0:9] = EXT_REF_STATUS
SET_LIST = Boiler1;
```

The example above is valid. Ten individual External References specified are specified in the Alias Variable array Var1Full_Name[0:9]. The status from the EXEC DDA MODIFY CIO for each of these ten references will be placed in the corresponding elements of the array Var1_Ext_Ref_Status[0:9].

Invalid Example:

```
Var1[0:9] = 1.0 /REAL
Var1Status[0:9] = INPUT CURRENT (AXM, *.*(*) )
Var1Full_Name[0:9] = FULL_NAME
Var1_Ext_Ref_Status = EXT_REF_STATUS
SET_LIST = Boiler1;
```

This is not valid. The FULL_NAME Alias Variable has been defined as an array. Therefore either Var1_Ext_Ref_Status also needs to be an array (see the previous example), or Var1Full_Name can be specified to **not** be an array, acting as an anchor point, and matching Var1_Ext_Ref_Status.

Examples,
continued***Valid Example:***

```
Var1[0:9] = 1.0 /REAL  
Var1Status[0:9] = INPUT CURRENT (AXM, *.*(*) )  
Var1Full_Name = FULL_NAME  
Var1_Ext_Ref_Status = EXT_REF_STATUS  
SET_LIST = Boiler1;
```

This is valid. Ten values will be accessed starting at the value specified by the FULL_NAME alias. The FULL_NAME alias variable must have an index specified as the anchor where to begin obtaining values. Refer to “Example 3” in Indexes and Dynamic Selection EDB Dynamic Selection Variables on page 84.

5.9 Modify Code Variables

Overview A Modify Code Variable is an optional variable that can be used to determine whether or not External Reference names in the memory resident CIO will be updated with names in Alias Variables by the next EXEC DDA MODIFY CIO statement.

EDB Syntax Syntax of the Modify Code Variable statement is:
modify_code_variable = MODIFY_CODE

The Modify Code Variable can be loaded with either of two Modify Code constants that are defined in the include files dda_c.h and dda_fortran.f. The following table lists the available Modify Codes and indicates their effect on EXEC DDA MODIFY CIO processing.

Table 5-7 Modify Code Values

Modify Code	Effect on External Reference Name
DDA_MODIFY	Updated with contents of Alias Variable(s)
DDA_KEEP	No change

Rules The following rules apply when using Modify Code Variables.

- When a FULL_NAME alias array is used, the Modify Code Variable must also be an array of the same size.
- There can only be one Modify Code Variable per External Data Statement.

Validation of Modify Codes Modify Codes are validated during execution of the EXEC DDA MODIFY CIO statement. If an invalid code (one that is not shown in the above table) is detected, the following will occur.

- The External References contained in the associated Alias Variables will not be dynamically selected (the modify will not take place).
- Subsequent LOAD ALIAS execution statements will return previous External Reference component value.
- The External Reference Status will be set to DDA_EXTREF_INDETERMINATE.

Default If an Alias Variable does not have an associated Modify Code Variable, OpenDDA will use the value DDA_MODIFY.

Declaring Modify Code Variables

The following examples show how to declare Modify Code Variables. The C example uses the type provided in the include file `dda_c.h`. FORTRAN applications use `INTEGER*4`.

C Example:

```
DDA_STATUS_TYPE modify_code_var;
```

FORTTRAN Example:

```
INTEGER*4 modify_code_var
```

Examples

The following is an example of an EDB External Data Statement that includes a Modify Code Variable.

Valid Example:

```
Var1 = 1.0 /REAL
Var1Status = INPUT CURRENT (AXM, *.*(*) )
Var1Full_Name = FULL_NAME
Var1_Modify_Code = MODIFY_CODE
SET_LIST = Boiler1;
```

Following are some examples of code to load the Modify Code Variable.

Valid Example:

```
Var1_Modify_Code = DDA_MODIFY
```

In this example, the Modify Code Variable is being set to the Modify Code that will enable modification of the External Reference by an EXEC DDA MODIFY CIO.

Valid Example:

```
Var1_Modify_Code = DDA_KEEP
```

In this example, the Modify Code Variable is being set to the Modify Code that will "keep" the External Reference unchanged by an EXEC DDA MODIFY CIO.

Section 6—OpenDDA Execution Statements

6.1 Overview

Introduction	OpenDDA Execution Statements allow you to specify certain actions to take place in your applications. To accomplish this, you embed the OpenDDA Execution Statements in the source code of your applications.
OpenDDA precompiler	The OpenDDA precompiler expands OpenDDA Execution Statements at application build time into code that will perform the semantics of the statements at execution time.
Advantages of OpenDDA Execution Statements	OpenDDA Execution Statements provide several advantages: • The semantics of your OpenDDA applications will carry forward to future releases of OpenDDA without requiring changes to your OpenDDA source code — even though the underlying OpenDDA-created code that supports the execution statements may change. • You only need to be familiar with the semantics of the action. • You do <i>not</i> need to write source code that requires in-depth knowledge of OpenDDA functions or data structures.
Print options	If any of the DEF file print options (VALUES, STATS, or MESSAGES) are set ON, the appropriate information will be written to the General Output file.

6.2 OpenDDA Execution Statement Syntax

Elements of an EXEC DDA statement

The following table shows the elements that make up an OpenDDA Execution Statement.

Table 6-1 EXEC DDA Statement Elements

Item	Description	Examples
Introduction String	The string "EXEC DDA" signals the OpenDDA precompiler that an OpenDDA Execution Statement follows. There must be one and only one space between "EXEC" and "DDA".	EXEC DDA
Command	Indicates which action is to be performed by OpenDDA.	MODIFY
Qualifier	Tells OpenDDA what to act upon.	CIO
Keyword	Precedes a parameter (see below). It tells OpenDDA what kind of parameter follows.	SET STATUS
Parameter or Parameter List	Information supplied by the OpenDDA user. If a list, individual parameters are separated by commas.	exec_status 'Boiler_1' ('Boiler_1', 'Boiler_2')
Equal Sign (=)	"Connects" keywords and parameters.	SET='Boiler_1' STATUS=exec_status
Expression	Shorthand nomenclature for a Keyword=Parameter combination or Keyword=Parameter List combination	STATUS=exec_status SET='Boiler_1' SET=('Boiler_1', 'Boiler_2')
Comma(,)	Separates qualifiers, expressions, and elements of Parameter Lists from each other	SET='Boiler_1', STATUS=exec_status
Semicolon(;)	Statement terminator	STATUS=exec_status;

Required elements All Execution Statements include the following elements:

- Introduction String,
- Command, and
- Semicolon terminator.

The other elements only occur in some statements.

Order	The elements that are included in an Execution Statement must occur in the following order (some elements are not included in some statements): <Introduction String><Command><Qualifier>,<Expression>,...<Expression>;
Example	<pre>EXEC DDA MODIFY CIO, SET='Boiler_1', STATUS=exec_status;</pre>
Character strings	Use these rules to specify character strings: • Each character string passed to the OpenDDA Execution Statements must be enclosed in single quotation marks (') to differentiate a literal string from the name of an HLL variable. • When quotes are required, the phrase 'quoted character string literal' appears in the "type" definition for the parameter.
Brackets used in examples	In the OpenDDA Execution Statement examples in this section, brackets ([]) indicate that the enclosed item is optional.

6.3 Include File

Description

The constants and internal variables used by OpenDDA are defined in the OpenDDA HLL Include files:

```
/opt/DDA/include/dda_fortran.f (FORTRAN compiler)
/opt/DDA/include/dda_c.h       (ANSI C/C++ compiler)
```

These constants and variables must be defined in each module that executes an OpenDDA Execution Statement.

The OpenDDA application template supplies the correct Include file.

Referencing the Include file

The OpenDDA application templates supply the INCLUDE statement. If a FORTRAN or C application consists of more than one subroutine, then all subroutines that use EXEC DDA statements must contain the INCLUDE statement:

```
INCLUDE 'dda_fortran.f' (FORTRAN compiler)
#include "dda_c.h"       (ANSI C/C++ compiler)
```

This “relative” form of include requires the use of the following include path option on the compile command:

```
-I/opt/DDA/include
```

For an example, refer to the sample script file:

```
/opt/DDA/examples/dda_create_fortran_executable (FORTRAN)
/opt/DDA/examples/dda_create_c_executable       (C)
/opt/DDA/examples/dda_create_cpp_executable     (C++)
```

FORTTRAN include file example

The following FORTRAN example illustrates the location of a second include file.

```

PROGRAM TESTAPP
  INCLUDE 'dda_fortran.f'

  INTEGER*4 STATUS, READ_ROUTINE

  STATUS=READ_ROUTINE( )
  IF (IAND(STATUS,1) .NE. 1) THEN
    IF (STATUS .NE. HGSC_CHECK_STATUS) THEN
      .....error processing
    END IF
  END IF

  END

  INTEGER*4 FUNCTION READ_ROUTINE ( )
  INCLUDE 'dda_fortran.f'

  INTEGER*4 GET_STATUS
  REAL*4     VALUE

  C  EXTERNAL DATA BEGIN

  C      EXTDEF (AXM, DEFAULT)
  C      VALUE=12.34/REAL
  C      GET_STATUS=INPUT CURRENT (, TSTPOINT.PV)
  C      SET_LIST='SET_TEST';

  C  EXTERNAL DATA END

  EXEC DDA READ SET='SET_TEST',
    STATUS=READ_ROUTINE;

  RETURN
  END

```

6.4 EXEC Statement Summary

Commands The following is a list of OpenDDA Execution Statement commands:

- CLEAR
- HIBERNATE
- INITIALIZE
- LOAD
- MODIFY
- READ
- SELECT
- TERMINATE
- WRITE

Qualifiers The following is a list of OpenDDA Execution Statement command qualifiers:

- ALIAS
- CIO
- EVENT
- MODIFY_CODE
- STORE_CODE

Keywords The following is a list of OpenDDA Execution Statement keywords:

- EVENT_DATA
- FILE
- ID
- OVERWRITE
- SET
- STATUS
- VALUE

Summary of statements A summary of OpenDDA Execution statements follow, including the syntax of each.

CLEAR CIO EXEC DDA CLEAR CIO,
[ID=cio_id,]
STATUS=exec_status;

HIBERNATE EVENT	EXEC DDA HIBERNATE EVENT, EVENT_DATA=evt_data, STATUS=exec_status;
INITIALIZE	EXEC DDA INITIALIZE STATUS=exec_status;
LOAD ALIAS	EXEC DDA LOAD ALIAS, SET=(selection_set_name_1[,selection_set_name_2...]), STATUS=exec_status;
LOAD MODIFY_CODE	EXEC DDA LOAD MODIFY_CODE, VALUE=modify_code, STATUS=exec_status;
LOAD STORE_CODE	EXEC DDA LOAD STORE_CODE, SET=set_name, STATUS=exec_status;
MODIFY CIO	EXEC DDA MODIFY CIO, SET=(selection_set_name_1[,selection_set_name_2...]), STATUS=exec_status;
READ	EXEC DDA READ SET=set_name, STATUS=exec_status;
READ CIO	EXEC DDA READ CIO, [FILE=file_name,] [ID=cio_id,] STATUS=exec_status;
SELECT CIO	EXEC DDA SELECT CIO, [ID=cio_id,] STATUS=exec_status;
TERMINATE	EXEC DDA TERMINATE STATUS=exec_status;
WRITE	EXEC DDA WRITE SET=set_name, STATUS=exec_status;
WRITE CIO	EXEC DDA WRITE CIO, [FILE=file_name,] [ID=cio_id,] [OVERWRITE=overwrite_option,] STATUS=exec_status;

6.5 Summary of EXEC DDA Variables

Purpose of the variables

The Execution Statement Variables are used to interact with the EXEC DDA statements. These HLL variables allow the user to either pass data to the Execution Statement or receive data from the Execution Statement.

Summary of Execution Statement variables

The following table shows the format and gives a general description of the HLL variables as they would be used within the EXEC DDA statements. The variable names are the same as shown in the summary of the EXEC DDA statements. (Refer to 6.4 on page 96.) Each variable, as it is used within a specific EXEC DDA statement is described in subsections 6.7-6.19.

Table 6-2 OpenDDA Execution Statement Variables

Variable	Format	Description
exec_status	Integer of 4 bytes	Status of OpenDDA Execution Statement—returns the status of the OpenDDA Execution Statement. The status value indicates the overall status of the operation as well as the status of any external data access. Refer to 6.20.
set_name	String of up to 40 characters	Data Access Set Name—a user-defined name for a Data Access Set. This must be a literal string.
selection_set_name	String of up to 40 characters	Dynamic Selection Set Name—a user-defined name for a Dynamic Selection Set. This must be a literal string.
file_name	String of up to 255 characters	The CIO file name. The value is case sensitive. This can be a literal string or a string variable. An absolute path is allowed. If no path is specified, the current working directory is used.
cio_id	String of up to 40 characters	A reference identifier associated with a specific, memory resident CIO file. The value is not case sensitive. This can be a literal string or a string variable.
overwrite_option	Integer of 4 bytes	A flag indicating whether the specified file can be overwritten. This can be an integer constant or an integer variable.
modify_code	Integer of 4 bytes	Value that Modify Code variables are to be loaded with the LOAD MODIFY_CODE statement. This can be an integer constant or an integer variable.
evt_data	Structure of type DDA_HIBER_EVENT_TYPE1	Structure returned by an EXEC DDA HIBERNATE EVENT call. This must be a record structure (not a pointer to a record structure).

6.6 EXEC DDA Statement Notation for this Section

EXEC definition notation	This is an explanation of the notation used to describe the EXEC DDA statements' arguments in the following subsections.
DDA usage	DDA Usage – indicates how OpenDDA will use this argument. For example, DDA usage: exec_status will use this argument to return an execution status.
Type	Type – indicates the type of data OpenDDA expects to use. This is the actual data type. For example: Type: integer - 4 bytes (signed) is a 32-bit signed integer: • in FORTRAN it is INTEGER*4, or • in C or C++ it is long int
DDA data type	DDA Data Type – identifies type definition (or record structure) name in the HLL Include files for this argument data type. • C typedefs used to declare the variable • FORTRAN parameters or record structures used to declare the variable • "n/a" indicates that there is no definition for this type in the include file
Access	Access – indicates what access to the argument OpenDDA will use. The choices are: • Read Only – OpenDDA can only read the argument values • Write Only – OpenDDA can only write the argument values
Purpose	Purpose – is a short description of the argument.
Using the Status Mnemonic	Use the Status Mnemonics listed in the various status tables in the following sections to check for specific errors.

6.7 CLEAR CIO Statement

Description The OpenDDA Execution Statement CLEAR CIO is used to remove a CIO file from memory.

Format EXEC DDA CLEAR CIO,
[ID=cio_id,]
STATUS=exec_status;

[ID=cio_id] DDA Usage: cio_id
Type: string variable or quoted character string literal (max. of 40 characters)
DDA Data Type: n/a
Access: read only
Purpose: specifies the ID of the CIO file that is to be cleared

The variable `cio_id` is a literal character string surrounded by single quotes, or a string variable. The argument `ID=cio_id` indicates a memory resident CIO file that will be cleared. The argument is optional, and if not present, the ID defaults to the currently active memory resident CIO file.

STATUS=
exec_status

DDA Usage: exec_status
Type: Unsigned Integer (4 bytes)
DDA Data Type: C: DDA_STATUS_TYPE
FORTRAN: n/a

Access: write only
Purpose: receives the overall status for the CLEAR CIO operation

Table 6-3 Execution Status for the CLEAR CIO Statement

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	The CLEAR CIO operation was successful	None
DDA_NOACTCIO	There is no active CIO file	A memory resident CIO file must be made active (using SELECT CIO)
DDA_CIOIDTOOLONG	The CIO identifier was too long	Choose a shorter CIO identifier
DDA_INVCIOID	Invalid CIO ID	Choose a valid CIO ID
GDXS_MEMALLOCERR	Unable to allocate memory	Reduce the amount of memory needed by this application or try executing when the system is less loaded
Other	Other system or platform-specific status	Run dda_xstatus (in /opt/DDA/bin) to get the text message associated with the status value. Refer to Section 7 in the <i>OpenDDA User's Guide</i> for more information on OpenDDA statuses.

Declaring exec_status variables

Refer to “Declaring exec_status variables” on page 155.

Example

The following example shows the use of the EXEC DDA CLEAR CIO statement:

```
EXEC DDA CLEAR CIO,
      ID = 'tank1',
      STATUS = exec_status;
```

After the execution of the above statement, the memory resident CIO file that was associated with the CIO identifier 'tank1' will be cleared from memory and no longer accessible.

6.8 HIBERNATE EVENT Statement

Description The OpenDDA Execution Statement HIBERNATE EVENT causes the application to suspend execution awaiting an event initiated from an A^XM Control Language (CL) application. Either of the CL calls AMCL06\$Activate_Task or AMCL06\$Terminate_Task can cause the application to leave hibernation and resume execution. When the application resumes execution, it completes the HIBERNATE EVENT statement execution and returns the event data and status to the application. For additional information, refer to Appendix I of the *CL/AM Reference Manual*.

Format EXEC DDA HIBERNATE EVENT,
EVENT_DATA=evt_data,
STATUS=exec_status;

EVENT_DATA=evt_data DDA Usage: evt_data
Type: structure
DDA Data Type: C: DDA_HIBER_EVENT_TYPE1
FORTRAN: DDA_HIBER_EVENT_TYPE1
Access: write only
Purpose: Fills in the structure with the event data listed in the following table. This variable must be declared as a structure within the calling program. It cannot be a pointer to a structure.

Table 6-4 Elements of the DDA_HIBER_EVENT_TYPE1 Structure

Field Name	Type	Description
event_type	4-byte integer	An integer code that represents the type of the event (ACTIVATE, TERMINATE, or NO_EVENT)
event_status	4-byte integer	The status associated with the event function—see Table 6-6 on page 105
point_name	string of 19 characters	The entity that initiated the wakeup call
event_block	string of 8 characters	Name of the CL block that activated/terminated the application
init_block	string of 8 characters	Name of the CL block that initiated the application
event_time	DDA_TIME_TYPE	The HP-UX time when the event occurred
init_time	DDA_TIME_TYPE	The LCN time when the application was initiated
event_string	string of 80 characters	The CL event string

OpenDDA types supplied

OpenDDA provides predefined constructs that can be used in OpenDDA applications when declaring high-level language variables for the variables in the evt_data structure. These constructs are contained in the high-level language include files dda_c.h and dda_fortran.f. Honeywell recommends using these constructs to assure migration and portability. Note: FORTRAN programs use INTEGER*4 for event_type and event_status variables.

C Examples:

```
DDA_EVENT_TYPE event_type_var;
DDA_STATUS_TYPE event_status_var;
DDA_POINT_NAME_TYPE point_name_var;
DDA_BLOCK_NAME_TYPE event_block_var;
DDA_BLOCK_NAME_TYPE init_block_var;
DDA_TIME_TYPE event_time_var;
DDA_TIME_TYPE init_time_var;
DDA_CL_STRING_TYPE event_string_var;
```

FORTRAN Examples:

```
INTEGER*4 event_type_var
INTEGER*4 event_status_var
CHARACTER*(DDA_POINT_NAME_SIZE) point_name_var
CHARACTER*(DDA_BLOCK_NAME_SIZE) event_block_var
CHARACTER*(DDA_BLOCK_NAME_SIZE) init_block_var
RECORD /DDA_TIME_TYPE/ event_time_var
RECORD /DDA_TIME_TYPE/ init_time_var
CHARACTER*(DDA_CL_STRING_SIZE) event_string_var
```

STATUS=
exec_status

DDA Usage: exec_status
Type: Unsigned Integer (4 bytes)
DDA Data Type: C: DDA_STATUS_TYPE
FORTRAN: n/a
Access: write only
Purpose: receives the overall status for the HIBERNATE
EVENT operation

Table 6-5 Execution Status for the HIBERNATE EVENT Statement

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	The HIBERNATE EVENT operation was successful	None
HGSC_CHECK_STATUS	There was a problem accessing the event data	Check the event_status in the evt_data structure
DDA_INCONSISTERR	evt_data variable was declared incorrectly	Declare evt_data variable with the DDA_HIBER_EVENT_TYPE1 structure
GDXS_MEMALLOCERR	Unable to allocate memory	Reduce the amount of memory needed by this application or try executing when the system is less loaded
Other	Other system or platform-specific status	Run dda_xstatus (in /opt/DDA/bin) to get the text message associated with the status value. Refer to Section 7 in the <i>OpenDDA User's Guide</i> for more information on OpenDDA statuses.

**Declaring
exec_status
variables**

Refer to “Declaring exec_status variables” on page 155.

Event Status values

The following table shows the mnemonics for the status values that can be returned in the Event Status Variable field of the Event Data structure.

Table 6-6 Event Status Values

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	Action requested was successful	None
HGSC_NO_EVENT_REG	The application is not registered for events	Make sure that the application was initiated by a CL program using the AMCL06\$Initiate_Task call. Only then can the application hibernate.
HGSC_SERV_BUSY	The DCS communication is busy	Make sure the DCS communication is not overloaded. Reissue the command.

Example

The following C example shows the use of the EXEC DDA HIBERNATE EVENT statement:

```
...
DDA_HIBER_EVENT_TYPE1 evt_data;
DDA_STATUS_TYPE exec_status;
...
do
{
EXEC DDA HIBERNATE EVENT,
    EVENT_DATA = evt_data,
    STATUS=exec_status;
if (exec_status == HGSC_VALID)
{
switch (evt_data.event_type)
{
case DDA_EVENT_ACTIVATE:
    < do active activities >

    break;
case DDA_EVENT_TERMINATE:
    < do active terminate >

    break;

default:
    < error in data occurred >
}
}
}
```

```
        else  
            < error during HIBERNATE EVENT occurred >  
        }  
while (exec_status == HGSC_VALID &&  
       evt_data.event_type == DDA_EVENT_ACTIVATE);
```

In this example, the loop will continue to execute as long as the event type is "ACTIVATE."

6.9 INITIALIZE Statement

Description	The OpenDDA Execution Statement INITIALIZE sets up the OpenDDA environment for the application. This statement must be executed prior to any other EXEC DDA statement.
What INITIALIZE does	The INITIALIZE statement will • Initialize the structures needed by OpenDDA. • Load the definition file <i>application_name.def</i> from the Development Directory. If this file is not found, load the system default definition file <i>/opt/DDA/bin/dda_system.def</i>. If this file is not found, load internal default values (same as the values as in <i>/opt/DDA/bin/dda_system.def</i> as distributed). • Load the CIO file which defines the external data to be accessed (<i>application_name.cio</i>). If this file does not exist in the Development Directory, return a warning status. A .cio file must be read into the application before any data is accessed or a modify action is requested.
Location	The INITIALIZE statement must be the first EXEC statement that is executed. If the application consists of multiple modules, the INITIALIZE statement does not have to be in the main module (as in previous releases), but it must be executed before any other EXEC statements.
Format	<pre>EXEC DDA INITIALIZE STATUS=exec_status;</pre>
Development Directory	This is the directory where your OpenDDA application resides. This is usually the directory where the application is created and built. This directory contains the CIO file that supports your application. When the application executes, the General Output File is created here as well.

STATUS=
exec_status

DDA Usage: exec_status
 Type: Unsigned Integer (4 bytes)
 DDA Data Type: C: DDA_STATUS_TYPE
 FORTRAN: n/a

Access: write only
 Purpose: receives the overall status for the INITIALIZE operation

Table 6-7 Returned Status for the INITIALIZE Statement

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	The INITIALIZE operation was successful	None
DDA_INITIALRDYCALLED	Initialize statement already executed	Remove this INITIALIZE statement
DDA_SYSDEFFLUSED	System DEF file used	Informational status—no action required
DDA_SYSDFLTUSED	System default value used	Informational status—no action required
DDA_BADDEFFILE	DEF file contains errors	Correct the error(s) in the DEF file and run <code>dda -dv</code> . Refer to 5.12 in the <i>OpenDDA User's Guide</i> .
DDA_CIONOTFOUND	CIO file was not found—a CIO file will need to be read in before any data access can occur	Warning status—check for existence of the application CIO file in the application directory. It must have the same name as the application.
DDA_CIOSIGMISMATCH	A CIO signature mismatch was detected	Check that all EDBs have the same number of variables, in the same order by type. Refer to 8.2 in the <i>OpenDDA User's Guide</i> .
DDA_CIOSIGMISMATCHIGNORED	A CIO signature mismatch was detected, but was ignored because the CIO_SIGNATURE statement in the DEF file specified IGNORE	Informational status—no action required
DDA_PROCCREATFAIL	An error occurred trying to create a child process	Examine your system kernel parameter that determines the maximum number of processes each user can have. This may need to be increased.

Table 6-7 Returned Status for the INITIALIZE Statement

Status Mnemonic	Meaning	Corrective Action
DDA_NOAPPNAME	OpenDDA was unable to retrieve the executing application's name	Contact Honeywell Technical Assistance Center
GDXS_MEMALLOCERR	Unable to allocate memory	Reduce the amount of memory needed by this application or try executing when the system is less loaded
GDXS_WRONG_FILE_TYPE	The CIO file being read is not a valid CIO file	Make sure the application is referencing the proper CIO file
GDXS_FILE_VERSION_INCOMPATIBLE	The CIO file being read is not compatible with the current OpenDDA release	Recreate the CIO file. Refer to 5.9 of the <i>OpenDDA User's Guide</i> .
Other	Other system or platform-specific status	Run <code>dda_xstatus</code> (in <code>/opt/DDA/bin</code>) to get the text message associated with the status value. Refer to Section 7 in the <i>OpenDDA User's Guide</i> for more information on OpenDDA statuses.

Declaring exec_status variables

Refer to “Declaring exec_status variables” on page 155.

Example

The following FORTRAN example illustrates the use of the INITIALIZE DDA Execution Statement.

```
PROGRAM TESTAPP
INCLUDE 'dda_fortran.f'
.
.
C  EXTERNAL DATA BEGIN
...
C  EXTERNAL DATA END
.
.
INTEGER*4 exec_status

EXEC DDA INITIALIZE STATUS=exec_status;
```

6.10 LOAD ALIAS Statement

Description The EXEC DDA LOAD ALIAS will load Alias Variables for a specified Dynamic Selection Set or Sets with the corresponding External Reference component values and the associated External Reference Statuses that are stored in the active memory resident CIO. This execution statement allows applications to query or verify the dynamic External Reference component names and statuses. It also provides a mechanism to assist in making a large number of incremental changes or transferring contents of External References from one memory resident CIO to another.

The components of the External Reference are populated into the Alias Variables regardless of the current reference status. For example, if a current reference status is invalid due to illegal characters, the data will nevertheless be brought back into Alias Variables by the LOAD ALIAS execution.

The EXEC DDA LOAD ALIAS statement is part of the Dynamic Selection Sets functionality. For more information on Dynamic Selection Sets, refer to Section 9 “Dynamic Selection Sets” in the *OpenDDA User Guide*, and to Section 5, “EDB Dynamic Selection Variables,” in this publication.

Format

```
EXEC DDA LOAD ALIAS,
    SET = (selection_set_name_1 [,selection_set_name_2...] ),
    STATUS = exec_status;
```

SET=

DDA Usage:	selection_set_name
Type:	quoted character string literal (max. of 40 characters)
DDA Data Type:	n/a
Access:	read only
Purpose:	defines the name(s) of the OpenDDA Dynamic Selection Set(s) to be modified

(selection_set_name_1 [,selection_set_name_2...]) is a list of 1-100 selection_set_names. A selection_set_name is the name of a Dynamic Selection Set. It is a literal character string framed by single quotation marks. Parentheses around the list are required if more than one selection_set_name is specified. Parentheses are optional if only one selection_set_name is specified.

If selection_set_name is undefined At build time, if the selection_set_name does not match the name of a set defined in the EDB

- this statement will not be expanded into code
- an error message will be output and the expanded source file will not be created
- since the expanded source file is not created, it cannot be compiled or executed.

STATUS= DDA Usage: exec_status
exec_status Type: Unsigned Integer (4 bytes)
 DDA Data Type: C: DDA_STATUS_TYPE
 FORTRAN: n/a
 Access: write only
 Purpose: receives the overall status for the LOAD ALIAS operation

Table 6-8 Returned Status for the LOAD ALIAS Statement

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	The LOAD ALIAS operation was successful	None
DDA_INCONSISTERR	An external variable in the CIO file does not match the HLL Alias Variable in data size and/or array size	Make the Alias Variable match in the EDB and the HLL variable declaration
DDA_CIOSIGMISMATCH	A CIO signature mismatch was detected	Check that all EDBs have the same number of variables, in the same order by type. Refer to 8.2 in the <i>OpenDDA User's Guide</i> .
DDA_CIOSIGMISMATCHIGNORED	A CIO signature mismatch was detected, but was ignored because the CIO_SIGNATURE statement in the DEF file specified IGNORE	Informational status—no action required
DDA_NOACTCIO	There is no active CIO file	A memory resident CIO file must be made active (using SELECT CIO)
Other	Other system or platform-specific status	Run dda_xstatus (in /opt/DDA/bin) to get the text message associated with the status value. Refer to Section 7 in the <i>OpenDDA User's Guide</i> for more information on OpenDDA statuses.

Declaring exec_status variables

Refer to “Declaring exec_status variables” on page 155.

Example

This example illustrates overloading of Alias Variables across multiple EDB statements. Given the following EDB block:

```
Var1 = 1.0 /REAL
Var1_Status = INPUT CURRENT (AXM, *.* )
Var1_Point = POINT_NAME
Var1_Param = PARAM_NAME
SET_LIST = Set1;

Var1 = 1.0 /REAL
Var1_Status = INPUT CURRENT (AXM, *.* )
Var1_Point = POINT_NAME
Var1_Param = PARAM_NAME
SET_LIST = Set2;
```

and given the following C code:

```
strcpy(Var1_Point, "SIM1"); /* point name for 1st statemnt */
strcpy(Var1_Param, "PV"); /* param name for 1st statemnt */
```

```
EXEC DDA MODIFY CIO,
      SET='Set1',
      STATUS=exec_status;
```

After completion of the above statement, the first reference is SIM1.PV.

```
strcpy(Var1_Point, "SIM2"); /* point name for 1st statemnt */
strcpy(Var1_Param, "SP"); /* param name for 1st statemnt */
```

```
EXEC DDA MODIFY CIO,
      SET='Set2',
      STATUS=exec_status;
```

After completion of the above statement, the second reference is SIM2.SP

```
EXEC DDA LOAD ALIAS,
      SET='Set1',
      STATUS=exec_status;
```

After completion of the EXEC DDA LOAD ALIAS function, the Alias Variables Var1_Point and Var1_Param will contain "SIM1" and "PV" respectively.

```
EXEC DDA LOAD ALIAS,
      SET='Set2',
      STATUS=exec_status;
```

After completion of this EXEC DDA LOAD ALIAS function, the Alias Variables Var1_Point and Var1_Param will contain "SIM2" and "SP," respectively.

6.11 LOAD MODIFY_CODE Statement

Description	The EXEC DDA LOAD MODIFY_CODE statement will automatically load all defined Modify Code variables with a user-specified Modify Code. This execution statement is provided to allow applications to easily load all Modify Code variables with a specific Modify Code. The EXEC DDA LOAD MODIFY_CODE statement is part of the Dynamic Selection Sets functionality. For more information on Dynamic Selection Sets, refer to Section 9 “Dynamic Selection Sets” in the <i>OpenDDA User Guide</i> and to Section 5, “EDB Dynamic Selection Variables,” in this publication.
Format	<pre>EXEC DDA LOAD MODIFY_CODE, VALUE = modify_code, STATUS = exec_status;</pre>
VALUE= modify_code	DDA Usage: modify_code Type: Integer (4 bytes) DDA Data Type: C: DDA_CODE_TYPE FORTRAN: n/a Access: read only Purpose: specifies the value all Modify Code variables will be initialized to modify_code can be either of the following: • One of the Modify Code Value constants listed in Table 5-7 on page 89. • An HLL variable that is loaded with a Modify Code constant prior to executing the EXEC DDA LOAD MODIFY_CODE statement.
Function	The EXEC DDA LOAD MODIFY_CODE statement acts on the set 'ALL.' All Modify Code variables in the application will be loaded. If the Modify Code Variable is an array, all elements of the array will be loaded.

STATUS=
exec_status

DDA Usage: exec_status
Type: Unsigned Integer (4 bytes)
DDA Data Type: C: DDA_STATUS_TYPE
FORTRAN: n/a

Access: write only
Purpose: receives the overall status for the
LOAD MODIFY_CODE operation

Table 6-9 Returned Status for the LOAD MODIFY_CODE Statement

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	The LOAD MODIFY_CODE operation was successful	None
DDA_INCONSISTERR	An external variable in the CIO file does not match the HLL modify variable in data size and/or array size	Make the Modify Code variable match in the EDB and the HLL variable declaration
DDA_CIOSIGMISMATCH	A CIO signature mismatch was detected	Check that all EDBs have the same number of variables, in the same order by type. Refer to 8.2 in the <i>OpenDDA User's Guide</i> .
DDA_CIOSIGMISMATCHIGNORED	A CIO signature mismatch was detected, but was ignored because the CIO_SIGNATURE statement in the DEF file specified IGNORE	Informational status—no action required
DDA_NOACTCIO	There is no active CIO file	A memory resident CIO file must be made active (using SELECT CIO)
Other	Other system or platform-specific status	Run dda_xstatus (in /opt/DDA/bin) to get the text message associated with the status value. Refer to Section 7 in the <i>OpenDDA User's Guide</i> for more information on OpenDDA statuses.

**Declaring
exec_status
variables**

Refer to “Declaring exec_status variables” on page 155.

**Comparison to
other LOAD
qualifiers**

Unlike LOAD ALIAS and LOAD STORE_CODE, this function does not populate the HLL variables from the memory resident CIO file. Instead, the modify_code specified in the statement defines what value will be populated into all Modify Code variables.

Examples

The following are examples of the use of the EXEC DDA LOAD MODIFY_CODE statement.

```
EXEC DDA LOAD MODIFY_CODE,  
    VALUE = DDA_KEEP,  
    STATUS = exec_status;
```

After execution of the above statement, all Modify Code Variables in the application will be set to the value DDA_KEEP.

```
modify_code_var = DDA_MODIFY;  
EXEC DDA LOAD MODIFY_CODE,  
    VALUE = modify_code_var,  
    STATUS = exec_status;
```

After execution of the above statement, all Modify Code Variables in the application will be set to the value DDA_MODIFY.

6.12 LOAD STORE_CODE Statement

Description	The OpenDDA Execution Statement LOAD STORE_CODE will load your Store Code Variables with the values in the memory resident CIO file. The values in the memory resident CIO file were set by the DEF file INITIAL_VALUES directive, or, if a write was performed, they were set during the last write.
Format	EXEC DDA LOAD STORE_CODE, SET=set_name, STATUS=exec_status;
SET=set_name	DDA Usage: set_name Type: quoted character string literal (max. 40 characters) DDA Data Type: n/a Access: read only Purpose: defines the name of the OpenDDA Data Set to be read The set_name is a literal character string surrounded by single quotes. The set_name must match the name of a set defined in the EDB for this application or must be the string ALL (indicating all of the data defined in the EDB).
If set_name is undefined	At build time, if the set_name does not match the name of a set defined in the EDB • this statement will not be expanded into code • an error message will be output and the expanded source file will not be created • since the expanded source file was not created, it cannot be compiled or executed.

STATUS=
exec_status

DDA Usage: exec_status
Type: Unsigned Integer (4 bytes)
DDA Data Type: C: DDA_STATUS_TYPE
FORTRAN: n/a
Access: write only
Purpose: receives the overall status for the LOAD STORE_CODE statement

Table 6-10 Returned Status for the LOAD STORE_CODE Statement

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	The LOAD STORE_CODE operation was successful	None
DDA_NOACTCIO	There is no active CIO file	A memory resident CIO file must be made active (using SELECT CIO)
DDA_CIOSIGMISMATCH	A CIO signature mismatch was detected	Check that all EDBs have the same number of variables, in the same order by type. Refer to 8.2 in the <i>OpenDDA User's Guide</i> .
DDA_INCONSISTERR	An external variable in the CIO file does not match the HLL Store Code Variable in data size and/or array size	Make the Store Code Variable match in the EDB and the HLL variable declaration
GDXS_MEMALLOCERR	Unable to allocate memory	Reduce the amount of memory needed by this application or try executing when the system is less loaded
Other	Other system or platform-specific status	Run dda_xstatus (in /opt/DDA/bin) to get the text message associated with the status value. Refer to Section 7 in the <i>OpenDDA User's Guide</i> for more information on OpenDDA statuses.

**Declaring
exec_status
variables**

Refer to “Declaring exec_status variables” on page 155.

**What the LOAD
STORE_CODE
statement does**

The memory resident CIO file contains store codes for each value that can be written. The LOAD STORE_CODE statement loads your Store Code Variables with the store codes values contained in the memory resident CIO file.

**Memory resident
CIO file store
codes**

The following table illustrates how the memory resident store codes are modified.

Table 6-11 How Memory Resident CIO File Store Codes are Modified

This Execution Statement...	Gets the Store Code value from...
EXEC DDA INITIALIZE	The DEF file INITIAL_VALUE directive.
EXEC DDA READ CIO	The values contained in the CIO file read.
EXEC DDA WRITE	The Store Code Variables.

**When to use the
LOAD
STORE_CODE
statement**

You should use the LOAD STORE_CODE statement whenever you want your Store Code Variables to contain the same values as the memory resident CIO file store codes.

A common practice is to initialize all Store Code Variables to the value specified by the DEF file INITIAL_VALUE directive. To accomplish this, the LOAD STORE_CODE Execution Statement should be used after the INITIALIZE statement and before the first WRITE statement.

Example The following FORTRAN example illustrates the use of the LOAD STORE_CODE Execution Statement.

```
PROGRAM TESTAPP
INCLUDE 'dda_fortran.f'

INTEGER*4 EXEC_STATUS
INTEGER*4 GET_STATUS
INTEGER*4 STORE_CODE
REAL*4     VALUE
...
C  EXTERNAL DATA BEGIN
C  EXTDEF (AXM,DEFAULT)

C  VALUE=12.34/REAL
C    GET_STATUS=INPUT CURRENT (,TSTPOINT.PV)
C    STORE_CODE=OUTPUT
C    SET_LIST='SET_TEST';

C  EXTERNAL DATA END
...
EXEC DDA INITIALIZE STATUS=EXEC_STATUS
...
EXEC DDA LOAD STORE_CODE,
      SET='SET_TEST',
      STATUS=EXEC_STATUS;
...
```

6.13 MODIFY CIO Statement

Description	The EXEC DDA MODIFY CIO statement selects new External References for items that are members of the specified Dynamic Selection Set. The Alias Variables associated with the Selection Set(s) must be loaded with the desired External Reference names prior to invocation of this statement. A Modify Code Variable associated with an External Reference must be loaded with or must already contain the desired value. The EXEC DDA MODIFY CIO statement is part of the Dynamic Selection Sets functionality. For more information on Dynamic Selection Sets, refer to Section 9 “Dynamic Selection Sets” in the <i>OpenDDA User’s Guide</i>, and to Section 5— “EDB Dynamic Selection Variables,” in this publication.
Data access priority	The Data Access Priority specified in the DEF file is used when reading the data. Refer to “DATA_PRIORITY Directive” on page 166.
Format	<pre>EXEC DDA MODIFY CIO, SET = (selection_set_name_1 [,selection_set_name_2...]), STATUS = exec_status;</pre>
SET=	DDA Usage: selection_set_name Type: quoted character string literal (max. of 40 characters) DDA Data Type: n/a Access: read only Purpose: defines the name(s) of the OpenDDA Dynamic Selection Set(s) to be modified (selection_set_name_1 [,selection_set_name_2...]) is a list of 1-100 selection_set_names. A selection_set_name is the name of a Dynamic Selection Set. It is a literal character string framed by single quotation marks. Parentheses around the list are required if more than one selection_set_name is specified. Parentheses are optional if only one selection_set_name is specified.
If selection_set_name is undefined	At build time, if the selection_set_name does not match the name of a set defined in the EDB • this statement will not be expanded into code • an error message will be output and the expanded source file will not be created • since the expanded source file is not created, it cannot be compiled or executed.

STATUS=
exec_status

DDA Usage: exec_status
Type: Unsigned Integer (4 bytes)
DDA Data Type: C: DDA_STATUS_TYPE
FORTRAN: n/a

Access: write only
Purpose: receives the overall status for the MODIFY CIO operation

The exec_status indicates the overall status of the MODIFY CIO operation. If this status is HGSC_VALID, then none of the individual External Reference Statuses reflects an error. If this status indicates HGSC_CHECK_STATUS, then one or more of the individual External Reference Statuses not-valid External Reference Status, and the individual External Reference Statuses should be checked to determine which reference or references caused the problem. If exec_status is neither HGSC_VALID nor HGSC_CHECK_STATUS, then some portion of the MODIFY CIO completely failed and the exec_status indicates the reason.

Table 6-12 Returned Status for the MODIFY CIO Statement

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	The MODIFY CIO operation was successful	None
HGSC_CHECK_STATUS	An error occurred on one or more items for the specified set during the MODIFY operation	Check the individual External Reference Statuses to determine which one(s) failed and what the error was
DDA_INCONSISTERR	An external variable in the CIO file does not match an HLL variable in data size and/or array size	Make all HLL variables match in the EDB and the HLL variable declarations
DDA_CIOSIGMISMATCH	A CIO signature mismatch was detected	Check that all EDBs have the same number of variables, in the same order by type. Refer to 8.2 in the <i>OpenDDA User's Guide</i> .
DDA_CIOSIGMISMATCHIGNORED	A CIO signature mismatch was detected, but was ignored because the CIO_SIGNATURE statement in the DEF file specified IGNORE	Informational status—no action required
DDA_NOACTCIO	There is no active CIO file	A memory resident CIO file must be made active (using SELECT CIO)
GDXS_MEMALLOCERR	Unable to allocate memory	Reduce the amount of memory needed by this application or try executing when the system is less loaded

Table 6-12 Returned Status for the MODIFY CIO Statement

Status Mnemonic	Meaning	Corrective Action
GDXS_INITCHAN	Unable to initialize the communication link to the data	Make sure all A ^X M system components are executing (see subsection 2.1 in <i>Application Module^X Troubleshooting</i>)
GDXS_OPENCHAN	Unable to open the communication link to the data platform	Make sure all A ^X M system components are executing including the platform itself (see subsection 2.1 in <i>Application Module^X Troubleshooting</i>)
GDXS_NO_LOW_SERVERS	Unable to communicate with the data platform because there are no low priority servers configured	Configure a low priority server on the data platform or change the priority of the application (see Section 6, "Configuration of Data Access Priority" in the <i>A^XM User Guide</i>)
Other	Other system or platform-specific status	Run <code>dda_xstatus</code> (in <code>/opt/DDA/bin</code>) to get the text message associated with the status value. Refer to Section 7 in the <i>OpenDDA User's Guide</i> for more information on OpenDDA statuses.

Declaring exec_status variables

Refer to "Declaring exec_status variables" on page 155.

MODIFY CIO operation

The following actions take place as part of the EXEC DDA MODIFY CIO operation:

- The External Reference name components contained in the Alias Variables are moved into the memory resident CIO.
- All External References which were updated with new External Reference names are resolved and validated to establish a new External Data binding with the associated Value Variable.
- OpenDDA Status Values associated with the resolution and validation of the new External References are moved into the External Reference Status Variables defined.

If the new External References produce errors

If any External Reference does not successfully resolve and validate, the overall status returned in the `exec_status` variable will be `HGSC_CHECK_STATUS`, and the following will apply:

- The individual External Reference status variables will contain the detailed OpenDDA status value for each External Reference that was modified.
- The External Reference name kept in the memory resident CIO will contain the External Reference name that produced the error.

Effect of MODIFY CIO errors at data access time

Once an `EXEC DDA MODIFY CIO` operation is completed, a data access operation (`EXEC DDA READ` or `EXEC DDA WRITE`) can be performed to use the new External References to access External Data. For those External References that encountered errors during `EXEC DDA MODIFY CIO` processing, the following will apply:

- External Data will not be accessed for those Value Variables associated with the External References that caused the errors.
- `DDA_EXTREF_NOTBOUND` will be returned for the `value_status` or `store_status`. This indicates there is no External Reference binding for the associated `value_variable` due to the error encountered during the `EXEC DDA MODIFY CIO` processing. `DDA_EXTREF_NOTBOUND` is an error status.

Effect of NULL External References at data access time

By using the OpenDDA-provided constant `DDA_NULL`, you can perform an `EXEC DDA MODIFY CIO` operation which creates NULL External References. (Refer to “Loading `DDA_NULL`” in EDB Dynamic Selection Variables on page 81. This reference provides more information on using `DDA_NULL` to create NULL External References.)

When NULL External References are used at data access time (`EXEC DDA READ` or `EXEC DDA WRITE`), the following will apply:

- External Data will not be accessed for those Value Variables associated with the External References that were set to NULL.
- `DDA_EXTREF_NULL` will be returned for the `value_status` or `store_status`. This indicates the External Reference is NULL for the associated `value_variable`. `DDA_EXTREF_NULL` is not an error status—it is simply informational.

**Incremental
changes**

Incremental changes to External Reference Names can be made by limiting the scope of an EXEC DDA MODIFY CIO operation. There are two types of incremental changes that are made possible via two different methods:

Table 6-13 Techniques for Incremental Changes

To accomplish this...	Do this...
Limit the External Reference components of a given External Reference that get modified.	• Use EXEC DDA LOAD ALIAS to populate the Alias Variables with the current External Reference Components. • Only modify the Alias Variable associated with the component that is desired to be changed. Leave the other Alias Variables populated by the EXEC DDA LOAD ALIAS operation alone. • Perform the EXEC DDA MODIFY CIO operation.
Limit the number of External References in a given selection set that get modified.	• Set the Modify Code of those External References which you don't want changed to DDA_KEEP. • Perform the EXEC DDA MODIFY CIO operation

**External
Reference Status
values returned**

The following table shows the status values that can be returned in the External Reference Status Variable.

Table 6-14 Values of the External Reference Status Variable

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	No errors were detected.	None
DDA_EXTREF_INVALID	The dynamically selected External Reference name was invalid (for example, contained invalid characters or exceeded platform supported size). The item will not be accessed by a subsequent read or write operation and DDA_EXTREF_NOTBOUND will be returned in the associated Value Status or Store Status.	Check the corresponding External Reference Name components and remove unsupported characters and/or decrease their length.
DDA_EXTREF_INCOMPLETE	The dynamically selected External Reference name is not complete. One or more Alias Variables were not loaded with an External Reference name. The item will not be accessed by a subsequent read or write operation and the associated DDA_EXTREF_NULL will be returned in the associated Value Status or Store Status.	Make sure all corresponding External Reference Name components are being assigned prior to the EXEC DDA MODIFY CIO call.
DDA_EXTREF_INDETERMINATE	The dynamically selected External Reference is indeterminate. The item will not be accessed by a subsequent read or write operation and the associated DDA_EXTREF_NOTBOUND will be returned in the associated Value Status or Store Status.	Use valid Modify Codes for all Modify Code Variable assignments.
HGSC_BADARRIDX	Array index out of range	Specify a valid index in your Alias Variable for the associated External Reference defined on your External Data Platform.
HGSC_CVT_POINT	Point name not found	Specify a point in your Alias Variable that is a valid and defined reference on your External Data Platform.
HGSC_CVT_MATCH	Parameter name or parameter id was not found	Specify a parameter in your Alias Variable that is defined on your External Data Platform.

Table 6-14 Values of the External Reference Status Variable

Status Mnemonic	Meaning	Corrective Action
HGSC_CVT_NOPAR	Invalid parameter name	Specify a parameter in your Alias Variable that is valid for your External Data Platform.
HGSC_CVT_DUP	Duplicate point name found	Eliminate point name duplication on your External Data Platform.
HGSC_CVT_INDX	Internal error—too many items in list	Contact the Honeywell Technical Assistance Center.
HGSC_CVT_NG	Network Gateway is not supported on LCN	Make the specification of the Network Gateway in your Alias Variable the proper length and form.
HGSC_CVT_PTLEN	Point name length is not supported on the LCN	Change the point name specified in your Alias Variable to a length that is supported by your External Data Platform.
HGSC_CVT_ACCFAIL	Platform failed to return data.	Contact the Honeywell Technical Assistance Center.
HGSC_CVT_ARRAY	Array size too small.	Make the index specified in your Alias Variable within the range for the associated External Reference defined on your External Data Platform.
HGSC_CVT_TYPE	Invalid data type code	Make the data type of the associated External Reference as defined on your External Data Platform the same as the definition in your EDB.
HGSC_CVT_PARAMLEN	Parameter name length not supported on the LCN	Change the parameter name length specified in your Alias Variable to a length that does not exceed the length supported by your External Data Platform.
Other	Other system or platform-specific statuses	Run <code>dda_xstatus</code> (in <code>/opt/DDA/bin</code>) to get the text message associated with the status value. Refer to Section 7 in the <i>OpenDDA User's Guide</i> for more information on OpenDDA statuses.

Examples

The following are some examples of EXEC DDA MODIFY CIO statements.

Valid Example:

```
EXEC DDA MODIFY CIO,  
    SET = 'Boiler1',  
    STATUS = exec_status;
```

The above example is valid—parentheses are not required when one Selection Set is specified.

Valid Example:

```
EXEC DDA MODIFY CIO,  
    SET = ('Boiler1'),  
    STATUS = exec_status;
```

The above example is valid—parentheses are optional when one Selection Set is specified.

Valid Example:

```
EXEC DDA MODIFY CIO,  
    SET = ('Boiler1', 'Boiler2', 'Boiler3'),  
    STATUS = exec_status;
```

The above example is valid—parentheses are required when more than one Selection Set is specified.

6.14 READ Statement

Description	The OpenDDA Execution Statement READ reads all input data associated with the named Data Access Set. If the name of the Data Access Set is ALL, all data entities defined for input in the EDB for this application are read. After the data is read, the input HLL variables, specified for this Data Access Set in the EDB for the application, are updated with the new values.
Data access priority	The Data Access Priority specified in the DEF file is used when reading the data. Refer to “DATA_PRIORITY Directive” on page 166.
Format	EXEC DDA READ SET=set_name, STATUS=exec_status;
SET=set_name	DDA Usage: set_name Type: quoted character string literal (max. of 40 characters) DDA Data Type: n/a Access: read only Purpose: defines the name of the OpenDDA Data Set to be read The set_name is a literal character string surrounded by single quotes. The set_name must match the name of a set defined in the EDB for this application or must be the string ALL (indicating all of the data defined in the EDB).
If set_name is undefined	At build time, if the set_name does not match the name of a set defined in the EDB • this statement will not be expanded into code • an error message will be output and the expanded source file will not be created • since the expanded source file is not created, it cannot be compiled or executed.

**STATUS=
exec_status**

DDA Usage: exec_status
 Type: Unsigned Integer (4 bytes)
 DDA Data Type: C: DDA_STATUS_TYPE
 FORTRAN: n/a
 Access: write only
 Purpose: receives the overall status for the READ operation

The exec_status indicates the condition of all of the data values read. If this status is HGSC_VALID, then none of the individual Value Statuses reflects an error. If this status indicates HGSC_CHECK_STATUS, then one or more of the individual Value Statuses returned a non-valid Value Status, and the individual Value Statuses should be checked to determine which reference or references caused the problem. If exec_status is neither HGSC_VALID nor HGSC_CHECK_STATUS, then some portion of the READ completely failed and the exec_status indicates the reason.

Table 6-15 Returned Status for the READ Statement

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	The READ operation was successful	None
HGSC_CHECK_STATUS	An error occurred on one or more items for the specified set during the READ operation	Check the individual Value Statuses to determine which one(s) failed and what the error was
DDA_INITNOTDONE	An INITIALIZE statement was not executed prior to this READ	The INITIALIZE statement must be added or relocated so as to be executed before any other EXEC DDA statements
DDA_NOACTCIO	There is no active CIO file	A memory resident CIO file must be made active (using SELECT CIO)
DDA_CIOSIGMISMATCH	A CIO signature mismatch was detected	Check that all EDBs have the same number of variables, in the same order by type. Refer to 8.2 in the <i>OpenDDA User's Guide</i> .
DDA_INCONSISTERR	An external variable in the CIO file does not match the HLL Value Variable in data size and/or array size	Make the HLL variable match in the EDB and the HLL variable declaration
GDXS_MEMALLOCERR	Unable to allocate memory	Reduce the amount of memory needed by this application or try executing when the system is less loaded
GDXS_INITCHAN	Unable to initialize the communication link to the data	Make sure all A ^X M system components are executing (see subsection 2.1 in <i>Application Module^X Troubleshooting</i>)

Table 6-15 Returned Status for the READ Statement

Status Mnemonic	Meaning	Corrective Action
GDXS_OPENCHAN	Unable to open the communication link to the data platform	Make sure all A ^X M system components are executing including the platform itself (see subsection 2.1 in <i>Application Module^X Troubleshooting</i>)
GDXS_NO_LOW_SERVERS	Unable to communicate with the data platform because there are no low priority servers configured	Configure a low priority server on the data platform or change the priority of the application (see Section 6, "Configuration of Data Access Priority" in the <i>A^XM User Guide</i>)
Other	Other system or platform-specific status	Run <code>dda_xstatus</code> (in <code>/opt/DDA/bin</code>) to get the text message associated with the status value. Refer to Section 7 in the <i>OpenDDA User's Guide</i> for more information on OpenDDA statuses.

**Declaring
exec_status
variables**

Refer to "Declaring exec_status variables" on page 155.

**Value Status
values returned**

The following table lists the Value Status values and their messages. These status codes are returned in the `value_status_variable` by OpenDDA when your application is accessing external data using the EXEC DDA READ statement.

Table 6-16 Values of the Value Status Variable

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	Action request was successful	None
HGSC_DA_INTERNAL_ERR	Internal subsystem error found	Contact the Honeywell Technical Assistance Center.
HGSC_DA_ENUMCVT	No enumeration set is defined for the named parameter	Contact the Honeywell Technical Assistance Center.
HGSC_DA_FUNC	Function level error	Contact the Honeywell Technical Assistance Center.
HGSC_DA_DATATYPE	Data type error	Contact the Honeywell Technical Assistance Center.
HGSC_LCNCVTERR	An error occurred during LCN Point ID conversion	Contact the Honeywell Technical Assistance Center.
DDA_EXTREF_NULL	The external reference contained NULL components	Action based upon receipt of this status is application-specific; however, if this status is received for an application which does not expect it, then you should verify that an External Reference is being dynamically selected (EXEC DDA MODIFY CIO) prior to executing an EXEC DDA READ for the associated Value Variable.
DDA_EXTREF_NOTBOUND	The external reference is not bound	This status is returned as a result of an error during EXEC DDA MODIFY CIO when an External Reference could not be successfully resolved. You should examine the associated dynamic selection to determine possible corrective actions.
HGSC_NAN	Value received is an IEEE Not A Number.	Action based upon receipt of this status is application-specific. Examine the External Reference on your External Data Platform to view the value or to set the value to something other than NAN.
In addition to the statuses in this table, any of the error statuses listed in Table 6-15 on page 130 can be returned in the <code>value_status_variable</code> .		

Example

The following FORTRAN example illustrates the use of the READ DDA Execution Statement.

```

PROGRAM TESTAPP
INCLUDE 'dda_fortran.f'
  INTEGER*4 EXEC_STATUS
  INTEGER*4 GET_STATUS
  INTEGER*4 PUT_STATUS
  INTEGER*4 STORE_CODE
  REAL*4    VALUE
  .
  .
  .
C  EXTERNAL DATA BEGIN
C    EXTDEF (AXM,DEFAULT)
C    VALUE=12.34/REAL
C    GET_STATUS=INPUT CURRENT ( ,TSTPOINT.PV)
C    STORE_CODE=OUTPUT
C    PUT_STATUS=STORE_STATUS
C    SET_LIST='SET_TEST';
C  EXTERNAL DATA END
  .
  .
  .
  EXEC DDA READ SET='SET_TEST',
           STATUS=exec_status;
  IF(exec_status .NE. HGSC_VALID) THEN !ERROR DURING READ
    IF(exec_status .NE. HGSC_CHECK_STATUS) THEN
      ... ! Report the error
    ELSE !Some data is not valid
      .. ! Check individual value status
    END IF
  END IF
  .
  .
  .

```

6.15 READ CIO Statement

Description	The OpenDDA Execution Statement READ CIO reads a new CIO file from disk to memory. It automatically makes the CIO file that is read into memory the active CIO file.										
CIO Signature validation	Each CIO file will have a checksum associated with it, known as the "CIO Signature." The application also contains a CIO Signature. If the CIO_SIGNATURE directive is set to ERROR in the DEF file (or by default), the CIO Signature in the application is compared to the CIO Signature in the CIO file that is read in. These signatures must be the same or an error status is returned.										
Format	<pre>EXEC DDA READ CIO, [FILE=file_name,] [ID=cio_id,] STATUS=exec_status;</pre>										
[FILE=file_name]	<table><tr><td>DDA Usage:</td><td>file_name</td></tr><tr><td>Type:</td><td>string variable or quoted character string literal (max. of 255 characters)</td></tr><tr><td>DDA Data Type:</td><td>n/a</td></tr><tr><td>Access:</td><td>read only</td></tr><tr><td>Purpose:</td><td>defines the name of the CIO file to be read</td></tr></table> The <code>file_name</code> is a literal character string surrounded by single quotes or a string variable. The <code>file_name</code> is optional, and if not present, the CIO file <i>application_name.cio</i> will be read.	DDA Usage:	file_name	Type:	string variable or quoted character string literal (max. of 255 characters)	DDA Data Type:	n/a	Access:	read only	Purpose:	defines the name of the CIO file to be read
DDA Usage:	file_name										
Type:	string variable or quoted character string literal (max. of 255 characters)										
DDA Data Type:	n/a										
Access:	read only										
Purpose:	defines the name of the CIO file to be read										
[ID=cio_id]	<table><tr><td>DDA Usage:</td><td>cio_id</td></tr><tr><td>Type:</td><td>string variable or quoted character string literal (max. of 40 characters)</td></tr><tr><td>DDA Data Type:</td><td>n/a</td></tr><tr><td>Access:</td><td>read only</td></tr><tr><td>Purpose:</td><td>assigns a reference id to the CIO file</td></tr></table> The variable <code>cio_id</code> is a literal character string surrounded by single quotes or a string variable. It specifies a reference ID to associate with the CIO file being read into memory. The argument <code>ID=cio_id</code> is optional, and if not present, the ID will be assigned 'DDA_DEFAULT_CIO_ID.'	DDA Usage:	cio_id	Type:	string variable or quoted character string literal (max. of 40 characters)	DDA Data Type:	n/a	Access:	read only	Purpose:	assigns a reference id to the CIO file
DDA Usage:	cio_id										
Type:	string variable or quoted character string literal (max. of 40 characters)										
DDA Data Type:	n/a										
Access:	read only										
Purpose:	assigns a reference id to the CIO file										

STATUS=
exec_status

DDA Usage: exec_status
 Type: Unsigned Integer (4 bytes)
 DDA Data Type: C: DDA_STATUS_TYPE
 FORTRAN: n/a
 Access: write only
 Purpose: receives the overall status for the READ CIO operation

Table 6-17 Returned Status for the READ CIO Statement

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	The READ CIO operation was successful	None
DDA_FILENAMETOOLONG	The pathname specified was too long	The CIO file must be specified with a shorter pathname
DDA_CIOIDTOOLONG	The CIO identifier was too long	Choose a shorter CIO identifier
DDA_CIONOTFOUND	The CIO file specified was not found	Check the pathname of the CIO file specified
DDA_CIOSIGMISMATCH	A CIO signature mismatch was detected	Check that all EDBs have the same number of variables, in the same order by type. Refer to 8.2 in the <i>OpenDDA User's Guide</i> .
DDA_CIOSIGMISMATCHIGNORED	A CIO signature mismatch was detected, but was ignored because the CIO_SIGNATURE statement in the DEF file specified IGNORE	Informational status - no action required
GDXS_MEMALLOCERR	Unable to allocate memory	Reduce the amount of memory needed by this application or try executing when the system is less loaded
GDXS_WRONG_FILE_TYPE	The CIO file being read is not a valid CIO file	Make sure the application is referencing the proper CIO file
GDXS_FILE_VERSION_INCOMPATIBLE	The CIO file being read is not compatible with the current OpenDDA release	Recreate the CIO file. Refer to 5.9 of the <i>OpenDDA User's Guide</i> .
Other	Other system or platform-specific status	Run dda_xstatus (in /opt/DDA/bin) to get the text message associated with the status value. Refer to Section 7 in the <i>OpenDDA User's Guide</i> for more information on OpenDDA statuses.

**Declaring
exec_status
variables**

Refer to “Declaring exec_status variables” on page 155.

READ CIO usage illustrations

The following table illustrates what happens during certain scenarios, and indicates the side effects that occur.

IF this is true...	THEN this occurs...
The CIO file is read successfully	- Any memory resident CIO file previously associated with the cio_id used is cleared before the new CIO file is read into memory - The CIO file just read becomes the active memory resident CIO file - STATUS returns the good status HGSC_VALID
The CIO file is read with errors	- The cio_id is cleared and not available for selection - STATUS returns the error status GDX_WRONG_FILE_TYPE
The CIO file to be read is not found in the current directory	- The cio_id is cleared and not available for selection - STATUS returns the "file not found" error DDA_CIONOTFOUND
The CIO file is read successfully, but the CIO Signature does not match (and the CIO_SIGNATURE directive is set to ERROR)	- The cio_id is cleared and not available for selection - STATUS returns with the "CIO signature mismatch" error DDA_CIOSIGMISMATCH - Subsequent actions on memory resident CIO are not allowed
The CIO file is read successfully, but the CIO Signature does not match (and the CIO_SIGNATURE directive is set to IGNORE)	- Any memory resident CIO file previously associated with the cio_id used is cleared before the new CIO file is read into memory - The CIO file just read becomes the active memory resident CIO file - STATUS returns with the "CIO signature mismatch ignored" informational status DDA_CIOSIGMISMATCHIGNORED - Subsequent actions on memory resident CIO are allowed, but inconsistency errors are possible

Example

The following example shows the use of the EXEC DDA READ CIO statement:

```
EXEC DDA READ CIO,
      FILE='tank1.cio',
      ID = 'tank1',
      STATUS=exec_status;
```

After the execution of the above statement, the CIO file 'tank1.cio' will be read into memory. It will be associated with the reference ID 'tank1'. In addition, it will become the active memory resident CIO file.

6.16 SELECT CIO Statement

Description The OpenDDA Execution Statement SELECT CIO is used to determine which memory resident CIO file is to be active.

Format

```
EXEC DDA SELECT CIO,  
      [ID=cio_id,]  
      STATUS=exec_status;
```

[ID=cio_id]

DDA Usage:	cio_id
Type:	string variable or quoted character string literal (max. of 40 characters)
DDA Data Type:	n/a
Access:	read only
Purpose:	specifies the ID of the CIO file that is to be active

The variable `cio_id` is a literal character string surrounded by single quotes, or a string variable. The argument `ID=cio_id` is optional, and if not present, the ID defaults to 'DDA_DEFAULT_CIO_ID.' The 'cio_id' indicates the reference ID associated with a memory resident CIO file that will be active. If the ID specified is not found, an error is returned and the currently active CIO file will remain active.

STATUS=
exec_status

DDA Usage: exec_status
Type: Unsigned Integer (4 bytes)
DDA Data Type: C: DDA_STATUS_TYPE
FORTRAN: n/a
Access: write only
Purpose: receives the overall status for the SELECT CIO operation

Table 6-18 Execution Status for the SELECT CIO Statement

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	The SELECT CIO operation was successful	None
DDA_CIOIDTOOLONG	The CIO identifier was too long	Choose a shorter CIO identifier
DDA_INVCIOD	Invalid CIO ID	Verify that the CIO ID is valid
GDXS_MEMALLOCERR	Unable to allocate memory	Reduce the amount of memory needed by this application or try executing when the system is less loaded
Other	Other system or platform-specific status	Run <code>dda_xstatus</code> (in <code>/opt/DDA/bin</code>) to get the text message associated with the status value. Refer to Section 7 in the <i>OpenDDA User's Guide</i> for more information on OpenDDA statuses.

**Declaring
exec_status
variables**

Refer to “Declaring exec_status variables” on page 155.

**Impact on HLL
variables**

A successful SELECT CIO action has no impact on the current values held in high-level language variables. For example, Value Variable and Value Status Variables are those populated during the last READ action. Store Codes and Store Status variables are those depopulated/populated during the last WRITE action.

Alias Variables and External Reference Status Variables are those populated during the last LOAD ALIAS action or depopulated during the last MODIFY CIO action. If incremental changes are desired, the user may populate their variables using the LOAD ALIAS action, which will bring back the current memory resident CIO to the HLL variables.

Example

The following example shows the use of the EXEC DDA SELECT CIO statement:

```
EXEC DDA SELECT CIO,  
    ID = 'tank1',  
    STATUS=exec_status;
```

After the execution of the above statement, the memory resident CIO file that is associated with the CIO identifier 'tank1' will be the active memory resident CIO file.

6.17 TERMINATE Statement

Description	The OpenDDA Execution Statement TERMINATE performs a shutdown of the OpenDDA environment for an application. TERMINATE is the last EXEC DDA statement to be executed.
What TERMINATE does	The TERMINATE statement will: • release OpenDDA structures allocated during INITIALIZE • clear all memory resident CIO files • terminate any connection established with the DCS.
Location	The TERMINATE statement must be the last EXEC statement that is executed. If the application consists of multiple modules, the TERMINATE statement is not required to be in the main module (as in prior releases).
Format	<pre>EXEC DDA TERMINATE STATUS=exec_status;</pre>
STATUS= exec_status	DDA Usage: exec_status Type: Unsigned Integer (4 bytes) DDA Data Type: C: DDA_STATUS_TYPE FORTRAN: n/a Access: read only Purpose: identifies the exit status for this application

Table 6-19 Returned Status for the TERMINATE Statement

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	The TERMINATE operation was successful	None
DDA_TERMNOTNEEDED	This TERMINATE is unnecessary	Remove this TERMINATE statement
GDXS_MEMALLOCERR	Unable to allocate memory	Reduce the amount of memory needed by this application or try executing when the system is less loaded
Other	Other system or platform-specific status	Run dda_xstatus (in /opt/DDA/bin) to get the text message associated with the status value. Refer to Section 7 in the <i>OpenDDA User's Guide</i> for more information on OpenDDA statuses.

**Declaring
exec_status
variables**

Refer to “Declaring exec_status variables” on page 155.

Example

The following FORTRAN example illustrates the use of the TERMINATE DDA Execution Statement.

```
PROGRAM TESTAPP
INCLUDE 'dda_fortran.f'
...
C  EXTERNAL DATA BEGIN
...
C  EXTERNAL DATA END
...
C  Terminating OpenDDA environment
  EXEC DDA TERMINATE STATUS=EXEC_STATUS;
...
```

6.18 WRITE Statement

Description The OpenDDA Execution Statement WRITE writes all output data associated with the named Data Access Set. If the name of the Data Access Set is ALL, all data entities defined for output in the EDB for this application are written.

The values contained in the HLL variables, specified for this Data Access Set in the EDB for the application, are written. If the store status was specified in the EDB, it is also updated.

Whether or not values are actually written to the LCN is controlled by the Store Code and the security configuration in the Application Module^x (refer to your A^xM manual and 3.8 in the *OpenDDA User's Guide*).

Data access priority The Data Access Priority specified in the DEF file is used when writing the data. Refer to "DATA_PRIORITY Directive" on page 166.

Controlling writing of elements of a set Each Value Variable to be written has a Store Code Variable associated with it – the variable preceding the OUTPUT directive.

The Store Code Variable value controls whether the Value Variable will actually be written.

IF Store Code Variable is...	THEN the action is...
DDASC_STORE	Store the value to the external location.
DDASC_NO_STORE	The value will not be written.
DDASC_STORE_NAN	The Not-a-Number value will be written to the external location. This is only supported for REAL data types.
Refer to 4.12 for more information on Store Codes.	

Format EXEC DDA WRITE SET=set_name,
STATUS=exec_status;

SET=set_name DDA Usage: set_name
Type: quoted character string literal (max. 40 characters)
DDA Data Type: n/a
Access: read only
Purpose: defines the name of the OpenDDA Data Set to be read

The set_name is a literal character string surrounded by single quotes. The set_name must match the name of a set defined in the EDB for this application or must be the string ALL (indicating all of the data defined in the EDB).

If set_name is undefined

At build time, if the `set_name` does not match the name of a set defined in the EDB

- this statement will not be expanded into code
- an error message will be output and the expanded source file will not be created
- since the expanded source file is not created, it cannot be compiled or executed.

**STATUS=
exec_status**

DDA Usage: `exec_status`

Type: Unsigned Integer (4 bytes)

DDA Data Type: C: `DDA_STATUS_TYPE`

FORTRAN: n/a

Access: write only

Purpose: receives the overall status for the WRITE operation

The `exec_status` indicates the condition of all of the Store Statuses from the WRITE. If this status is `HGSC_VALID`, then all of the individual data value writes returned a valid Store Status. If this status is `HGSC_CHECK_STATUS`, then one or more of the individual data value writes returned a nonvalid Store Status, and the individual Store Statuses should be checked to determine which reference or references caused the problem. If this status is neither `HGSC_VALID` nor `HGSC_CHECK_STATUS`, then some portion of the WRITE completely failed and the `exec_status` indicates the reason.

Table 6-20 Returned Status for the WRITE Statement

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	The WRITE operation was successful	None
HGSC_CHECK_STATUS	An error occurred on one or more items for the specified set during the WRITE operation	Check the individual Store Statuses to determine which one(s) failed and what the error was
DDA_INITNOTDONE	An INITIALIZE statement was not executed prior to this READ	The INITIALIZE statement must be added or relocated so as to be executed before any other EXEC DDA statements
DDA_NOACTCIO	There is no active CIO file	A memory resident CIO file must be made active (using SELECT CIO)
DDA_CIOSIGMISMATCH	A CIO signature mismatch was detected	Check that all EDBs have the same number of variables, in the same order by type. Refer to 8.2 in the <i>OpenDDA User's Guide</i> .
DDA_INCONSISTERR	An external variable in the CIO file does not match the HLL variable in data size and/or array size	Make the HLL variable match in the EDB and the HLL variable declaration

Table 6-20 Returned Status for the WRITE Statement

Status Mnemonic	Meaning	Corrective Action
GDXS_MEMALLOCERR	Unable to allocate memory	Reduce the amount of memory needed by this application or try executing when the system is less loaded
GDXS_INITCHAN	Unable to initialize the communication link to the data	Make sure all A ^X M system components are executing
GDXS_OPENCHAN	Unable to open the communication link to the data platform	Make sure all A ^X M system components are executing including the platform itself
GDXS_NO_LOW_SERVERS	Unable to communicate with the data platform because there are no low priority servers configured	Configure a low priority server on the data platform or change the priority of the application (see Section 6, "Configuration of Data Access Priority" in the <i>A^XM User Guide</i>)
Other	Other system or platform-specific status	Run <code>dda_xstatus</code> (in <code>/opt/DDA/bin</code>) to get the text message associated with the status value. Refer to Section 7 in the <i>OpenDDA User's Guide</i> for more information on OpenDDA statuses.

Declaring exec_status variables

Refer to "Declaring exec_status variables" on page 155.

**Store Status
values returned**

The following table lists the LCN data access generic status codes and their messages. These status codes are returned in the store_status_variable by OpenDDA when your application is accessing external data using the EXEC DDA WRITE statement.

Table 6-21 Values of the Store Status Variable

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	Action request was successful	None
HGSC_DA_INTERNAL_ERR	Internal subsystem error found	Contact the Honeywell Technical Assistance Center.
HGSC_DA_TOOLONG	String was too long, truncated value was stored	Contact the Honeywell Technical Assistance Center.
HGSC_DA_STORETYP	Value type presented for store was not the expected type	Contact the Honeywell Technical Assistance Center.
HGSC_DA_ENUMCVT	No enumeration set is defined for the named parameter	Contact the Honeywell Technical Assistance Center.
HGSC_DA_FUNC	Function level error	Contact the Honeywell Technical Assistance Center.
HGSC_DA_DATATYPE	Data type error	Contact the Honeywell Technical Assistance Center.
HGSC_LCNCVTERR	An error occurred during LCN Point ID conversion	Contact the Honeywell Technical Assistance Center.
HGSC_NAN	Value received is an IEEE Not A Number.	Action based upon receipt of this status is application-specific. Examine the External Reference on your External Data Platform to view the value or to set the value to something other than NAN.
HGSC_NAN_INVALID	Store NAN not supported for specified data type	Do not attempt to store NAN for this data type
HGSC_DA_VALRANGE	Value out of range, store not performed	Contact the Honeywell Technical Assistance Center.
HGSC_DA_SDESTORE	SDE store not allowed	Contact the Honeywell Technical Assistance Center.
HGSC_CVT_POINT	Point name not found	Verify that the ASCII External ID you are trying to store is defined for your associated External Data Platform.
HGSC_CVT_PTLEN	Point name length is not supported on the LCN	Verify that the ASCII External ID you are trying to store does not exceed the length supported by your External Data Platform.

Table 6-21 Values of the Store Status Variable

Status Mnemonic	Meaning	Corrective Action
HGSC_CVT_NG	Network Gateway is not supported on the LCN	Verify that the Network Gateway specification used in the ASCII External ID you are trying to store is supported by your External Data Platform.
HGSC_NULL_VALUE	This status will be returned when the associated Store Code Variable is set to DDASC_NO_STORE prior to the EXEC DDA WRITE. This indicates no value was stored to the External Data Platform.	The action to take upon receipt of this status is application-specific.
DDA_EXTREF_NULL	The external reference contained NULL components	Action based upon receipt of this status is application-specific; however, if this status is received for an application which does not expect it, then you should verify that an External Reference is being dynamically selected (EXEC DDA MODIFY CIO) prior to executing an EXEC DDA READ for the associated Value Variable.
DDA_EXTREF_NOTBOUND	The external reference is not bound	This status is returned as a result of an error during EXEC DDA MODIFY CIO when an External Reference could not be successfully resolved. You should examine the associated dynamic selection to determine possible corrective actions.
DDA_STORE_CODE_INVALID	Invalid store code specified	Correct the store code specified
In addition to the statuses in this table, any of the error statuses listed in Table 6-20 on page 144 can be returned in the store_status_variable.		

Example

The following FORTRAN example illustrates the use of the WRITE DDA Execution Statement.

```

PROGRAM TESTAPP
  INCLUDE 'dda_fortran.f'

  INTEGER*4 EXEC_STATUS
  INTEGER*4 GET_STATUS
  INTEGER*4 PUT_STATUS
  INTEGER*4 STORE_CODE
  REAL*4 VALUE

  .
  .
  .
C  EXTERNAL DATA BEGIN

C    EXTDEF (AXM,DEFAULT)
C    VALUE=12.34/REAL
C    GET_STATUS=INPUT CURRENT (,TSTPOINT.PV)
C    STORE_CODE=OUTPUT
C    PUT_STATUS=STORE_STATUS
C    SET_LIST='SET_TEST';

C  EXTERNAL DATA END
  .
  .
  .
C  Set Store Code to enable the write
  STORE_CODE=DDASC_STORE
EXEC DDA WRITE SET='SET_TEST',
STATUS=EXEC_STATUS;
  IF(EXEC_STATUS .NE. HGSC_VALID) THEN !ERROR DURING WRITE
    IF(EXEC_STATUS .NE. HGSC_CHECK_STATUS) THEN
      ... ! Report the error
    ELSE ! Some data is not valid
      ... ! Check individual store status
    END IF
  END IF

  .
  .
  .

```

**Writing External
IDs in C**

When using a C program to write out an entity name to variable of type `Entity_Id`, the value to be written is usually copied into the Value Variable using the `strcpy` function. If the string contains a backslash ('\') character, `strcpy` will treat it as an escape character. If the entity name has an NG name preceding the point name, then the NG name and point name must be separated by a double backslash ('\\'). This will then be treated as a single backslash internally.

Example:

To copy the entity name "SS\A100" into a Value Variable:

```
strcpy(External_Id_Variable, "SS\\A100");
```

NOTE: This is only true for C programs. FORTRAN programs use a single backslash.

6.19 WRITE CIO Statement

Description	The OpenDDA Execution Statement WRITE CIO writes a memory resident CIO file to disk. This action does not change which CIO file is currently active.		
CIO Signature unchanged	Since a user application is only able to change external references, the CIO Signature of the CIO that is written will be the same value as it was when the CIO was read into memory.		
Format	<pre>EXEC DDA WRITE CIO, [FILE=file_name,] [ID=cio_id,] [OVERWRITE=overwrite_option,] STATUS=exec_status;</pre>		
[FILE=file_name]	DDA Usage:	file_name	
	Type:	string variable or quoted character string literal (max. of 255 characters)	
	DDA Data Type:	n/a	
	Access:	read only	
	Purpose:	defines the filename to write the CIO file to	
	The <code>file_name</code> is a literal character string surrounded by single quotes, or a string variable. The argument <code>file_name</code> is optional, and if not present, the CIO filename associated with the <code>cio_id</code> specified will be used. If <code>cio_id</code> is also not specified, the CIO defaults to the currently active one, and filename defaults to the filename that was specified when it was read.		
[ID=cio_id]	DDA Usage:	cio_id	
	Type:	string variable or quoted character string literal (max. of 40 characters)	
	DDA Data Type:	n/a	
	Access:	read only	
	Purpose:	reference id assigned to the CIO file	
	The variable <code>cio_id</code> is a literal character string surrounded by single quotes, or a string variable. The argument <code>ID=cio_id</code> is optional, and if not present, the ID defaults to the <code>cio_id</code> of the currently active memory resident CIO file. If none is currently active, a warning status is returned.		

**[OVERWRITE=
overwrite_option]**

DDA Usage: overwrite_option
Type: Integer (4 bytes)
DDA Data Type: C: DDA_CODE_TYPE
FORTRAN: n/a
Access: read only
Purpose: specifies whether or not the WRITE CIO is allowed to overwrite an existing CIO file

If the file to be written to currently exists, this option specifies whether or not the existing file can be overwritten. If the file does not exist, this option is ignored. There are two legal values for `overwrite_option`:

- DDA_OVERWRITE_YES
- DDA_OVERWRITE_NO

These two integer values for `overwrite_option` are defined in the application include files `dda_fortran.f` and `dda_c.h`.

The OVERWRITE argument is optional, and if not present, the default is DDA_OVERWRITE_NO.

**STATUS=
exec_status**

DDA Usage: exec_status
Type: Unsigned Integer (4 bytes)
DDA Data Type: C: DDA_STATUS_TYPE
FORTRAN: n/a
Access: write only
Purpose: receives the overall status for the WRITE CIO operation

Table 6-22 Returned Status for the WRITE CIO Statement

Status Mnemonic	Meaning	Corrective Action
HGSC_VALID	The WRITE CIO operation was successful	None
DDA_FILENAMETOOLONG	The pathname specified was too long	The CIO file must be specified with a shorter pathname
DDA_CIOIDTOOLONG	The CIO identifier was too long	Choose a shorter CIO identifier
DDA_NOACTCIO	There is no active CIO file	A memory resident CIO file must be made active (using SELECT CIO)
DDA_INVCIOID	Invalid CIO ID	Use a valid CIO ID
GDXS_MEMALLOCERR	Unable to allocate memory	Reduce the amount of memory needed by this application or try executing when the system is less loaded
GDXS_E_WRTCIO	Unable to write the CIO file	Check the file and/or directory permissions and ensure sufficient disk space exists

Table 6-22 Returned Status for the WRITE CIO Statement

Status Mnemonic	Meaning	Corrective Action
GDXS_E_INVOVERWRTOPT	Invalid WRITE CIO file overwrite option value	Change the application to use a proper WRITE CIO overwrite option
GDXS_E_CIOEXISTS	The OVERWRITE option was set to DDA_OVERWRITE_NO and the disk file exists	Change the OVERWRITE option DA_OVERWRITE_YES, select a new CIO file name, or delete the existing CIO file
Other	Other system or platform-specific status	Run <code>dda_xstatus</code> (in <code>/opt/DDA/bin</code>) to get the text message associated with the status value. Refer to Section 7 in the <i>OpenDDA User's Guide</i> for more information on OpenDDA statuses.

Declaring exec_status variables

Refer to “Declaring exec_status variables” on page 155.

Example

The following example shows the use of the EXEC DDA WRITE CIO statement:

```
EXEC DDA WRITE CIO,
      FILE='tank1.cio',
      ID = 'tank1',
      OVERWRITE=DDA_OVERWRITE_YES,
      STATUS=exec_status;
```

After the execution of the above statement, the CIO file associated with the reference ID 'tank1' will be written to the file 'tank1.cio'. If 'tank1.cio' already exists, it will be overwritten.

WRITE CIO usage illustrations

The following table illustrates what happens during certain scenarios, and indicates the side effects that occur.

Note: For the following examples, assume that the CIO file does not exist at the time the write takes place. This will eliminate the effects of the OVERWRITE option. OVERWRITE options will be covered by separate examples.

Table 6-23 WRITE CIO Usage Scenarios

IF this syntax is used...	And this is true...	THEN this occurs...
EXEC DDA WRITE CIO, STATUS=sts;	The CIO file was written to disk successfully	- The active memory resident CIO file was written to disk using the filename specified when it was read in - The CIO file remains active - STATUS returns a good status HGSC_VALID
EXEC DDA WRITE CIO, FILE='new_cio.cio', STATUS=sts;	The CIO file was written to disk successfully	- The active memory resident CIO file was written to the disk file 'new_cio.cio' - The CIO file remains active - STATUS returns a good status HGSC_VALID
EXEC DDA WRITE CIO, ID='boiler1', STATUS=sts;	'boiler1' is a valid ID - The CIO file was written to disk successfully	- The memory resident CIO file associated with the ID 'boiler1' was written to the disk file that it was read from - The CIO file that was active before this EXEC DDA statement was executed, remains active - The memory resident CIO file associated with 'boiler1' will not become active (unless it was the active CIO file) - STATUS returns a good value HGSC_VALID
EXEC DDA WRITE CIO, FILE='new_cio.cio', ID='boiler1', STATUS=sts;	'boiler1' is a valid ID - The CIO file was written to disk successfully	- The memory resident CIO file associated with the ID 'boiler1' was written to the the disk file 'new_cio.cio' - The CIO file that was active before this EXEC DDA statement was executed, remains active - The memory resident CIO file associated with 'boiler1' will not become active (unless it was the active CIO file) - STATUS returns a good value HGSC_VALID
EXEC DDA WRITE CIO, STATUS=sts;	Errors occur writing the CIO file	- STATUS returns an error status GDXS_E_WRTCIO - The active memory resident CIO file remains active
EXEC DDA WRITE CIO, FILE='new_cio.cio', ID='boiler2', STATUS=sts;	'boiler2' is an invalid ID	- STATUS returns an error indicating that the ID was invalid DDA_INVCIOD - The active memory resident CIO file remains active

**OVERWRITE
option usage
illustrations**

The following table illustrates some scenarios involving the use of the OVERWRITE option:

Table 6-24 OVERWRITE option Usage Scenarios

IF this is true...	THEN this occurs...
• OVERWRITE = DDA_OVERWRITE_NO • The disk file exists	• The file is not overwritten • The error GDX_E_CIOEXISTS is returned
• OVERWRITE = DDA_OVERWRITE_YES • The disk file exists	• The old file will be deleted • The new file will be written out • The status associated with deleting and writing out the new file is returned
• OVERWRITE = DDA_OVERWRITE_YES or OVERWRITE = DDA_OVERWRITE_NO • The disk file does not exist	• The new file will be written out • The status associated with writing out the file is returned
• OVERWRITE = an invalid overwrite_option	• The error GDXS_E_INVOVERWRTOPT is returned

6.20 OpenDDA exec_status

exec_status	OpenDDA will return the status of the OpenDDA Execution Statement in the <code>exec_status</code> variable. This status represents the overall result of the OpenDDA Execution Statement and may indicate complete success, partial success, or complete failure.
User-specified status variable	The <code>exec_status</code> variable is a user-specified variable. It is specified as a parameter to an OpenDDA Execution Statement. The <code>exec_status</code> will be updated as a result of performing the action specified by an OpenDDA Execution Statement. Each OpenDDA Execution Statement can have a unique <code>exec_status</code> variable or use the same one; it is up to your discretion exactly how the <code>exec_status</code> variable will be used.
Declaring exec_status variables	OpenDDA provides a construct in the include file <code>dda_c.h</code> that can be used to declare <code>exec_status</code> variables. Honeywell recommends using this construct for portability and migration considerations. FORTRAN applications use <code>INTEGER*4</code> to declare the <code>exec_status</code> variable. C Example: <pre>DDA_STATUS_TYPE exec_status;</pre> FORTRAN Example: <pre>INTEGER*4 exec_status</pre>
Attributes	All statuses returned in the status variables conform to the OpenDDA Error and Message code format. Refer to Section 7 of the <i>OpenDDA User's Guide</i> for more information. A working example on how to use these statuses can be found in 7.2 in the <i>OpenDDA User's Guide</i>.
Good Status	A Good Status (one that has an odd numerical value) is returned when the function has completed successfully.

This table lists programming examples for checking the condition of the call status variable.

Table 6-25 Determining Status

Language	Check for Good, Successful...	Check for Error, Warning...
FORTRAN	IF (IAND(exec_status,1) .EQ. 1) THEN...	IF (IAND(exec_status,1) .NE. 1) THEN...
C	if ((exec_status & 1) == 1) ... or if (exec_status & 1) ...	if ((exec_status & 1) == 0) or if (!(exec_status & 1)) ...

FORTRAN example:

```
EXEC DDA READ SET='list_a', STATUS=read_status;  
IF(IAND(read_status,1) .EQ 1) THEN... ! the data is good
```

C Example:

```
EXEC DDA READ SET='list_a', STATUS=read_status;
if ( (read_status & 1) == 1)
{
    /* status is odd – the data is good */
    ...
}
```

exec_status generation

The `exec_status` is the overall status for a particular OpenDDA Execution Statement.

A single OpenDDA Execution Statement may be expanded into many OpenDDA operations during the Execution Statement expansion. At run time, the most severe error encountered by all operations is returned in the Execution Status Variable.

Some Execution Statements involve returning individual statuses for each item acted on (HIBERNATE EVENT, MODIFY CIO, READ, and WRITE). For these statements, the overall status will be set according to the following table.

Table 6-26 `exec_status` Generation with Individual Statuses

IF the Execution produces...	THEN the <code>exec_status</code> contains...
All individual statuses valid.	HGSC_VALID
Not all individual statuses valid—one or more warning or error statuses.	HGSC_CHECK_STATUS
One or more system errors that prevent the individual items from being acted upon.	The system status

Some Execution Statements only have an overall status and do not involve returning individual statuses (CLEAR CIO, INITIALIZE, LOAD ALIAS, LOAD MODIFY_CODE, LOAD STORE_CODE, READ CIO, SELECT CIO, TERMINATE, and WRITE CIO). For these statements, the overall status will be set according to the following table.

Table 6-27 `exec_status` Generation without Individual Statuses

IF the Execution produces...	THEN the <code>exec_status</code> contains...
Successful operation	HGSC_VALID
A system error occurred during the operation	The system status

Section 7—Definition (DEF) File

7.1 Overview

Definition (DEF) file description

The Definition (DEF) file is a text file containing keyword directives, qualifiers, and subqualifiers that allow you to declare specific information about your application.

The DEF file uses a free-format syntax that allows forward compatibility and makes editing or adding new information easy.

OpenDDA supplies a template file that you can customize.

What the DEF file does

Your application's optional Definition (DEF) file lets you specify certain initialization and runtime characteristics. The DEF file consists of a list of optional directives. These directives allow you to specify:

- Priority for DCS data access
- Initial value for store_code variables
- Whether or not the CIO Signature will be checked when the CIO file is read into memory
- Whether to abort the application or return a status if an inconsistency error is detected
- The items that will be sent to the General Output file or Standard Output as the application executes

Directives are optional

At this time, all directives are optional. The following directives were required in R100 and R110, but are no longer supported:

- NAME
- DESCRIPTION
- TYPE

These directives are allowed for compatibility with the earlier releases, but are not used. If they are present, one of the following will be generated:

- An error if the syntax check fails, or
- A warning that the directives are not required

DEF file is not required

The DEF file is not required. When the EXEC DDA INITIALIZE statement is executed, the system will load:

- The file *application_name.def* in the current directory, or if not present
- The system default definition file */opt/DDA/bin/dda_system.def*, or if not present
- The default values

7.2 DEF File

Creating a DEF file Copy the DEF template file into your working directory, then modify it for your application. The DEF template file is:

`/opt/DDA/bin/dda_template.def`

Syntax statements These are the syntax statement formats:

`DIRECTIVE = VALUE;`

`DIRECTIVE      QUALIFIER = VALUE      ...      QUALIFIER = VALUE;`

`DIRECTIVE      QUALIFIER = VALUE;`

- The `DIRECTIVE` and `QUALIFIER` keywords must be separated by tab(s), space(s), or a carriage return.
- Depending on the context, a `VALUE` can be:
 - a keyword
 - a string
 - a quoted string (a string enclosed in single quotes ' ')

Rules

Use these rules when editing your DEF file.

- All directives, qualifiers, and keywords are case insensitive.
- The entire word of any directive, qualifier, or keyword needs to be entered.
- Any string or number can be put in quotes.
- Strings with embedded spaces *must* be in quotes.
- All statements are terminated by a semicolon (;).
- For string values, two adjacent single quotes (' ') indicate to use the default for the value.
- Do not use a single quote (') within the application description.

Duplicate statements

If duplicate statements appear in a DEF file, OpenDDA uses the last statement and will give you a warning.

Checking the DEF file for errors

At the command level, use the DEF file validate command to check a DEF file for errors. Refer to 5.12 in the *OpenDDA User's Guide*, for a description of the validate DEF file command.

Errors and warnings in DEF files

The possible errors and warnings you might expect in a DEF file are

- Errors
 - a violation of the syntax
- Warnings
 - duplicate statements
 - using statements that are no longer supported (NAME, DESCRIPTION, or TYPE)

7.3 DEF File Format

Format of DEF file The general format of the DEF file is listed below. All statements are optional.

```
INITIAL_VALUES
  {STORE_CODE=[STORE|NOSTORE]};
DATA_PRIORITY=[CONTROL|NOCONTROL|RCASENB];
CIO_SIGNATURE=[IGNORE|ERROR];
INCONSISTENCY=[ABORT|RETURN];
PRINT
  {VALUES=[ON|OFF|YES|NO|TRUE|FALSE]}
  {STATS=[ON|OFF|YES|NO|TRUE|FALSE]}
  {MESSAGES=[ON|OFF|YES|NO|TRUE|FALSE]};
```

Legend:

| indicates an exclusive or

{ } indicates optional qualifier

[] indicates one option within the brackets must be chosen if the qualifier is included

; each statement must end with a semicolon

Choose whichever positive (ON, YES, TRUE) or negative (OFF, NO, FALSE) value complies with your standards.

Statements no longer supported

The following statements are no longer supported:

```
NAME=application_name;
DESCRIPTION=application_description;
TYPE=[FORTRAN | C];
```

If present, they will be allowed for compatibility with earlier releases, but will not be used. However, if present, they will be checked for syntax.

7.4 System Default DEF File

Customizing your system default DEF file

You can easily customize the system default DEF file by editing the following file:

`/opt/DDA/bin/dda_system.def`

At the command level, use your editor to modify the DEF template file.

ATTENTION

Changes made to the system default DEF file are system-wide.

Take care when making changes and inform all OpenDDA users of your modifications and their effects.

7.5 DEF File Example

DEF file example The following is a DEF file example.

```
DATA_PRIORITY=CONTROL;  
INITIAL_VALUES  
    STORE_CODE=STORE;  
CIO_SIGNATURE=ERROR;  
INCONSISTENCY=ABORT  
PRINT  
    VALUES=OFF  
    STATS=ON  
    MESSAGES=ON;
```

In this example

- Data access will be at the CONTROL level
- The STORE_CODE setting of STORE will be available to the application
- The CIO Signature will be checked when a CIO file is read into memory, and an error will cause an error status return
- The application will abort if an inconsistency error is detected
- The application's performance statistics and messages are set to print.

7.6 DATA_PRIORITY Directive

Definition	The DATA_PRIORITY directive allows you to specify the transaction priority for DCS data access. It is also used to determine the priority of the Data Access Server. Refer to the subsection 6.1 of the <i>Application Module^X User Guide</i> for a description of the Data Access Server Configuration and associated priorities.
Syntax	DATA_PRIORITY=[CONTROL NOCONTROL RCASENB];
Keywords	This directive requires one of three possible keywords: • NOCONTROL—specifies the DCS Read/Write/Modify request is to be handled in a normal manner. The request will be queued on a LOW priority Data Access Server. • CONTROL—specifies a higher priority data access should be given to the DCS Read/Write/Modify requests. The request will be queued on a HIGH priority Data Access Server. • RCASENB—specifies a high priority data access should be given to the DCS Read/Write requests. The request will be queued on a HIGH priority Data Access Server. RCASENB also allows the DCS to do continuous control of the process.
No LOW DA Servers Configured	If the Priority specified in the DEF file is NOCONTROL, this indicates that the request will be queued in a LOW Data Access server. If there isn't a LOW Data Access Server configured, the following actions will be taken depending on the type of request. • Build Time Validation—the request will be reissued to a HIGH Data Access Server. • Run Time Read/Write/Modify—an error will be returned indicating that no LOW priority Data Access Servers are configured.
Default	If the optional DATA_PRIORITY directive is not specified, the default is DATA_PRIORITY=NOCONTROL;
Example:	In this example, the application has transaction priority CONTROL. DATA_PRIORITY=CONTROL;

7.7 INITIAL_VALUES Directive

Definition The INITIAL_VALUES directive allows you to specify an initial value to be stored in the memory resident CIO file each time it is read into memory. The CIO file is read into memory whenever an EXEC DDA INITIALIZE or EXEC DDA READ CIO Execution Statement is performed. When you execute an EXEC DDA LOAD STORE_CODE Execution Statement, your Store Code Variables will receive the value contained in the memory resident CIO file.

NOTE: An EXEC DDA WRITE Execution Statement will modify the memory resident CIO file with the values contained in your Store Code Variables.

Syntax INITIAL_VALUES
STORE_CODE=[STORE|NOSTORE];

Qualifiers This directive uses one qualifier:

- STORE_CODE

Runtime behavior This directive sets the initial starting values in the application, but these are not automatically propagated to your application's HLL variables. This provides a means of communicating the default values to the application.

To load the HLL variables, use the EXEC DDA LOAD STORE_CODE statement.

Refer to 6.12 for more information.

STORE_CODE=[STORE|NOSTORE] The STORE_CODE qualifier sets the value used to initialize all application Store Code Variables. It takes effect at the beginning of application execution.

- Possible keyword values: STORE, NOSTORE
- Default: NOSTORE
- Example: STORE_CODE=NOSTORE

In this example, all Store Code Variables defined in the application will be initialized to NOSTORE when the application executes the EXEC DDA LOAD statement.

7.8 CIO_SIGNATURE Directive

Definition	The CIO_SIGNATURE directive allows you to specify whether or not the CIO Signature check will cause an error if a CIO Signature mismatch occurs. Note: The CIO Signature check is made for the following statements: • EXEC DDA READ • EXEC DDA WRITE • EXEC DDA LOAD STORE_CODE • EXEC DDA LOAD MODIFY_CODE • EXEC DDA LOAD ALIAS • EXEC DDA MODIFY CIO • EXEC DDA READ CIO
Syntax	<pre>CIO_SIGNATURE=[IGNORE ERROR] ;</pre>
Keywords	This directive uses one of two possible keywords: • IGNORE When the CIO Signature check is made, a signature mismatch will not cause an error. An informational status is returned if a mismatch occurs. • ERROR When the CIO Signature check is made, a signature mismatch will cause an error. An error status is returned if a mismatch occurs.
Default	If the optional CIO_SIGNATURE directive is not specified, the default is: <pre>CIO_SIGNATURE=ERROR;</pre>
Example	In this example, the application ignores a CIO signature mismatch. <pre>CIO_SIGNATURE=IGNORE;</pre>

7.9 INCONSISTENCY Directive

Definition	The INCONSISTENCY directive allows you to specify whether to abort the application upon finding an inconsistency error, or to continue executing the application. If the application continues to execute, an error status will be returned indicating an inconsistency error(s) was detected.
Syntax	<code>INCONSISTENCY = [ABORT RETURN];</code>
Keywords	This directive requires one of two possible keyword values: • ABORT—If an inconsistency error is detected, information about the inconsistency error will be printed to standard out and the application will abort. • RETURN—If an inconsistency error is detected, information about the inconsistency error(s) will be printed to standard out and the application will continue to execute. The error status <code>DDA_INCONSISTERR</code> will be returned in the execution status variable of the <code>EXEC DDA</code> statement that was executing when the inconsistency error was detected.
Default	If the optional INCONSISTENCY directive is not specified, the default is: <code>INCONSISTENCY = ABORT;</code>
Example	In the following example, the application will continue to execute after an inconsistency error, but will return an error status. <code>INCONSISTENCY = RETURN;</code>

7.10 PRINT Directive

Definition The PRINT directive allows you to determine whether an application will “print” various types of information regarding its execution.

Printing, in the case of this directive, indicates:

- the output is written to the General Output file (`application_name.dda_out`) which can be viewed.
- the environment variable `DDA_REDIRECT_OUTPUT` can be used to send the output to the terminal. If `DDA_REDIRECT_OUTPUT = N`, then the output will be sent to standard output; otherwise, it will be sent to the General Output file. Refer to 3.7 in the *OpenDDA User’s Guide* for a description of the redirection environment variable.

Syntax PRINT
VALUES=[ON|OFF|YES|NO|TRUE|FALSE]
STATS=[ON|OFF|YES|NO|TRUE|FALSE]
MESSAGES=[ON|OFF|YES|NO|TRUE|FALSE];

Keywords This directive requires one or more of these possible keywords:

- MESSAGES
- STATS
- VALUES

Default The default value for all PRINT directive qualifiers is OFF.

Example In this example, all PRINT qualifiers have been set to print.

```
PRINT
MESSAGES=ON
STATS=YES
VALUES=TRUE;
```

**MESSAGES=[ON
| OFF | YES | NO |
TRUE | FALSE]**

The MESSAGES qualifier controls printing of general information messages that are generated during the application's execution.

Application-generated messages written to General Output (such as WRITE or PRINT statements in FORTRAN) are not affected by any PRINT directive qualifier.

- Possible keyword values: ON, OFF, YES, NO, TRUE, FALSE
- Default: OFF
- Example: MESSAGES=YES
In this example, any general information messages that occur during the application's execution will be printed.

ATTENTION

Inconsistency error messages will always print regardless of the MESSAGES qualifier setting.

The following is a sample output of ddademo with MESSAGES=YES. Lines in bold are generated by OpenDDA. Lines that are not bold are generated by the application.

```
Begin DDA Initialization for Application: ddademo
Application's data access priority is set to: NOCONTROL
DDA Application Initialization completer.
Populated Store-code Variables (Set = "ALL")
LOAD STORE_CODEs was successful
De-populated Variable(s) (Set = "STRING_SET")
Write External Data completed (Set = "STRING_SET")
Populated Variable(s) after Write (Set = "STRING_SET")
First WRITE of "STRING_SET" was successful
Read External Data completed (Set = "REAL_SET")
Populated Variable(s) from Read (Set = "REAL_SET")
READ of set "REAL_SET" was successful
REAL1_VALUE = 656.2498
REAL2_VALUE = 91179.01
New values being written
REAL1_VALUE = 667.3498
REAL2_VALUE = 91201.21
De-populated Variable(s) (Set = "REAL_SET")
Write External Data completed (Set = "REAL_SET")
Populated Variable(s) after Write (Set = "REAL_SET")
WRITE of set "REAL_SET" was successful
De-populated Variable(s) (Set = "STRING_SET")
Write External Data completed (Set = "STRING_SET")
Populated Variable(s) after Write (Set = "STRING_SET")
Second WRITE of set "STRING_SET" was successful
End of DDA Termination for Application: ddademo
```

- STATS=[ON | OFF | YES | NO | TRUE | FALSE]** The STATS qualifier controls printing of performance statistics related to OpenDDA Execution Statements.
- Possible keyword values: ON, OFF, YES, NO, TRUE, FALSE
 - Default: OFF
 - Example: STATS=YES—In this example, the application's execution performance statistics will be printed. The following is a sample output of ddademo with STATS=YES. Lines in bold are generated by OpenDDA. Nonbold lines are generated by the application.

Begin DDA Initialization for Application: ddademo

DDA Application Initialization completed.

ELAPSED: 0 00:00:00.09 CPU: 0:00:00.02 BUFIO: n/a DIRIO: n/a FAULTS: n/a

Populated Store-code Variables (Set = "ALL")

ELAPSED: 0 00:00:00.01 CPU: 0:00:00.01 BUFIO: n/a DIRIO: n/a FAULTS: n/a

LOAD STORE_CODEs was successful

De-populated Variable(s) (Set = "STRING_SET")

ELAPSED: 0 00:00:00.00 CPU: 0:00:00.00 BUFIO: n/a DIRIO: n/a FAULTS: n/a

Write External Data completed (Set = "STRING_SET")

ELAPSED: 0 00:00:00.95 CPU: 0:00:00.10 BUFIO: n/a DIRIO: n/a FAULTS: n/a

Populated Variable(s) after Write (Set = "STRING_SET")

ELAPSED: 0 00:00:00.00 CPU: 0:00:00.00 BUFIO: n/a DIRIO: n/a FAULTS: n/a

First WRITE of set "STRING_SET" was successful

Read External Data completed (Set = "REAL_SET")

ELAPSED: 0 00:00:01.01 CPU: 0:00:00.04 BUFIO: n/a DIRIO: n/a FAULTS: n/a

Populated Variable(s) from Read (Set = "REAL_SET")

ELAPSED: 0 00:00:00.00 CPU: 0:00:00.00 BUFIO: n/a DIRIO: n/a FAULTS: n/a

READ of set "REAL_SET" was successful

REAL1_VALUE = 656.2498

REAL2_VALUE = 91179.01

New values being written

REAL1_VALUE = 667.3498

REAL2_VALUE = 91201.21

De-populated Variable(s) (Set = "REAL_SET")

ELAPSED: 0 00:00:00.00 CPU: 0:00:00.00 BUFIO: n/a DIRIO: n/a FAULTS: n/a

Write External Data completed (Set = "REAL_SET")

ELAPSED: 0 00:00:00.98 CPU: 0:00:00.06 BUFIO: n/a DIRIO: n/a FAULTS: n/a

Populated Variable(s) after Write (Set = "REAL_SET")

ELAPSED: 0 00:00:00.00 CPU: 0:00:00.00 BUFIO: n/a DIRIO: n/a FAULTS: n/a

WRITE of set "REAL_SET" was successful

De-populated Variable(s) (Set = "STRING_SET")

ELAPSED: 0 00:00:00.00 CPU: 0:00:00.00 BUFIO: n/a DIRIO: n/a FAULTS: n/a

Write External Data completed (Set = "STRING_SET")

ELAPSED: 0 00:00:00.78 CPU: 0:00:00.06 BUFIO: n/a DIRIO: n/a FAULTS: n/a

Populated Variable(s) after Write (Set = "STRING_SET")

ELAPSED: 0 00:00:00.00 CPU: 0:00:00.00 BUFIO: n/a DIRIO: n/a FAULTS: n/a

Second WRITE of set "STRING_SET" was successful

TOTAL Application statistics

ELAPSED: 0 00:00:03.83 CPU: 0:00:00.29 BUFIO: n/a DIRIO: n/a FAULTS: n/a

End of DDA Termination for Application: ddademo

**VALUES=[ON |
OFF | YES | NO |
TRUE | FALSE]**

The VALUES qualifier controls printing of all of the values that are input to the application and output by the application.

A message for value statuses will be displayed.

- Possible keyword values: ON, OFF, YES, NO, TRUE, FALSE
- Default: OFF
- Example: VALUES=YES
In this example, all the application's values will be printed.

ATTENTION

The use of the MESSAGES, STATS, and VALUES qualifiers cause additional information to be placed in the . dda_out file.

To avoid filling your disk, you should consider the following when setting these directive values for your applications (especially if your application remains memory resident through multiple executions):

- the need for the STATS and VALUES qualifiers
- available disk space

The following shows some examples of the output OpenDDA produces when VALUES=ON for various EXEC DDA statements.

Given the following EDB:

```
C      EXTERNAL DATA BEGIN
C
C      EXTDEF (AXM, DEFAULT)
C
C      REAL1_VALUE = 100.0 /REAL
C      REAL1_VALUE_STAT.VS = INPUT CURRENT(, fc101.pv)
C      REAL1_VALUE_STAT.SC = OUTPUT
C      REAL1_VALUE_STAT.SS = STORE_STATUS
C      REAL1_VALUE_TS      = TIMESTAMP
C      REAL1_VALUE_TEST    = TEST
C      SET_LIST = REAL_SET;
C
C      VALUE1 = 200.0 /REAL
C      VALUE1_STAT.VS      = INPUT  CURRENT(, *.pv)
C      VALUE1_STAT.SC      = OUTPUT
C      VALUE1_STAT.SS      = STORE_STATUS
C      VALUE1_ALIAS        = POINT_NAME
C      VALUE1_EXT_REF_STATUS = EXT_REF_STATUS
C      VALUE1_MODIFY_CODE   = MODIFY_CODE
C      SET_LIST = VALUESET;
C
C      EXTERNAL DATA END
```

Example 1:

The execution of the following EXEC DDA statement

```
EXEC DDA READ SET = 'REAL_SET', STATUS = IOSTATUS;
```

produces the following output when no error is encountered:

```
=====
Seq-# Dir Store-Code Variable-Name <Item-number>
      DataType Value-Status Data-Value
=====
  1 IN      REAL1_VALUE  <1>
      REAL Valid 689.549756
      Timestmp 03-OCT-1995 13:29:32.30
      TESTVAL 100.000000
```

This output indicates that during the EXEC DDA READ, reading of the Value Variable REAL1_VALUE and its ancillary variables was valid. This indicates the Value Status associated with the Value Variable REAL1_VALUE (which is REAL1_VALUE_STAT.VS) was valid, which indicates that it was loaded with HGSC_VALID. The Value Variable REAL1_VALUE was loaded with 689.549756, its associated TIMESTAMP variable (REAL1_VALUE_TS) was loaded with the "03-OCT-1995 13:29:32.30", and its associated TEST variable (REAL1_VALUE_TEST) was loaded with 100.0.

Alternatively, the following output could be produced when an error is encountered:

```
  1 IN      REAL1_VALUE  <1>
      REAL Status[536871554] 100.000000
          LCN-E, DAC no such param on pt
      Timestmp
      TESTVAL 100.000000
```

This shows that the reading of the Value Variable REAL1_VALUE and its ancillary variables was not valid. The Value Status variable associated with the Value Variable REAL1_VALUE (which is REAL1_VALUE_STAT.VS) was loaded with 536871554. As a convenience to the user, the textual message associated with this status is shown ("LCN-E, DAC no such param on pt"). The Value Variable REAL1_VALUE contains the test value (100.0), but it is not guaranteed it will always contain the test value. Its associated TIMESTAMP variable (REAL1_VALUE_TS) was not loaded with a value, and its associated TEST variable (REAL1_VALUE_TEST) was loaded with 100.0.

Example 2:

The execution of the following EXEC DDA statement

```
EXEC DDA LOAD STORE_CODE,  
      SET = 'REAL_SET',  
      STATUS = IOSTATUS;
```

produces the following output:

```
      2 IN      REAL1_VALUE  <1>  
STORCODE Store
```

This output indicates that during the EXEC DDA LOAD STORE_CODE, the Store Code Variable associated with the Value Variable REAL1_VALUE (which is REAL1_VALUE_STAT.SC) was loaded with the Store value.

Example 3:

The execution of the following EXEC DDA statement

```
EXEC DDA WRITE SET = 'REAL_SET', STATUS = IOSTATUS;
```

produces the following output when no error is encountered:

```
      3 OUT Store REAL1_VALUE  <1>  
REAL  700.649780  
  
      4 IN      REAL1_VALUE  <1>  
STORSTAT Valid
```

This output indicates that during the EXEC DDA WRITE, the value in the Value Variable REAL1_VALUE (700.64978) was stored. After the store took place, the Store Status indicates that the store was Valid. "Valid" indicates that the Store Status Variable associated with the Value Variable REAL1_VALUE (which is REAL1_VALUE_STAT.SS) was loaded with HGSC_VALID.

Alternatively, the following output could be produced when an error is encountered:

```
      3 OUT Store REAL1_VALUE  <1>  
REAL  700.649780  
      4 IN      REAL1_VALUE  <1>  
STORSTAT Status[536871690]  
          LCN-E, DAC cannot chg PV
```

After the store took place, the Store Status Variable does not show "Valid." This indicates an error occurred writing this value. In the example, the Store Status Variable associated with the Value Variable REAL1_VALUE (which is REAL1_VALUE_STAT.SS) was loaded with 536871690. As a convenience to the user, the textual message associated with this status is shown ("LCN-E, DAC cannot chg PV").

Example 4:

The execution of the following EXEC DDA statement

```
EXEC DDA MODIFY CIO, SET='VALUESET', STATUS=IOSTATUS;
```

produces the following output when no error is encountered:

```
=====
EDB#  Ctl-Rec#  Alias-Type  Alias-Value
                        Value-Variable <---[ External-Reference ]
=====
  2      2      POINT_NAME  sim1
                        VALUE1 <---[ sim1.pv ]

  2      2      EXT_REF_STATUS  243957777
                        HGSC-S, Action requested was successful
```

This output indicates the Alias Variable associated with the Value Variable VALUE1 (which is VALUE1_ALIAS) contains the value "sim1" and is of POINT_NAME type. The complete External Reference of "sim1.pv" is now associated with the value variable VALUE1. External validation was done and it was successful. The External Reference Status Variable associated with the value variable VALUE1 (which is VALUE1_EXT_REF_STATUS) was loaded with the value 243957777.

The "2 2" indicate which EDB statement is being acted on. The first "2" is the EDB statement number relative to the beginning of the EDB. The second "2" is the control record number (an internal record not accessible by the user). These numbers will be the same unless the EDB statement is using a FULL_NAME array where the control record number will increment for each element of the array.

Alternatively, the following output could be produced when an error is encountered:

```

      2      2      POINT_NAME  sim1
                        VALUE1 <---[ sim1.pv ]

      2      2      EXT_REF_STATUS  243960162
                        HGSC-E, Entity not found

```

This indicates that the external validation was not successful. The External Reference Status Variable associated with the Value Variable VALUE1 (which is VALUE1_EXT_REF_STATUS) was loaded with the value 243960162. As a convenience to the user, the textual message associated with this status is shown ("HGSC-E, Entity not found").

PRINT output example

The following sample PRINT output is an example of the results when all PRINT qualifiers are set to ON (YES/TRUE):

Begin DDA Initialization for Application: ddademo

Application's data access priority is set to: NOCONTROL

DDA Application Initialization completed.

ELAPSED: 0 00:00:00.02 CPU: 0:00:00.02 BUFIO: n/a DIRIO:
n/a FAULTS: n/a

```

=====
Seq-# Dir Store-Code Variable-Name <Item-number>
      DataType Value-Status Data-Value
=====

```

```

      1 IN      REAL_VALUE  <1>
          STORCODE Store
      2 IN      STRING_VALUE <1>
          STORCODE Store

```

Populated Store-code Variables (Set = "ALL")

ELAPSED: 0 00:00:00.00 CPU: 0:00:00.00 BUFIO: n/a DIRIO:
n/a FAULTS: n/a

```

      3 OUT Store REAL_VALUE  <1>
          REAL Status[243973762] 50.000000

```

```

      4 OUT Store STRING_VALUE <1>
          LONG STR Status[243973762] DDA Demo Beginning

```

De-populated Variable(s) (Set = "ALL")

ELAPSED: 0 00:00:00.01 CPU: 0:00:00.01 BUFIO: n/a DIRIO:
n/a FAULTS: n/a

Write External Data completed (Set = "ALL")

Definition (DEF) File
7.10 PRINT Directive

ELAPSED: 0 00:00:03.81 CPU: 0:00:00.07 BUFIO: n/a DIRIO:
n/a FAULTS: n/a

5 IN STRING_VALUE <1>
STORSTAT 243957777

Populated Variable(s) after Write (Set = "ALL")

ELAPSED: 0 00:00:00.00 CPU: 0:00:00.00 BUFIO: n/a DIRIO:
n/a FAULTS: n/a

TOTAL Application statistics

ELAPSED: 0 00:00:03.90 CPU: 0:00:00.10 BUFIO: n/a DIRIO:
n/a FAULTS: n/a

End of DDA Termination for Application: ddademo

Section 8—FORTRAN Applications

8.1 Overview

Introduction	OpenDDA simplifies access to DCS data using FORTRAN applications and the structured environment provided by OpenDDA speeds application development and eases application maintenance.
In this section	This section contains • the listing for the supplied FORTRAN template • any special syntax restrictions • a FORTRAN DEF file example • a FORTRAN EDB example, and • a sample OpenDDA FORTRAN application.
Using FORTRAN programs	With OpenDDA FORTRAN applications, you write programs and subroutines within your application in the usual manner. A blank external data block (EDB) is in the template FORTRAN application supplied by OpenDDA. Any data that is defined in your EDB is accessible by your subroutines using standard FORTRAN data passing techniques.
FORTRAN Include file	The FORTRAN Include file can be found in the <code>/opt/DDA/include</code> directory, file name <code>dda_fortran.f</code> . Any FORTRAN program or subroutine that uses OpenDDA Execution Statements or OpenDDA supplied parameters must include this file.
OpenDDA types supplied	OpenDDA provides predefined constructs that can be used in OpenDDA applications when declaring HLL Alias Variables. For FORTRAN, these constructs are contained in the HLL include file <code>dda_fortran.f</code> . The FORTRAN constructs provided support character variables, but not real or integer variables because of portability considerations.
Sample FORTRAN applications	There are sample FORTRAN applications provided in the <code>/opt/DDA/examples</code> directory.

8.2 OpenDDA FORTRAN Template

Sample FORTRAN template The following is the OpenDDA FORTRAN template that OpenDDA supplies. This file is: /opt/DDA/bin/fortran.f.

```
C
      PROGRAM FORTRAN_TEMPLATE
      IMPLICIT NONE
      INCLUDE 'dda_fortran.f'

C
C
C      EXTERNAL DATA BEGIN
C
C
C      EXTERNAL DATA END

      INTEGER*4 IOSTATUS

C      Initializing Application Environment
      EXEC DDA INITIALIZE STATUS = IOSTATUS;
      IF (IOSTATUS .NE. HGSC_VALID) THEN
        PRINT *, 'ERROR DURING INITIALIZE, STATUS=', IOSTATUS
        GOTO 999
      ENDIF
C      Terminating Application Environment
      EXEC DDA TERMINATE STATUS = IOSTATUS;
      IF (IOSTATUS .NE. HGSC_VALID) THEN
        PRINT *, 'ERROR DURING TERMINATE, STATUS=', IOSTATUS
        GOTO 999
      ENDIF

999    CONTINUE
      END
```

ATTENTION

Your FORTRAN variables must be declared in your source file before the external data is read.

**FORTTRAN EDB
syntax**

FORTTRAN EDB syntax follows the standard OpenDDA EDB syntax with the following three special syntax rules:

- Comment characters:
Use an upper- or lowercase "C", "*", or "!" in the first column of each line
- Array subrange value delimiters:
Use a colon to delimit the range values: `array_variable(n:m)`
- Array range delimiters
Use parentheses to enclose the array range: `array_variable(n:m)`
- Use regular FORTRAN syntax for multidimensional array:
`array_variable(5,5,1:10)`.

Reference

For more information about the EDB (external data block), refer to Section 3— “External Data” and Section 4— “EDB Data Access Variables.”

8.3 DEF File Template

DEF template file The DEF file template can be found in:
 /opt/DDA/bin/dda_template.def

Same template for FORTRAN and C The same DEF template is used for FORTRAN and C applications.

Changing the DEF template file You can easily customize the DEF template file found in the /opt/DDA/bin directory. At the command level, use your editor to modify the DEF template file, for example:
vi /opt/DDA/bin/dda_template.def

WARNING

A modified DEF file template becomes the new template for your OpenDDA system. It may be more desirable to copy and rename the template (for example, *my_application.def*) and then modify the new file.

8.4 FORTRAN Application Example

Application example The following FORTRAN application example shows how all of the “pieces” fit together.

```

Program FEXAMPLE
IMPLICIT NONE
INCLUDE 'dda_fortran.f'

C
C  EXAMPLE FORTRAN PROGRAM TO CHANGE A SETPOINT
C
C  EXTERNAL DATA BEGIN
C
C  EXTDEF ( AXM, DEFAULT )
C
C  QA101_CASREQ = 'NOTREQ' /STRING ! IS SET TO 'REQUEST' WHEN
C                                ! CAS IS REQUESTED
C    QA101_CASREQ_VS = INPUT CURRENT(, QA101.CASREQ)
C    QA101_CASREQ_SC = OUTPUT
C    QA101_CASREQ_SS = STORE_STATUS
C    SET_LIST        = IN_SET, OUT_SET1;
C
C  QA101_MODE = 'CAS' /ENUMERATED
C    QA101_MODE_VS = INPUT CURRENT(, QA101.MODE)
C    SET_LIST      = IN_SET;
C
C  QA101_SP = 75.0 /REAL
C    QA101_SP_SC = OUTPUT CURRENT(, QA101.SP)
C    QA101_SP_SS = STORE_STATUS
C    SET_LIST    = OUT_SET2;
C
C  TI103_PV = 400.0 /REAL
C    TI103_PV_VS = INPUT CURRENT(, TI103.PV)
C    SET_LIST    = IN_SET;
C
C  EXTERNAL DATA END
C
  INTEGER*4 IOSTATUS
  CHARACTER*(DDA_STRING_SIZE) QA101_CASREQ
  INTEGER*4 QA101_CASREQ_VS, QA101_CASREQ_SS
  INTEGER*4 QA101_CASREQ_SC

  CHARACTER*(DDA_ENUMERATED_SIZE) QA101_MODE
  INTEGER*4 QA101_MODE_VS

```

FORTRAN Applications

8.4 FORTRAN Application Example

```
REAL*4 QA101_SP
INTEGER*4 QA101_SP_SC
INTEGER*4 QA101_SP_SS

REAL*4 TI103_PV
INTEGER*4 TI103_PV_VS

C   Initializing Application Environment
EXEC DDA INITIALIZE STATUS = IOSTATUS;
IF (IOSTATUS .NE. HGSC_VALID) THEN
    PRINT *, 'ERROR DURING INITIALIZE, STATUS=', IOSTATUS
    GOTO 999
ENDIF

C   Could fetch the configured default for the store codes
C   (from the DEF file), but we will explicitly set them
C   instead.
QA101_CASREQ_SC = DDASC_STORE
QA101_SP_SC     = DDASC_STORE

C   READ THE INPUT DATA
EXEC DDA READ SET = 'IN_SET', STATUS = IOSTATUS;
IF (IOSTATUS .NE. HGSC_VALID) THEN
    PRINT *, 'ERROR DURING READ, STATUS=', IOSTATUS
    GOTO 990
ENDIF

C   CHECK FOR CASCADE CLOSURE REQUEST (CASCADE IS CLOSED FROM SECONDARY)
IF (QA101_CASREQ .EQ. 'REQUEST') THEN
    QA101_CASREQ = 'NOTREQ'
    EXEC DDA WRITE SET = 'OUT_SET1', STATUS = IOSTATUS;
    IF (IOSTATUS .NE. HGSC_VALID) THEN
        PRINT *, 'ERROR DURING WRITE, STATUS=', IOSTATUS
        PRINT *, '      QA101_CASREQ STORE STATUS=', QA101_CASREQ_SS
    ENDIF

    GOTO 990 ! LET OPERATOR SEE CAS STATUS BEFORE CHANGING SP
END IF
IF (QA101_MODE .NE. 'CAS') GOTO 990

C   ADJUST THE SETPOINT
QA101_SP = TI103_PV / 8.0 + 22.5
EXEC DDA WRITE SET = 'OUT_SET2', STATUS = IOSTATUS;
IF (IOSTATUS .NE. HGSC_VALID) THEN
    PRINT *, 'ERROR DURING WRITE, STATUS=', IOSTATUS
    PRINT *, '      QA101_SP STORE STATUS=', QA101_SP_SS
ENDIF
```

```
990  CONTINUE
C    Terminating Application Environment
      EXEC DDA TERMINATE STATUS = IOSTATUS;
      IF (IOSTATUS .NE. HGSC_VALID) THEN
        PRINT *, 'ERROR DURING TERMINATE, STATUS=', IOSTATUS
        GOTO 999
      ENDIF

999  CONTINUE

      END
```

8.5 FORTRAN External Data Block Example

EDB example

The following is an example of a FORTRAN External Data Block declaration utilizing all of the available data types including the appropriate FORTRAN variable declarations.

```

C  EXTERNAL DATA BEGIN
C
C  DDA_REAL = 5.0 /REAL
C    DDA_REAL_STAT = INPUT CURRENT (, VALVE.VARRAY(1000));
C
C  DDA_PT_ID = 0 /POINT_ID
C    DDA_PT_ID_STAT = INPUT CURRENT (, BOIL100.POINTVAL(1));
C
C  DDA_EXT_PT_ID = 'TEST' /EXTERNAL_ID
C    DDA_EXT_PT_ID_STAT = INPUT CURRENT (, BOIL100.POINTVAL(1));
C
C  DDA_STRING = 'STRING' /STRING
C    DDA_STRING_STAT = INPUT CURRENT (, A001.PTDESC);
C
C  DDA_DELTA = '0 00:00:00' /DELTA
C    DDA_DELTA_STAT = INPUT CURRENT (, BOIL100.TIMEVAL(1));
C
C  DDA_ABSOLUTE = '10-SEP-1968 00:00:00' /ABSOLUTE
C    DDA_ABSOLUTE_STAT = INPUT CURRENT (, BOIL100.TIMEVAL(2));
C
C  DDA_ENUM = 'RED' /ENUMERATED
C    DDA_ENUM_STAT = INPUT CURRENT (, BOIL100.GDXENM(1));
C
C  DDA_ORD = 0 /ORDINAL
C    DDA_ORD_STAT = INPUT CURRENT (, BOIL100.GDXLOG(1));
C
C  DDA_INT = 100 /INTEGER
C    DDA_INT_STAT = INPUT CURRENT (, A001.OVERVAL);
C
C  EXTERNAL DATA END
C
      REAL*4          DDA_REAL
      INTEGER*4        DDA_REAL_STAT
      RECORD /DDA_POINT_ID_TYPE/  DDA_PT_ID
      INTEGER*4        DDA_PT_ID_STAT
      CHARACTER*(DDA_EXTERNAL_ID_SIZE) DDA_EXT_PT_ID
      INTEGER*4        DDA_EXT_PT_ID_STAT
      CHARACTER*(DDA_STRING_SIZE)    DDA_STRING
      INTEGER*4        DDA_STRING_STAT
      RECORD /DDA_TIME_TYPE/        DDA_DELTA
      INTEGER*4        DDA_DELTA_STAT
      RECORD /DDA_TIME_TYPE/        DDA_ABSOLUTE
      INTEGER*4        DDA_ABSOLUTE_STAT

```

```

CHARACTER*(DDA_ENUMERATED_SIZE) DDA_ENUM
INTEGER*4          DDA_ENUM_STAT
INTEGER*4          DDA_ORD
INTEGER*4          DDA_ORD_STAT
INTEGER*4          DDA_INT
INTEGER*4          DDA_INT_STAT
    
```

8.6 Examples of Assigning Values to FORTRAN Variables

Examples

```
real_val = 1.2
integer_val = 10
ordinal_val = 1
enumerated_val = 'RED'
string_val = 'this is a string value'
external_id_val = 'A100'
```

```
absolute_time_val.year = 1995
absolute_time_val.month = 3
absolute_time_val.day = 19
absolute_time_val.hour = 12
absolute_time_val.minute = 30
absolute_time_val.second = 45
absolute_time_val.sec_100th = 0
```

```
delta_time_val.year = 0
delta_time_val.month = 0
delta_time_val.day = 15
delta_time_val.hour = 10
delta_time_val.minute = 0
delta_time_val.second = 0
delta_time_val.sec_100th = 0
```

```
point_id_val.part1 = 276
point_id_val.part2 = 3840
point_id_val.part3 = 2
point_id_val.part4 = 256
```

Section 9—C Applications

9.1 Overview

Introduction	OpenDDA simplifies access to DCS data for C applications, and the structured environment provided by OpenDDA speeds application development and eases application maintenance.
In this section	This section contains • the listing for the supplied C template • any special syntax restrictions • a C EDB example, and • a sample OpenDDA C application.
Using C programs	With OpenDDA C applications, you write programs and subroutines within your application in the usual manner. A blank external data block (EDB) is in the template C application supplied by OpenDDA. Any data that is defined in your EDB is accessible by your subroutines using standard C data passing techniques.
Using a C++ compiler	You can use the ANSI standard C++ compiler to compile your OpenDDA applications. You cannot use C++ functionality enhancements in the OpenDDA-specific sections—the EDB, declaration of HLL variables accessed by OpenDDA, or OpenDDA Execution Statements. Other portions of your application can use C++ extensions.
C Include file	The C Include file, <code>dda_c.h</code> , can be found in the <code>/opt/DDA/include</code> directory. This include file is for any C application that uses OpenDDA Execution Statements or OpenDDA supplied typedefs.
OpenDDA types supplied	OpenDDA provides predefined constructs that can be used in OpenDDA applications when declaring HLL Alias Variables. These constructs are contained in the HLL include file <code>dda_c.h</code>. These constructs provide the user with the following benefits: • Ease of use—the user is not burdened with having to remember specifics when defining high-level language Alias Variables. • Portability/migration—by coding with these definitions, users will only need to recompile code if the underlying representation of Alias Variables should change.
Sample C applications	There are sample C applications provided in the <code>/opt/DDA/examples</code> directory.

9.2 OpenDDA C Template

ATTENTION

You must declare your C variables in your source before reading external data.

Sample C template The following is the OpenDDA C template that OpenDDA supplies. This file is: /opt/DDA/bin/c.c

```
#include <stdio.h>
#include <string.h>
#include "dda_c.h"

main()
{
/*
    external data begin

    external data end
*/
DDA_STATUS_TYPE iostatus;

/* Initializing Application Environment */
exec dda initialize status = iostatus;

if (iostatus != HGSC_VALID)
{
    printf ("ERROR DURING INITIALIZE, STATUS=%1d\n",iostatus);
    goto exit_appl;
}

/* Terminating Application Environment */
exec dda terminate status = iostatus;

if (iostatus != HGSC_VALID)
{
    printf ("ERROR DURING TERMINATE, STATUS=%1d\n",iostatus);
    goto exit_app1;
}
exit_app1;;
}
```

C EDB syntax

EDB syntax follows the standard OpenDDA EDB syntax with the following three special syntax rules:

- **Comment characters:**
Use a `/*` to start a comment section and use a `*/` to end it. If you are using an ANSI C++ compiler, you can use either `/* .....*/` (same as for C), or you can use `//` at the beginning of each comment line.
- **Array subrange value delimiters:**
Use a colon to delimit the range values: `array_variable[n:m]`
- **Array range delimiters**
Use square brackets to enclose the array range:
`array_variable[n:m]`
- **Use standard C syntax for multi-dimensional array:**
`array_variable[1][2][0:5]`

Reference

For more information about the EDB (external data block), refer to Section 3— “External Data” and Section 4— “EDB Data Access Variables.”

9.3 DEF File Template

DEF template file The DEF file template can be found in
 /opt/DDA/bin/dda_template.def

Same template for FORTRAN and C The same DEF template is used for FORTRAN and C applications.

Changing the DEF template file You can easily customize the DEF template file found in the
 /opt/DDA/bin directory. At the command level, use your editor to
 modify the DEF template file, for example:
 vi /opt/DDA/bin/dda_template.def

WARNING

A modified DEF file template becomes the new template for your OpenDDA system. It may be more desirable to copy and rename the template (for example, *my_application.def*) and then modify the new file.

9.4 C Application Example

Application example

```
#include <stdio.h>
#include <string.h>
#include "dda_c.h"

main() /* CEXAMPLE */
{
/* example c program to change a setpoint */

/*
external data begin
    extdef ( axm, default)

        qa101_casreq = 'NOTREQ' /string ! is set to 'REQUEST'
                                when cas is requested!
        qa101_casreq_vs = input current (,qa101.casreq)
        qa101_casreq_sc = output
        qa101_casreq_ss = store_status
        set_list        = in_set, out_set1;

        qa101_mode = 'CAS' /enumerated
        qa101_mode_vs = input current (,qa101.mode)
        set_list      = in_set;

        qa101_sp = 75.0 /real
        qa101_sp_sc = output current (,qa101.sp)
        qa101_sp_ss = store_status
        set_list    = out_set2;

        ti103_pv = 400.0 /real
        ti103_pv_vs = input current (,ti103.pv)
        set_list    = in_set;

external data end
*/
DDA_STATUS_TYPE iostatus;

DDA_STRING_TYPE qa101_casreq;
DDA_STATUS_TYPE qa101_casreq_vs,qa101_casreq_ss;
DDA_CODE_TYPE qa101_casreq_sc;

DDA_ENUMERATED_TYPE qa101_mode;
```

C Applications

9.4 C Application Example

```
DDA_STATUS_TYPE qa101_mode_vs;

DDA_REAL_TYPE qa101_sp;
DDA_CODE_TYPE qa101_sp_sc;
DDA_STATUS_TYPE qa101_sp_ss;

DDA_REAL_TYPE ti103_pv;
DDA_STATUS_TYPE ti103_pv_vs;

/* Initializing Application Environment */
exec dda initialize status = iostatus;
if (iostatus != HGSC_VALID)
{
    printf ("ERROR DURING INITIALIZE, STATUS=%1d\n",iostatus);
    goto exit_appl;
}

/*
We could fetch the configured default for the store code
(from the DEF file), but we will explicitly set them instead.
*/
qa101_casreq_sc = DDASC_STORE;
qa101_sp_sc      = DDASC_STORE;

/* read the input data */
exec dda read set = 'in_set', status = iostatus;
if (iostatus != HGSC_VALID)
{
    printf ("ERROR DURING READ, STATUS=%1d\n",iostatus);
    goto error_exit;
}

/* check for cascade closure request (cascade is closed from secondary) */
if (strcmp (qa101_casreq,"REQUEST") == 0)
{
    strcpy (qa101_casreq,"NOTREQ");
    exec dda write set = 'out_set1', status = iostatus;
    if (iostatus != HGSC_VALID)
    {
        printf ("ERROR DURING WRITE, STATUS=%1d\n",iostatus);
        printf ("      qa101_casreq store status=%1d\n",qa101_casreq_ss);
    }
    goto error_exit; /* let operator see cas status before changing sp */
}

/* adjust the setpoint */
qa101_sp = ti103_pv / 8.0 + 22.5;
```

```
exec dda write set = 'out_set2', status = iostatus;
if (iostatus != HGSC_VALID)
{
    printf ("ERROR DURING WRITE, STATUS=%1d\n",iostatus);
    printf ("      qa101_sp store status=%1d\n",qa101_sp_ss);
}

error_exit:
/* Terminating Application Environment */
exec dda terminate status = iostatus;
if (iostatus != HGSC_VALID)
{
    printf ("ERROR DURING TERMINATE, STATUS=%1d\n",iostatus);
    goto exit_appl;
}

exit_appl;;
}
```

9.5 C External Data Block Example

EDB example

The following is an example of a C External Data Block declaration utilizing all of the available data types including the appropriate C variable declarations.

```

/*
external data begin

dda_real = 5.0 /real
  dda_real_stat = input current (,valve.varray (1000));

dda_pt_id = 0 /point_id
  dda_pt_id_stat = input current (,boil100.pointval(1));

dda_ext_pt_id = 'test' /external_id
  ddaA_ext_pt_id_stat = input current (,boil100.pointval(1));

dda_string = 'string' /string
  dda_string_stat = input current (,a001.ptdesc);

dda_delta = '0 00:00:00' /delta
  dda_delta_stat = input current (,boil100.timeval(1));

dda_absolute = '10-SEP-1968 00:00:00' /absolute
  dda_absolute_stat = input current (,boil100.timeval(2));

dda_enum = 'red' /enumerated
  dda_enum_stat = input current (,boil100.gdxnm(1));

dda_ord = 0 /ordinal
  dda_ord_stat = input current (,boil100.gdxlog(1));

dda_int = 100 /integer
  dda_int_stat = input current (,a001.oval);

external data end
*/
DDA_REAL_TYPE    dda_real;
DDA_STATUS_TYPE   dda_real_stat;
DDA_POINT_ID_TYPE dda_pt_id;
DDA_STATUS_TYPE   dda_pt_id_stat;
DDA_EXTERNAL_ID_TYPE dda_ext_pt_id;
DDA_STATUS_TYPE   dda_ext_pt_id_stat;
DDA_STRING_TYPE   dda_string;
DDA_STATUS_TYPE   dda_string_stat;
DDA_TIME_TYPE     dda_delta;
DDA_STATUS_TYPE   dda_delta_stat;
DDA_TIME_TYPE     dda_absolute;
DDA_STATUS_TYPE   dda_absolute_stat;

```

```
DDA_ENUMERATED_TYPE    dda_enum;  
DDA_STATUS_TYPE        dda_enum_stat;  
DDA_ORDINAL_TYPE       dda_ord;  
DDA_STATUS_TYPE        dda_ord_stat;  
DDA_INTEGER_TYPE       dda_int;  
DDA_STATUS_TYPE        dda_int_stat;
```

9.6 Examples of Assigning Values to C Variables

Examples

```
real_value = 1.2;
integer_val = 10;
ordinal_val = 1;
strcpy(enumerated_val, "RED");
strcpy(string1, "this is a string value");
strcpy(external_id, "A100");
```

```
absolute_time_val.year = 1995;
absolute_time_val.month = 3;
absolute_time_val.day = 19;
absolute_time_val.hour = 12;
absolute_time_val.minute = 30;
absolute_time_val.second = 45
absolute_time_val.sec_100th = 0;
```

```
delta_time_val.year = 0;
delta_time_val.month = 0;
delta_time_val.day = 15;
delta_time_val.hour = 10;
delta_time_val.minute = 0;
delta_time_val.aecond = 0;
delta_time_val.sec_100th = 0;
```

```
point_id_val.part1 = 276;
point_id_val.part1 = 3840;
point_id_val.part1 = 2;
point_id_val.part1 = 256;
```

Section 10—Troubleshooting

10.1 Overview

Introduction	This section contains: • any known and/or suspected problems that are being addressed and will be corrected in a future release, and • a listing of troubleshooting solutions.
The General Output file	DDA writes any messages (informational and error) to the General Output file (application name with an extension “.dda_out”). These messages can be directed to the standard output by setting the DDA_REDIRECT_OUTPUT environment variable to “N” or “n.” Refer to 3.7 of the <i>OpenDDA User’s Guide</i> for a description of the redirection environment variable.
Access the current OpenDDA version	To access the current version of OpenDDA on your system, enter the following command: <pre>what /opt/DDA/bin/dda</pre>
When reporting a problem	When reporting a problem to the Honeywell Technical Assistance Center (TAC), please include supporting documentation to help identify the source of the problem. Supporting documentation includes: • a copy of the External Data Block • the code fragment experiencing the problem • a copy of the application’s Definition File (DEF) • the contents of the General Output file (.dda_out) • and the version of OpenDDA you are using (see above).
Application execution produces a core dump	If your application’s execution produces a core dump, use the “xdb” command to view the core dump. Save a copy of the core dump to give to your system administrator. Example To view the core dump from the last <code>dda</code> command attempted, enter the following command: <pre>hp-ux> xdb /opt/DDA/bin/dda</pre> Your screen will be filled with the last statement executed before the core dump. Enter ‘t 3’ to view the name of the last 3 subroutines to be executed and include this information when contacting TAC.

10.2 Known and/or Suspected Problems

Array anomaly

OpenDDA cannot access arrays of points with an ENUMERATION index.

**Application
names conflict
with UNIX file
names**

Check your user pathnames. If they contain a UNIX executable file with the same name as your application file, the UNIX executable file might be what is actually executing.

10.3 Problem Symptoms and Possible Actions

PRINT VALUES shows status %HGSC-E- NULL_VALUE

Symptom:

With the **.def** file directive PRINT VALUES=ON, OUTPUT only variables show the following status:

%HGSC-E-NULL_VALUE, No value was returned or put

Action:

This is a result of an OUTPUT only variable not having a VALUE_STATUS variable, so the status shown in the PRINT VALUES display is the default null status. The value has been written to the external destination. The actual status of the write can be seen in the STORE_STATUS variable.

Consistency error

Symptom:

When trying to execute your application, you get the following error:

```
%DDA$_data type does not match database
*ERROR* Unhandled consistency error encountered.
Application terminated.
```

Action:

The application's HLL definitions for the EDB value_variables do not match the definitions in the **.cio** file (created from the EDB during the Parse EDB function). Perform the following:

- Make sure all value_variables are defined properly for the type of external data accessed.
- Remake the **.cio** file using the Parse EDB function (refer to 5.9 in the *OpenDDA User's Guide*).

Privilege problem installing or executing

Symptom:

Privilege problems when installing or executing an application.

Action:

You must be a member of the A^xM Group to install an application and should change your group to A^xM prior to developing your OpenDDA application, so that the owner of the OpenDDA application is A^xM.

Action:

If you have developed your OpenDDA application as "Superuser" or as "Root," you will still be required to be a member of the A^xM group before installing your application.

Action:

Verify your default "umask" has the privileges opened to GROUP. OpenDDA uses this mask by default when creating the soft links during installation.

Errors returned by CL initiation

Special errors can occur for a CL-initiated OpenDDA application that are not easily detected because the nature of the error does not allow error reporting during runtime.

Symptom:

A CL-initiated OpenDDA application does not execute when initiated from the Universal Station (US) operator's keyboard.

Action:

Check the case of the application's name. The US operator keyboard has only uppercase output. UNIX application names are case-sensitive so your CL programs must be written to acknowledge this issue.

Symptom:

If a CL-initiated OpenDDA application does not execute properly and there is no information provided within the application's output file, then one of the following errors could have occurred:

APP\$_E_NOAPPINIT

APP\$_E_MEMALLFAI

Action:

To determine if this is the problem, the Detailed Return Status parameter (Det_Status) associated with the A^xM CL subroutine call (AMCL06\$Execute_Task_With_Wait) should be examined while referring to the following table:

DDA Error	Value of Det_Status
APP\$_E_NOAPPINIT	34
APP\$_E_MEMALLFAI	26

ATTENTION

Executing the application "offline" on the UNIX-side will produce meaningful error messages for these errors.

Refer to your AMCL06 documentation.

No output file and bad status in CL block

Symptom:

An installed application does not produce an output file and has a bad status in the CL block.

Action:

Verify the application has an EXEC DDA INITIALIZE statement.

Application won't write values

Symptom:

Your application will not write values.

Action:

Verify the correct setting of the security switch. The security switch has three settings:

READONLY All applications have read-only access

RW_LCN_I Only CL-initiated applications have write access

READWRIT All applications have read/write access

Action:

Be sure the STORE_CODE value in your application's **.def** file is set to STORE or you have changed the individual values of the STORE_CODEs correctly within your application.

Can't access DDA executable

Symptom:

You have insufficient privilege to access a DDA executable.

Action:

The A^xM software was installed prior to OpenDDA and sets the owner of the /opt directory to "bin bin." Have your system administrator either open up the protection on the directory to WORLD access or change the GROUP and OWNER to "axm."

EDB parsing error

Symptom:

An error occurs while parsing the External Data Block. The value displayed appears to be the entire line of the EDB instead of just the point.parameter.

Action:

Verify that the data source words "CURRENT" or "LCN" on the INPUT and/or OUTPUT EDB statements are spelled correctly.

Erroneous data

Symptom:

Your HLL (High-Level Language) variable is getting erroneous data, but the return status shows valid.

Action:

Make sure your variable is declared as specified in subsection 3.5. OpenDDA does not verify that your HLL variable data type matches the corresponding EDB variable's data type.

Source file not found

Symptom:

You are working in a windowing environment or work with multiple terminal sessions and you get the following message:

DDA-E, Invalid argument or file not found: 'source_file.f'
when you use the **-cb**, **-pe**, or **-ee** qualifier.

Action:

If you just edited your source file, saved it, and immediately tried to use the above qualifiers, you may get the error because the operating system has not finished updating the file. All you need to do is try the command again.

10.4 Error Message Troubleshooting

"No .cio file"

Symptom:

When you execute your application and you get an error message that the **.cio** file cannot be loaded such as:

ERROR Error from SETUP_GDX

ERROR GDX-E, error opening control io file

Action:

Use the Parse EDB function of the OpenDDA command line interface to create the **.cio** (Control Input/Output) file. Refer to 5.9 in the *OpenDDA User's Guide*.

HGSC_E_ BADFLDCMB

Symptom:

The "Invalid combination of fields ..." error is a result of an EDB (External Data Block) item using an invalid set of fields.

For CURRENT data items you cannot OUTPUT EXTERNAL_ID values.

Action:

Modify the EDB to only use a proper combination of fields.

Troubleshooting

10.4 Error Message Troubleshooting

Glossary

Alias Variable	A variable that your application loads with an ASCII representation of an External Reference
Anchored Access	Anchored Access is the ability to specify the starting index of an External Reference array and read or write successive values into an HLL variable array.
Ancillary Variable	A general term used to refer to all EDB Data Access variables other than the Value Variable.
Application Module^x (A^xM)	A module on the Local Control Network that processes data points to provide supervisory control. Standard algorithms and custom programs written in Control Language are used to process data points.
Cache	Whenever OpenDDA accesses a DCS to resolve an External Reference, the information returned (Internal Id, data type, etc.) is entered into a Cache located in the HP-UX. Subsequent resolutions for this External Reference are satisfied by the Cache instead of accessing the DCS, which greatly improves performance.
Cache Management Tool (CMT)	The Cache Management Tool allows examination and/or deletion of Cache entries. It also supports checkpointing the cache.
CL	CL (Control Language) is a Honeywell proprietary software language used to build programs which will execute in an Application Module (AM).
Custom Display, (Schematic)	See Schematic.
Data Access Set	A Data Access Set is a collection of EDB statements whose Value Variables and Ancillary Variables will be acted on in one operation such as an EXEC DDA READ or EXEC DDA WRITE statement.
Data point	A TDC 3000^x data point is a named collection of related parameters and processing involved with the acquisition or calculation of a main item of information and may optionally include the data for an associated control loop. A data point has three parts: a tagname (also called point name or point ID), parameters, and functions.
DCS	Distributed Control System (e.g., TDC 3000 ^x).

Glossary

Depopulate HLL	HLL variables associated with the External Data Statement will be moved to OpenDDA in preparation for writing to an external location.
Dynamic Selection Set	A Dynamic Selection Set is a collection of EDB statements whose External References will be modified in one operation by an EXEC DDA MODIFY CIO statement.
Dynamic Selection Set Functionality (Dynamic Data Referencing)	Dynamic Selection Set Functionality (also known as Dynamic Data Referencing) allows an OpenDDA Application to modify the External Data References within the memory resident CIO to refer to different External Data values.
EDB (External Data Block)	An External Data Block establishes a relationship between your HLL variables and the external data values.
EXEC DDA Statement	EXEC DDA Statements are syntax that you add to your OpenDDA application source code to perform OpenDDA functions at run time.
External Data	A general term that refers to data values read or written from a DCS.
External Data Reference	The identification of a data value on a DCS. It consists of: • a name, which identifies the External Reference - for example, point1.paramter6(2) • a status, which indicates whether or not the External Reference represents a valid external data item
External Reference	Shorthand notation for External Data Reference.
External Reference Component	External Reference Names are composed of fields called "components." The name has the form POINT.PARAMETER(INDEX). The components are: • the POINT, the part before the period • the PARAMETER, the part after the period and before the optional index • the INDEX, the optional part after the PARAMETER and inclosed in parentheses
External Reference Name	The externally known ASCII identification of a data value on a DCS. For example, point1.parameter6(2).

Glossary

GDx (Generic Data eXchange)	GDx is a Honeywell set of software providing the exchange of data between the Host platform (HP-UX) and the DCS (TDC 3000 ^x).
General Output	The default output destination as defined by the Host Platform.
General Output file	A file OpenDDA creates to contain the general output of an executed application. This includes any normal printing statements of an application, along with any other application output.
HLL - High-Level Language	Computer languages (such as Fortran or C/C++) used to write OpenDDA applications.
Host Platform	The Host Platform is a hardware and software computer system that serves as an application development environment for OpenDDA. Host Platforms supported by OpenDDA include • Honeywell's Application Module^x (A^xM), which includes Hewlett-Packard's PA-RISC coprocessor running HP-UX operating system version 9.05 or later • A Hewlett-Packard workstation based on Hewlett-Packard's PA-RISC coprocessor running HP-UX operating system version 9.05 or later.
Inconsistency Error	An inconsistency error occurs when OpenDDA detects a mismatch in data size or array size between the application variable and the value being moved into or out of the application. An example is when the definition of an external data item in the CIO file does not match the corresponding HLL variable definition in data size and array size.
Input/Output	You can specify/control when I/O occurs to the data platform. HLL variables are populated/depoppedulated by OpenDDA-generated code within the body of the application.
Install Application	OpenDDA command line function which prepares an OpenDDA Application for on-line execution.
Internal Id	DCS specific internal representation for an External Reference Name. Typically, accessing data using the Internal Id is much faster than using the equivalent External Reference Name.
Local Control Network (LCN)	The local area network that interconnects the nodes in a TDC 3000 ^x system.

Glossary

Network Gateway	The node that logically connects two LCNs together. The Network Gateway name (also referred to as the LCN name) is prefixed to the point name when used in an External Reference Name. An example is: SS\POINT1.PARAMETER5 where "SS" is the LCN (or Network Gateway) name. The LCN name can also be used for local LCN access when there is no Network Gateway as long as the local LCN is configured with a name.
Off-line Application	OpenDDA Application that is not logically connected to the DCS for execution. These applications cannot be started by the DCS and cannot receive events from the DCS.
On-line Application	OpenDDA Application that has been logically connected by use of the Install Application command line function to a DCS for execution. These applications are typically started by the DCS and may receive events from the DCS.
OpenDDA	Open Data Definition and Access. A platform independent data abstraction layer that • provides a standard application programming interface • assistance for developing and implementing applications with the following qualities: – perform efficiently – read and write varying amounts of heterogeneous data – access one or more external data sources.
OpenDDA Application	A Program written in FORTRAN or C/C++ that uses OpenDDA extensions to the language (EDB and EXEC DDA statements) to access data on a DCS using DCS independent syntax.
OpenDDA Application Directory	The directory where your OpenDDA application physically resides. This directory is usually (but not required to be) where the application was created, compiled, and linked. The files which support your application (i.e., CIO file, DEF file) are also contained in this directory. For on-line applications, this directory is the directory from which the Install Application was performed. For off-line applications, this directory is the current directory.
OpenDDA Status Mnemonics	Mnemonic names for constants used in an OpenDDA application when dealing with OpenDDA status values. The mnemonics are used exclusively throughout the OpenDDA user documentation whenever a reference is made to an OpenDDA status value.

Glossary

OpenDDA Status Values	Values returned to an application by an OpenDDA Execution Statement. These values are used to indicate various degrees of success or failure.
Parameters(TDC) (also point.parameters)	A TDC 3000^x database parameter consists of a single value. The values can be • input data_flow, temperature, pressure, ... • tuning parameters_gain, time constants, bias, recipe data, ... • output data_valve outputs, alarm states, ... The TDC 3000^x bookset includes parameter reference dictionaries which list and define every parameter associated with specified system devices.
Persistence of External References	The procedure of modifying the External Reference Components, saving the memory resident CIO to disk, reloading the CIO later, and using the External References, is known as "persistence of External References." The dynamically selectable External References persist over time by writing them to the CIO on disk.
Platform	The term platform is used to reference the hardware and software computer system serving as an external data source. Platforms supported by OpenDDA include • Honeywell's Application Module^x (A^xM)
Point name (also called data point name, tagname)	A point name is a unique name identifying a data point. Each name is from one to sixteen characters long and the first character must always be an alphanumeric character. Examples: TIC100, FC_2000, PUMP200A
Populate HLL	HLL variables associated with the External Data Statement will be populated by OpenDDA with values read from the external locations.
Remap CIO	Remap CIO is the functionality which allows an OpenDDA application to dynamically associate the HLL variables to a different group of external data by reading multiple CIO files into memory.
Remote Node	A node on a network other than the A ^x M node that is used to obtain access to external data.
Schematic (Custom Display)	User-defined graphic and textual representations of the process or any other available information for the console operator on the TDC 3000 ^x system.

Glossary

TDC

Total Distributed Control - the Honeywell design philosophy of full distribution of architecture, full integration of functions, and distributed risk, allowing for continuous expansion and capability.

Index

▪

.dda_out file 199

A

ABSOLUTE

EDB data type 16

test value data type 38

Access name

changing the default 19

reserved names 45

Accessing value status variable 56

Alias Variable

declaring 69

external_reference_keyword 67

OpenDDA types supplied 69, 179, 189

size 69

type 69

Alias Variable arrays 75

using 76

Alias Variable statement 66

syntax 66

Alias Variables

declaring 69

loading 80

ALL data set 7

ALL Dynamic Selection Set 26

Ancillary EDB Variables 54

Ancillary variable syntax 54

Anomalies

array 200

Arrays

anomaly 200

declaring variables 32

how to read values 37

how to use 33

identifier syntax 14

subrange format 33

value variable 33

arrays

Alias Variable 75

B

Bracket symbols 3

C

C

customize the DEF file 192

default DEF file 192

EDB syntax 191

include file 189

programs 189

sample applications 189

template 190

Case-sensitivity 3

CLEAR CIO statement 96, 100

Comments

in EDB 11

Core dump 199

CURRENT location spec

format 45

Custom data segments

defined as LOGICAL 39

Customizing the DEF file 164

D

Data Access Sets 27

maximum sets 27

multiple 27

Data access time variable 60

declaring 61

Data points

definition 53

Data sets 6

ALL set 7

Data type 40

ABSOLUTE 16

defaults 17

DELTA 16

ENUMERATED 16

EXTERNAL_ID 16

INTEGER 16

ORDINAL 16

POINT_ID 16

REAL 16

rules 17

STRING 16

test value 38

data types 17

DATA_PRIORITY directive 166

DDA application

keywords 13

reserved words 13

DDA_POINT_ID_TYPE 18

DDA_TIME_TYPE 61

DDASC_store 53, 57, 143

Index

- declaring Alias Variables 69
- Declaring array variables 32
- Declaring data access time variables 61
- declaring Full name Arrays 77
- declaring Modify Code variables 90
- Declaring value status variables 56
- DEF file
 - create 161
 - customize 164
 - default
 - C 192
 - FORTTRAN 182
 - definition 159
 - duplicate statements 161
 - errors 162
 - examples
 - FORTTRAN 165
 - format 163
 - optional directive
 - CIO_SIGNATURE 168
 - DATA_PRIORITY 166
 - INITIAL_VALUES 167
 - PRINT 170
 - rules 161
 - syntax statements 161
- Default store codes 58
- Defaults
 - EDB data types 17
- Definition file (See DEF file)
- DELTA
 - EDB data type 16
 - test value data type 38
- Depopulate HLL 5
- Directive
 - EXTDEF 19
- Duplicate statements in DEF file 161
- dynamic components
 - initialization 73
- Dynamic Selection
 - External Reference Status Variables 86
 - Indexes 82
 - Modify Code Variables 89
- Dynamic Selection Indicators 71
- Dynamic Selection Sets 67
 - ALL Selection Set 26
 - concept 28
 - maximum 67
 - tools 29

E

- EDB
 - ancillary variables 54
 - array identifiers
 - syntax 14
 - data statement format 10
 - data types
 - ABSOLUTE 16
 - defaults 17
 - DELTA 16
 - ENUMERATED 16
 - EXTERNAL_ID 16
 - INTEGER 16
 - ORDINAL 16
 - POINT_ID 16
 - REAL 16
 - rules 17
 - STRING 16
 - definition 9
 - EXTDEF directive 19
 - keywords 13
 - location 9
 - location spec 43
 - max. per statement 38
 - parsing error 203
 - reserved words 13
 - rules 12
 - SET_LIST directive 63
 - STORE_CODE variable 55
 - STORE_STATUS variable 55
 - syntax 14
 - test data 39
 - TEST variable 55
 - TIMESTAMP variable 55
 - value variable statement format 10
 - VALUE_STATUS variable 55
- ENUMERATED
 - EDB data type 16
 - test value data type 38
- Error messages
 - (see *Appendix B "Troubleshooting"*)
 - "No .cio file" 205
 - HGSC_E_ BADFLDCMB 205
- Errors
 - inconsistency errors 21
- Errors in DEF file 162

Index

Examples

- array values 39
- both Input/Output qualifiers specified 43
- changing access name 19
- changing test value source 41
- conventions used in 3
- data access time variables 60
- data type declarations 32
- declare data access time variables 61
- DEF file
 - DATA_PRIORITY directive 166
 - FORTTRAN 165
 - PRINT directive 170
- EDB comments 11
- EXTDEF 19
- FORTTRAN application 183
- how to read array values 37
- I/O variable 42
- inconsistency error 21
- INPUT qualifier 42
- one location spec specified 43
- OpenDDA exec_status 155
- OUTPUT qualifier 42
- point.parameter 53
- PRINT directive 177
- Read data set 7
- store code variables 57
- store status variable 59
- TERMINATE statement 142
- test value source options 41
- using arrays of test values 39
- value status variables 56
- value variable arrays 35
- view a core dump 199
- WRITE statement 148

Exec statements

- CLEAR CIO 96, 100
- HIBERNATE EVENT 97, 102
- INITIALIZE 97, 107
- LOAD 117
- LOAD ALIAS 97, 111
- LOAD MODIFY_CODE 97, 114
- LOAD STORE_CODE 97, 117
- MODIFY CIO 97, 121
- READ 97, 129
- READ CIO 97, 134
- SELECT CIO 97, 138
- TERMINATE 97, 141
- WRITE 97, 143
- WRITE CIO 97, 150

exec_status 155

EXTDEF

- rules 20
- Symbolic platform name 19

EXTDEF directive 19

External Data Block (See EDB)

External Reference Keyword 67

External Reference Status

- declaring 86

- syntax 86

External Reference Status Variables 86

EXTERNAL_ID

- EDB data type 16

- Network Gateway 18

- test value data type 38

F

FORTTRAN

- Customize the DEF file 182

- DEF file example 165

- default DEF file 182

- EDB syntax 181

- example 183

- include file 179

- programs 179

- sample applications 179

- template 180

Full Name arrays

- declaring 77

FULL_NAME 67

G

General Output file 199

Generic Status Codes

- LCN data access status codes 132, 146

H

HIBERNATE EVENT statement 97, 102

HLL 5

Host platforms 209

How to

- change test value source 41

- read array values 37

- view a core dump 199

I

I/O terminology 5

Include file 94

- C 189

- FORTTRAN 179

Inconsistency error 21

Index

INDEX_NAME 67
Indexes and Dynamic Selection 82
INITIAL_VALUES directive 167, 168
 STORE_CODE qualifier 167
Initialization of dynamic components 73
INITIALIZE statement 97
 description 107
 location 107
INPUT keyword 42
INPUT qualifiers
 both I/O specified 43
 I/O variable 42
Input/Output 5, 6
 ALL data set 7
 data transfer sets 6
 processing 6
INTEGER
 EDB data type 16
 test value data type 38

K

Keywords 13

L

LCN data access status codes 132, 146
Length of strings 33
LOAD ALIAS statement 97, 111
LOAD MODIFY_CODE statement 97, 114
LOAD statement 117
LOAD STORE_CODE statement 97, 117
 SET 117
 STATUS 118
loading Alias Variables 80
Location spec
 definition 44
 format 44
 I/O reference 43

M

MESSAGES qualifier 171
MODIFY CIO statement 97, 121
Modify Code Variables 89
 declaring 90
 default 89
 syntax 89
 validation 89

N

Network Gateway (NG) 18
 NG field 45

NOX Location Specification 46

O

OpenDDA
 basic information 2
 current version 199
 FORTRAN references 2
 Host Platform 209
 include file 94
 keywords 13
 precompiler 91
 reserved words 13
OpenDDA application
 basic information 2
 C
 default DEF file 192
 EDB syntax 191
 include file 189
 programs 189
 template 190
 FORTRAN
 DEF file example 165
 default DEF file 182
 EDB syntax 181
 example 183
 include file 179
 programs 179
 references 2
 template 180
 I/O terminology 5
 naming conflict 200
 output file 199
OpenDDA data types 17
OpenDDA exec_status
 example 156
 variable 155
OpenDDA exec_status variable 155

Index

OpenDDA Execution Statement

- CLEAR CIO 100
- HIBERNATE EVENT 102
- INITIALIZE 107
- LOAD 117
- LOAD ALIAS 111
- LOAD MODIFY_CODE 114
- LOAD STORE_CODE 117
- MODIFY CIO 121
- READ 129
- READ CIO 134
- SELECT CIO 138
- summary 96
- TERMINATE 141
- WRITE 143
- WRITE CIO 150

Optional parameters indicators 3

Options

- TEST_VALUE, NOTEST_VALUE 40

ORDINAL

- EDB data type 16
- test value data type 38

OUTPUT keyword 42

OUTPUT qualifiers

- both I/O specified 43
- I/O variable 42

P

PARAM_NAME 67

Platform restrictions

- store code constants 58

Platforms

- valid OpenDDA 209

Point.parameter

- definition 53
- field 45

POINT_ID

- EDB data type 16
- Network Gateway 18
- test value data type 38

POINT_NAME 67

Populate HLL 5

Precompiler 91

PRINT directive 170

- MESSAGES qualifier 171
- STATS qualifier 172
- VALUES qualifier 173

Problems

- (see *Appendix B "Troubleshooting"*)
- application won't write values 203
- can't access DDA executable 203
- consistency error 201
- data is not written to external data source 143
- EDB parsing error 203
- erroneous data 203
- errors returned by CL initiation 202
- no output file and bad status in CL block 201
- PRINT VALUES shows status %HGSC-E-NULL_VALUE 201
- privilege problem installing or executing 201
- reporting 199
- source file not found 204

Q

Qualifiers

- INPUT 42
- OUTPUT 42
- TEST_VALUE/NOTEST_VALUE 40

R

READ CIO statement 97, 134

Read Data 5

READ statement 97, 129

- SET 111, 121, 129

- STATUS 130

REAL

- EDB data type 16
- test value data type 38

Remapping CIO

- concept 28
- tools 28

Reporting a problem 199

Reserved words 13

Rules

- DEF file 161
- EXTDEF 20
- for EDB 12
- SET_LIST directive 63
- value variable 32

S

Security switch settings 203

SELECT CIO statement 97, 138

SET_LIST directive

- definition 63
- rules 63
- syntax 63

Index

Sets

- ALL data set 7
- Data Access Sets 25
- Dynamic Selection Sets 25

Software version number 199

STATS qualifier 172

Store code constants 57

Store code defaults 58

Store code variable 57

- defaults 58

Store status variable 59

STORE_CODE

- EDB ancillary variable 55

STORE_CODE qualifier 167

STORE_STATUS

- EDB ancillary variable 55

STRING

- EDB data type 16

- test value data type 38

- truncation 18

String length 33

Subrange

- format 33

- specification 33

Summary

- exec statements 96

Symbolic platform name 19

Syntax

- EDB 14

T

TERMINATE statement 97, 141

- example 142

- STATUS 141

TEST

- EDB ancillary variable 55

Test data 39

Test value

- data types 38

- definition 38

- format 38

- max. values 38

- not enough values 39

- options 41

- overriding the EDB 41

Test Value Variable 62

TEST_VALUE/NOTEST_VALUE qualifier 40

TIMESTAMP

- EDB ancillary variable 55

Troubleshooting OpenDDA (*see Appendix B*

"Troubleshooting")

Truncation of string data 18

types, data 17

U

UNIX

- file naming conflict 200

Upper/lowercase

- as used in this guide 3

V

Value status variable

- accessing 56

- declaring 56

Value variable

- arrays 33, 34

- changing test value source 41

- data type 40

- declarations 32

- definition 32

- options 40

- rules 32

- statement format 10

- strings 33

- subrange specification 33

VALUE_STATUS

- EDB ancillary variable 55

VALUES qualifier 173

Version number 199

View a core dump 199

W

With 189

WRITE CIO statement 97, 150

Write Data 5

WRITE statement 97, 143

- example 148

- STATUS 101, 104, 108, 112, 115, 122, 135, 139, 144, 151

Writing data 143

Writing External IDs in C 18, 149

READER COMMENTS

Honeywell's IAC Automation College welcomes your comments and suggestions to improve future editions of this and other publications.

You can communicate your thoughts to us either by **mail** using this form, or a **toll-free telephone** call.

BY TELEPHONE: In the U.S.A. use our toll-free number 1*800-822-7673 (available in the 48 contiguous states except Arizona; in Arizona dial 1-602-313-5558).

BY MAIL: Use this form; detach, fold, tape closed, and mail to us. We would like to acknowledge your comments; please include your complete name and address.

Title of Publication: **OpenDDA Reference Guide**

Issue Date: **12/95**

Publication Number: **DD27-200**

Writer: **J Kennedy**

COMMENTS: _____

RECOMMENDATIONS: _____

NAME _____ DATE _____

TITLE _____

COMPANY _____

ADDRESS _____

CITY _____ STATE _____ ZIP _____

(If returning by mail, please tape closed; Postal regulations prohibit use of staples.)

Honeywell

Industrial Automation and Control
Honeywell Inc.
16404 North Black Canyon Highway
Phoenix, Arizona 85023-3033

Helping You Control Your World